


# GanadoTrack

Sistema web de gestión de feedlot bovino

**Documentación del Proyecto Final de Software**

**Institución:** IFES UCASAL
**Carrera:** Tecnicatura Superior en Análisis de Sistemas
**Materia:** Práctica Profesionalizante II
**Alumno:** Mateo Agustín Sgüerzo – Legajo N.° 4261
**Docente:** Silvia Hoferek
**Versión del sistema:** ganadotrack-1.1.1.1
**Fecha:** Noviembre de 2026

# Índice

1. Nombre del proyecto fundamentado ..... 5
2. Logo ..... 5
3. Link a código / aplicación ..... 5
4. Breve descripción ..... 6
5. Objetivo principal ..... 6
6. Objetivos específicos ..... 6
7. Límites ..... 7
8. Alcance ..... 8
8.1 Usuarios y roles ..... 8
9. Análisis del caso y herramientas utilizadas ..... 10
9.1 Descripción del caso ..... 10
9.2 Caso de pérdida por falta de información sanitaria ..... 10
9.3 Herramientas de relevamiento utilizadas ..... 10
9.4 Principales hallazgos del relevamiento ..... 11
10. Requerimientos funcionales ..... 12
11. Requerimientos no funcionales ..... 16
12. Modelos de procesos de negocio (BPMN) ..... 17
12.1 Procesos automatizados ..... 17
12.2 Modelos de los procesos ampliados y nuevos ..... 17
12.3 Procesos que se mantienen de la 1ª entrega ..... 20
13. Casos de uso ..... 22
13.1 Actores ..... 22
13.2 Lista de casos de uso ..... 22
13.3 Diagrama de casos de uso ..... 24
13.4 Fichas de casos de uso ..... 25
14. Arquitectura y herramientas de desarrollo ..... 34
14.1 Tipo de arquitectura ..... 34
14.2 Diagrama de arquitectura ..... 35
14.3 Capas y componentes del backend ..... 35
14.4 Comunicación entre frontend, backend y base de datos ..... 37
14.5 Diseño de la API ..... 38
14.6 Autenticación y autorización ..... 39
14.7 Consideraciones de seguridad ..... 39
14.8 Manejo de errores y validación de datos ..... 40
14.9 Escalabilidad y mantenimiento ..... 40
14.10 Atributos de calidad ..... 41
14.11 Herramientas y tecnologías ..... 42
14.12 Despliegue ..... 46
15. Modelo de datos ..... 49

15.1 Modelo conceptual: entidades principales ..... 49
15.2 Modelo lógico: diagrama entidad-relación ..... 50
15.3 Normalización ..... 53
15.4 Diccionario de datos ..... 54
15.5 Modelo físico: implementación en PostgreSQL ..... 58
15.6 Consultas de ejemplo ..... 59
16. Módulos implementados ..... 61
16.1 Integración entre módulos ..... 62
16.2 Reglas de negocio relevantes ..... 62
17. Descripción de la API ..... 63
17.1 Endpoints ..... 63
17.2 Ejemplo: ración sugerida para el encargado (CU16) ..... 65
17.3 Ejemplo: registrar alimentación (CU04 / CU21) ..... 65
17.4 Ejemplo: clima y riesgos (CU19) ..... 66
17.5 Ejemplo: indicadores de un lote (`GET /api/lotes/2`) ..... 66
17.6 Códigos de error ..... 66
18. Estrategia de pruebas y validación ..... 68
18.1 Estrategia ..... 68
18.2 Casos de prueba ..... 69
18.3 Incidencias detectadas y corregidas durante el desarrollo ..... 78
18.4 Matriz de trazabilidad ..... 79
18.5 Validación de aceptación ..... 80
18.6 Análisis de resultados y mejoras identificadas ..... 80
19. Seguimiento del proyecto (GitHub Projects) ..... 82
19.1 Herramientas ..... 82
19.2 Organización de las tareas ..... 82
19.3 Flujo de ramas ..... 84
19.4 Convención de commits (trazabilidad) ..... 84
19.5 Versionado ..... 85
19.6 Checklist de revisión de cada pull request ..... 85
19.7 Evidencias ..... 86
20. Resultados obtenidos y conclusiones ..... 92
20.1 Producto desarrollado ..... 92
20.2 Cumplimiento de los requerimientos no funcionales ..... 93
20.3 Métricas del proyecto ..... 93
20.4 Respuesta a los problemas del relevamiento ..... 93
20.5 Conclusiones ..... 94
21. Trabajo futuro ..... 95
21.1 Corto plazo (versión 1.1.3.x) ..... 95
21.2 Mediano plazo (versión 1.3.x.x) ..... 95
21.3 Largo plazo (versión 2.x) ..... 95

21.4 Mejoras técnicas ... 96
Anexo I: Documentación de relevamiento ... 97
Datos generales de la entrevista ... 97
Bloque 0: Tipo de operación y especie ... 97
Bloque 1: Contexto del negocio ... 97
Bloque 2: Compra e ingreso de animales ... 97
Bloque 3: Alimentación ... 98
Bloque 4: Seguimiento del crecimiento ... 98
Bloque 5: Sanidad ... 99
Bloque 6: Venta y rentabilidad ... 99
Bloque 7: Gestión actual y dolor real ... 99
Bloque 8: Tecnología y expectativas ... 100
Cierre: definición del alcance del MVP ... 101
Anexo II: Manual de instalación y configuración ... 102
Requisitos ... 102
Instalación local paso a paso ... 102
Verificar la instalación ... 103
Problemas frecuentes ... 104
Respaldo y restauración de la base ... 104
Despliegue online ... 104
Anexo III: Manual de usuario ... 106
1. Ingresar al sistema ... 106
2. Encargado: la comida de hoy ... 107
3. Veterinario ... 109
4. Administrador ... 113
5. Mi perfil ... 122
6. Instalar la app en el celular ... 123
7. Uso en el celular ... 123
8. Problemas frecuentes ... 124
Anexo IV: Material para la validación con el productor ... 125
Cuestionario (8 preguntas) ... 125
Anexo V: Correcciones respecto de la 1ª entrega ... 126

# 1. Nombre del proyecto fundamentado

**GanadoTrack**

El nombre combina dos conceptos directos:

- **Ganado:** hace referencia explícita al sector productivo al que está dirigido el sistema, la ganadería bovina, actividad estratégica de la Argentina y núcleo de la operación del cliente.
- **Track** (del inglés *to track*: rastrear, seguir, monitorear): representa la función central del sistema, el seguimiento continuo de animales, alimentación, sanidad y costos a lo largo del ciclo de feedlot.

El resultado es un nombre corto, memorable y bilingüe (común en el sector agropecuario tecnológico), que comunica de forma inmediata el propósito de la herramienta: seguimiento ganadero digital.

# 2. Logo




***Figura 1.*** *Logo de GanadoTrack*

El logo combina la silueta de un bovino con una línea de seguimiento, en los colores de la identidad visual del sistema: verde bosque, verde hoja y tonos tierra. Los mismos colores se usan en la interfaz (tema "Campo cálido"), en los informes PDF y en los diagramas de este documento.

# 3. Link a código / aplicación

***Tabla 1.*** *Link a código / aplicación*

| Recurso | Dirección |
|---|---|
| **Repositorio del código fuente** | https://github.com/mateogit114730/ganadotrack |
| **Tablero del proyecto** (GitHub Projects) | https://github.com/users/mateogit114730/projects/1 |
| **Integración continua** (GitHub Actions) | https://github.com/mateogit114730/ganadotrack/actions |
| **Aplicación online** | https://ganadotrack.vercel.app |
| **API** | https://ganadotrack-api.onrender.com/api/health (estado) |
| **Documentación de la API** (Swagger UI) | https://ganadotrack-api.onrender.com/api/docs |

- La aplicación se puede abrir en el navegador o **instalar en el celular** (ver el Anexo III, sección 6).

- La primera visita después de un rato sin uso puede tardar unos 50 segundos, mientras el servidor gratuito "despierta".
- Para ejecutar el sistema localmente, seguir el README del repositorio o el manual de instalación (Anexo II).

**Usuarios de demostración:** `admin@ganadotrack.local`, `encargado@ganadotrack.local` y `veterinario@ganadotrack.local`. La contraseña se informa por separado a la docente; no se publica en el repositorio.

# 4. Breve descripción

GanadoTrack es un sistema web de gestión para productores ganaderos que operan feedlots bovinos. Permite:

- Registrar los lotes de animales y su evolución de peso, con la Ganancia Diaria de Peso (GDP) y la proyección al peso objetivo.
- Controlar la alimentación diaria a partir de la ración de cada lote, el stock de insumos y su proyección.
- Gestionar la sanidad con un calendario armado automáticamente por protocolos y con alertas.
- Calcular el costo de producción por kilogramo de carne y el margen de cada lote.
- Anticipar riesgos a partir del clima de la zona y emitir informes en PDF y Excel.

Está diseñado para tres tipos de usuarios: el **administrador** (productor), el **encargado** de campo y el **veterinario**, cada uno con una interfaz adaptada a su rol. Se usa desde el navegador o instalado en el celular, y el encargado puede cargar la alimentación aunque no haya señal en el corral.

# 5. Objetivo principal

Desarrollar un sistema web que permita a productores ganaderos con operaciones de feedlot conocer en tiempo real el costo de producción por kilogramo de carne, la evolución de peso de sus lotes y el estado sanitario de sus animales, eliminando el registro manual en papel y reduciendo las pérdidas económicas causadas por falta de información oportuna.

# 6. Objetivos específicos

1. Registrar el ingreso de lotes al feedlot con peso promedio inicial, peso objetivo de salida y GDP objetivo.
2. Permitir al encargado registrar diariamente los kilogramos de alimento suministrados por lote, desde una interfaz simple y accesible desde el celular.
3. Calcular automáticamente la Ganancia Diaria de Peso (GDP) a partir de las pesadas mensuales y proyectar la fecha estimada de llegada al peso objetivo.
4. Generar alertas automáticas para eventos sanitarios programados (vacunaciones, tratamientos antiparasitarios) antes de su vencimiento.
5. Registrar eventos sanitarios y bajas de animales con sus causas, permitiendo al veterinario cargar esta información directamente en el sistema.
6. Calcular el costo acumulado por lote (alimentación + sanidad) y el costo de producción por kilogramo de carne, considerando el rendimiento al gancho (57 %, configurable).
7. Controlar el stock de insumos (suplemento proteico y silaje de sorgo) con alertas de nivel mínimo.

8. Proveer un dashboard visual con el estado actual de todos los lotes activos y las alertas pendientes.
9. Precargar la ración diaria de cada lote para que el encargado la confirme con un solo toque, y calcular la conversión alimenticia del engorde.
10. Generar automáticamente el calendario sanitario de cada tropa a partir de un protocolo definido por el veterinario.
11. Anticipar el quiebre de stock de los insumos y sugerir la compra necesaria, en especial ante un riesgo de sequía.
12. Informar el clima del establecimiento y los riesgos asociados para el ganado (estrés térmico, garrapata, sequía).
13. Emitir informes en PDF y una planilla Excel para el productor y su contador.
14. Permitir la carga de la alimentación en zonas del campo sin señal y su envío posterior sin duplicados.

Los objetivos 1 a 8 corresponden a la 1ª entrega (MVP); los objetivos 9 a 14 se agregaron en la versión final, a partir de problemas planteados en el relevamiento: la ración la define el veterinario, la sequía obliga a compras de urgencia, la garrapata es la principal amenaza sanitaria y hay sectores del campo sin señal.

# 7. Límites

El sistema **no** contempla:

- **Gestión del ciclo completo** (cría, parición, recría, pastoreo): estas etapas quedan fuera del alcance por la complejidad de las variables involucradas, tal como se acordó con el productor.
- **Identificación individual de animales por caravana:** el sistema trabaja exclusivamente por lote (tropa).
- **Integración directa con los sistemas del SENASA** (SIGSA, SIGAD): el sistema es complementario a la trazabilidad oficial, no la reemplaza.
- **Facturación o gestión contable:** los datos fiscales e impositivos siguen siendo responsabilidad del contador externo, que recibe una planilla Excel con los datos operativos.
- **Gestión de recursos humanos** (sueldos, legajos de empleados).
- **Gestión de maquinaria o vehículos** del campo.
- **Otras especies animales** (porcinos, ovinos, aves).
- **Aplicación móvil nativa (iOS/Android):** no se desarrolla. El sistema es una aplicación web *instalable* (PWA): se agrega a la pantalla de inicio del celular desde el navegador, sin pasar por las tiendas de aplicaciones.
- **Funcionamiento sin conexión de todos los módulos:** solo se contempla para la carga de la alimentación diaria del encargado, que es la tarea que se hace en el corral. Las demás pantallas trabajan con datos compartidos que deben estar actualizados; sin señal, el sistema lo indica con un aviso y no admite cambios hasta recuperarla.
- **Costo del animal al ingreso:** el costo calculado corresponde a la etapa de engorde (alimentación + sanidad). No incluye el costo de cría/recría del animal.
- **Gastos generales del establecimiento** (energía eléctrica, combustible, arrendamiento, mano de obra): quedan fuera de esta versión; se prevé incorporarlos como "otros gastos por lote" (ver Trabajo futuro).
- **Notificaciones fuera del sistema** (email, WhatsApp, notificaciones *push*): las alertas se consultan dentro del sistema.

- **Clima:** el pronóstico proviene de un servicio público (Open-Meteo). Los índices de riesgo son **orientativos** y no reemplazan el criterio del veterinario.

# 8. Alcance

La versión final de GanadoTrack cubre los siguientes módulos. Todos los importes se expresan en **pesos argentinos (ARS)**.

***Tabla 2.*** *Módulos y funcionalidades incluidas en la versión final*

| Módulo | Funcionalidades incluidas |
|---|---|
| Gestión de lotes | Crear, editar y cerrar lotes. Fecha de ingreso, cantidad de animales, peso de entrada, peso objetivo y GDP objetivo. Protocolo sanitario al crear el lote. Cierre con resumen final |
| Alimentación | Registro diario de los kg entregados por el encargado. Cálculo automático del costo diario. Corrección de registros cargados por error |
| Ración y conversión | Ración por lote en kg por cabeza por día, con historial de cambios. Carga sugerida para el encargado (confirmación en un toque). Conversión alimenticia en materia seca entre pesadas |
| Stock de insumos | Control de suplemento comprado y silaje de sorgo propio. Registro y corrección de compras. Alertas de stock mínimo |
| Proyección de stock | Consumo diario previsto, días de cobertura, fecha de quiebre, compra sugerida y alerta anticipada |
| Pesadas | Registro de pesadas mensuales. Cálculo automático de la GDP. Proyección de la fecha de llegada al peso objetivo. Alerta de GDP baja |
| Sanidad | Calendario sanitario por lote. Alertas previas a los vencimientos. Registro de tratamientos de garrapata. Registro de bajas con causa. Costo sanitario acumulado |
| Protocolos sanitarios | Plantillas de calendario sanitario editables por el veterinario. Generación automática de los eventos al crear un lote o al aplicarlas a un lote existente |
| Costos | Costo acumulado por lote. Costo de engorde por kg de carne (rendimiento al gancho configurable, 57 %). Margen bruto estimado de engorde |
| Clima y riesgos | Clima actual y pronóstico de 7 días de la localidad del establecimiento. Índice de estrés térmico y riesgos de garrapata y de sequía, con alertas |
| Dashboard | Vista general de los lotes activos, alertas pendientes, clima y resumen financiero |
| Reportes | Informe del lote (y de cierre) en PDF, reporte mensual en PDF y planilla Excel para el contador |
| App instalable y sin conexión | Instalación en el celular. Carga de alimentación sin señal, con envío automático al recuperar la conexión |
| Usuarios y roles | 3 roles: Administrador, Encargado y Veterinario, con acceso diferenciado. Perfil propio (nombre y contraseña) |

## 8.1 Usuarios y roles

El propio productor definió los tres roles durante el relevamiento (Anexo I, Bloque 8).

***Tabla 3.*** *Usuarios del sistema, responsabilidades, permisos y nivel de interacción*

| Usuario | Responsabilidades | Permisos en el sistema | Nivel de interacción |
|---|---|---|---|
| **Administrador** (productor) | Toma las decisiones productivas y comerciales; compra insumos; decide el cierre de los lotes | Acceso completo: dashboard, lotes, pesadas, costos, insumos, reportes, usuarios y configuración | Alto: consulta indicadores y reportes a diario desde la PC o el celular |
| **Encargado** | Prepara y entrega la ración diaria de cada lote | Solo la carga de la alimentación diaria (también sin conexión) y su perfil | Bajo y frecuente: uno o dos toques por lote cada día, desde el celular en el corral |

| Usuario | Responsabilidades | Permisos en el sistema | Nivel de interacción |
|---|---|---|---|
| **Veterinario** | Define el plan sanitario y la dieta; aplica tratamientos; registra las bajas | Sanidad (eventos, tratamientos, bajas), protocolos sanitarios, raciones, alertas sanitarias y su perfil | Medio: carga eventos en cada visita y define protocolos y raciones |

Además intervienen dos actores no humanos: el **Sistema** (proceso automático que genera las alertas) y el **servicio de clima** (Open-Meteo), descriptos en la sección 13.1.

# 9. Análisis del caso y herramientas utilizadas

## 9.1 Descripción del caso

El relevamiento se realizó sobre un establecimiento ganadero de ciclo completo ubicado en la región pampeana argentina. La operación comprende aproximadamente 450–600 cabezas de ganado bovino raza Angus, sobre 500 hectáreas de campo arrendado. El productor realiza todas las etapas del ciclo productivo: cría, recría, engorde en feedlot y comercialización a través de su propia carnicería.

Al momento del relevamiento, el establecimiento no contaba con ningún sistema de registro ni gestión administrativa. La información se manejaba de forma verbal o en papel, sin sistematización. El productor declaró desconocer su costo de producción por kilogramo de carne y no contar con seguimiento sanitario estructurado.

## 9.2 Caso de pérdida por falta de información sanitaria

Durante la entrevista, el productor describió un incidente crítico que evidencia la necesidad del sistema:

> *"La garrapata transmite una enfermedad llamada Anaplasmosis. Por falta de seguimiento sanitario y control de los tratamientos antiparasitarios, el sistema inmune de los animales colapsó. Perdimos 26 vacas."*

Estimando un valor promedio de mercado de USD 1.000 por cabeza (estimación de mercado del primer semestre de 2026), la pérdida representó aproximadamente USD 26.000, una cifra que podría haberse reducido significativamente con un sistema de alertas sanitarias y seguimiento de tratamientos.

Este caso constituye el principal argumento de negocio que justifica el desarrollo de GanadoTrack.

## 9.3 Herramientas de relevamiento utilizadas

**Entrevista semiestructurada:** se realizó una entrevista al productor y dueño del establecimiento mediante un cuestionario organizado en **9 bloques temáticos (0 a 8)**:

- Bloque 0: Tipo de operación y especie
- Bloque 1: Contexto del negocio
- Bloque 2: Compra e ingreso de animales
- Bloque 3: Alimentación
- Bloque 4: Seguimiento del crecimiento
- Bloque 5: Sanidad animal
- Bloque 6: Venta y rentabilidad
- Bloque 7: Gestión actual y dolor real
- Bloque 8: Tecnología y expectativas

**Mensajes de voz (WhatsApp):** el cierre del relevamiento se realizó mediante audios de WhatsApp, en los que el productor confirmó el foco del sistema en el feedlot y la necesidad de alertas sanitarias programadas. La transcripción completa está en el Anexo I.

## 9.4 Principales hallazgos del relevamiento

***Tabla 4.*** *Hallazgos del relevamiento y su impacto en el sistema*

| Hallazgo | Impacto en el sistema |
|---|---|
| No hay ningún sistema de registro hoy | No existe un sistema previo: no hay datos que migrar ni herramientas que reemplazar, por lo que el sistema puede diseñarse desde cero según el proceso real del establecimiento |
| Desconoce su costo de producción | El módulo de costos es la funcionalidad de mayor valor |
| 26 vacas muertas por falta de control sanitario | El módulo de alertas sanitarias es prioritario; en la versión final, además, el calendario se arma solo con protocolos y el clima anticipa el riesgo de garrapata |
| Trabaja por lote, no por animal individual | El sistema gestiona tropas (lotes), no individuos |
| Tiene Starlink y señal de celular, pero no en todo el campo | Aplicación web responsive; en la versión final, la carga de alimentación funciona también sin señal (RF14) |
| El veterinario se comunica por WhatsApp y define la ración | El veterinario es usuario del sistema: carga los eventos sanitarios, define los protocolos y la ración de cada lote |
| Produce su propio silaje de sorgo | El silaje se registra como insumo con su costo por kg; las cargas de silaje propio suman stock como una compra |
| Compra el suplemento proteico semanalmente | Cada compra genera un stock de corto plazo que se consume durante la semana. Registrar ese stock permite anticipar la próxima compra y conocer el precio vigente para calcular el costo diario |
| En las sequías debe comprar alimento de urgencia | Proyección de stock con compra sugerida y alerta de riesgo de sequía (RF11 y RF12) |
| Vende en su carnicería propia | La unidad de cálculo es el kg al gancho (rendimiento 57 %, configurable) |

# 10. Requerimientos funcionales

Los requerimientos RF01 a RF08 provienen de la 1ª entrega, corregidos y ampliados. Los RF09 a RF14 se agregaron en la versión final.

**RF01: Gestión de lotes**

- El sistema debe permitir crear un nuevo lote ingresando: nombre o número de lote, fecha de ingreso, cantidad de animales, peso promedio de entrada (kg), peso objetivo de salida (kg) y **GDP objetivo (kg/día)**.
- Al crear el lote se puede elegir un protocolo sanitario (RF10).
- El sistema debe mostrar automáticamente los días transcurridos desde el ingreso del lote.
- El sistema debe permitir registrar el cierre de un lote al momento de la venta o faena, con un resumen final. Un lote cerrado no admite nuevas cargas.

**RF02: Registro de alimentación diaria**

- El sistema debe permitir al encargado seleccionar un lote y registrar los kg de suplemento proteico y de silaje de sorgo dados en el día.
- El sistema debe calcular automáticamente el costo diario de alimentación del lote (kg × precio por kg de cada insumo).
- El sistema debe acumular el costo total de alimentación por lote desde su ingreso.

**RF03: Control de stock de insumos**

- El sistema debe mantener el stock actual de suplemento proteico y de silaje de sorgo. Internamente ambos se registran en **kg**; el silaje se muestra además en toneladas para facilitar la lectura.
- El sistema debe descontar del stock la cantidad registrada en cada alimentación diaria.
- El sistema debe permitir registrar nuevas compras o cargas de insumos. El administrador puede corregir una compra registrada; el stock y el precio se ajustan en la misma operación y el stock nunca queda negativo.
- El sistema debe emitir una alerta cuando el stock de un insumo caiga por debajo del nivel mínimo configurado.
- El sistema debe registrar el porcentaje de materia seca de cada insumo (RF09).

**RF04: Registro de pesadas y cálculo de GDP**

- El sistema debe permitir registrar una pesada mensual por lote: fecha y peso promedio del lote (kg).
- El sistema debe calcular automáticamente la Ganancia Diaria de Peso entre pesadas: GDP = (peso actual − peso anterior) / días transcurridos.
- El sistema debe proyectar la fecha estimada de llegada al peso objetivo según la GDP actual.
- El sistema debe emitir una alerta cuando la GDP del lote sea inferior a la GDP objetivo del lote.
- Se puede eliminar la **última** pesada de un lote si se cargó por error; la GDP de las anteriores no cambia. Cada pesada muestra la conversión alimenticia (RF09).

**RF05: Gestión sanitaria y alertas**

- El sistema debe permitir crear un calendario sanitario por lote con eventos programados (nombre del evento, fecha, producto a aplicar).

- El sistema debe emitir una alerta automática 7 días antes de la fecha de cada evento sanitario programado (configurable) y otra cuando el evento esté vencido.
- El sistema debe permitir al veterinario registrar la ejecución de un evento sanitario: fecha real, producto usado, dosis y costo total.
- El sistema debe permitir registrar tratamientos de garrapata/anaplasmosis: fecha de inicio, producto, duración del tratamiento y costo; al registrarlo, programa el próximo control.
- El sistema debe permitir registrar bajas (muertes) en el lote: fecha, cantidad de animales y causa (enfermedad, accidente, sequía u otra).
- El sistema debe mantener un acumulado del costo sanitario total por lote.
- Un evento pendiente se puede reprogramar o corregir; uno realizado, no. El administrador puede eliminar una baja cargada por error y las cabezas del lote se restituyen.

**RF06: Cálculo de costos y rentabilidad**

- El sistema debe calcular el costo acumulado total por lote: alimentación + sanidad.
- El sistema debe calcular las **cabezas actuales** del lote: cantidad de ingreso − bajas registradas.
- El sistema debe calcular el **costo de engorde por kg de carne**: costo total acumulado / (peso promedio actual × cabezas actuales × rendimiento al gancho). El rendimiento al gancho es un parámetro configurable (valor inicial 57 %).
- El sistema debe permitir al administrador ingresar el precio de venta por kg en la carnicería.
- El sistema debe mostrar el **margen bruto estimado de engorde** por lote: (precio de venta por kg × kg de carne estimados) − costo total acumulado.

**RF07: Dashboard y reportes**

- El sistema debe mostrar un dashboard con todos los lotes activos, indicando para cada uno: días en feedlot, peso promedio actual, GDP de la última medición, costo acumulado y alertas pendientes.
- El sistema debe mostrar en el dashboard un resumen de las alertas activas (sanitarias, de stock, de stock proyectado y de clima) y el clima con sus riesgos (RF11 y RF12).
- El sistema debe permitir ver el historial de un lote cerrado con su resumen financiero final.

**RF08: Gestión de usuarios y roles**

- El sistema debe soportar tres roles de usuario: Administrador, Encargado y Veterinario.
- El administrador tiene acceso completo a todos los módulos y gestiona los usuarios.
- El encargado solo puede acceder al módulo de registro de alimentación diaria.
- El veterinario solo puede acceder a los módulos de sanidad (eventos, ejecuciones y bajas), protocolos sanitarios y raciones.
- El sistema debe requerir autenticación con usuario y contraseña para todos los roles.
- Cada usuario puede cambiar su nombre y su contraseña (con verificación de la contraseña actual) desde "Mi perfil".

**RF09: Ración por lote y conversión alimenticia**

- El sistema debe permitir al veterinario o al administrador definir la ración de cada lote: kg por cabeza por día de cada insumo y la fecha desde la que rige.
- El sistema debe conservar el historial de raciones; un cambio con fecha futura no modifica la ración vigente hoy. Cargar 0 kg quita el insumo de la ración.

- El sistema debe calcular los kg del día de cada insumo: kg por cabeza por día × cabezas actuales del lote.
- El sistema debe mostrar al encargado la ración del día de cada lote y permitirle registrarla con un solo toque. También debe poder informar otros kilos si entregó una cantidad distinta.
- El sistema debe registrar el porcentaje de materia seca de cada insumo (configurable; valores iniciales: suplemento 90 %, silaje 35 %).
- El sistema debe calcular la conversión alimenticia entre pesadas y acumulada del lote: kg de materia seca consumidos / (aumento de peso promedio × cabezas).

**RF10: Protocolos sanitarios**

- El sistema debe permitir al veterinario crear, modificar y eliminar protocolos sanitarios. Cada protocolo es una lista de eventos (tipo, nombre, producto, día desde el ingreso) que pueden repetirse cada cierta cantidad de días.
- El sistema debe permitir marcar un protocolo como **por defecto**; solo uno puede serlo a la vez.
- Al crear un lote, el sistema debe generar los eventos sanitarios pendientes del protocolo elegido (fecha = ingreso + día del evento), en la misma operación que crea el lote.
- Si el lote se carga con fecha de ingreso anterior a hoy, el sistema debe permitir omitir los eventos con fecha ya pasada.
- El sistema debe permitir aplicar un protocolo a un lote ya existente.

**RF11: Proyección de stock de insumos**

- El sistema debe estimar el consumo diario de cada insumo como la suma de las raciones vigentes × cabezas actuales. Si no hay raciones definidas, debe usar el consumo real promedio de los últimos 14 días.
- El sistema debe calcular los días que alcanza el stock (cobertura), la fecha estimada de quiebre y la compra sugerida para cubrir un horizonte configurable (14 días) y terminar con el stock mínimo.
- El sistema debe emitir una alerta cuando la cobertura de un insumo sea menor a la mínima configurada (7 días).
- El sistema debe mostrar el stock proyectado a 30 días en un gráfico, junto con el stock mínimo.

**RF12: Clima y riesgos**

- El sistema debe permitir al administrador configurar la ubicación del establecimiento buscando la localidad por nombre.
- El sistema debe mostrar a todos los roles el clima actual. Al administrador le debe mostrar además el pronóstico de 7 días y la lluvia de los últimos 30 días.
- El sistema debe calcular tres índices orientativos:
    - **Índice de temperatura y humedad (ITH):** estrés térmico del ganado.
    - **Riesgo de garrapata:** según la temperatura y la humedad media de la semana.
    - **Riesgo de sequía:** según la lluvia de los últimos 30 días y la pronosticada para la semana.
- El sistema debe emitir alertas de clima:
    - Estrés térmico en nivel de peligro o superior.
    - Riesgo alto de garrapata en lotes sin tratamiento en los últimos 21 días.
    - Riesgo alto de sequía, junto con los días que alcanza el silaje.

- Si el servicio de clima no responde, el sistema debe seguir funcionando sin la información de clima.

**RF13: Reportes descargables**

- El sistema debe generar un **informe del lote en PDF** con:
  - Datos del lote e indicadores.
  - Tabla y gráfico de pesadas.
  - Alimentación por insumo, sanidad, bajas, costos, conversión alimenticia y margen.

  Para un lote cerrado, funciona como **informe de cierre**.
- El sistema debe generar un **reporte mensual en PDF** con los costos del mes por lote e insumo, los eventos sanitarios, las bajas y el stock.
- El sistema debe generar una **planilla Excel** para un rango de fechas, con una hoja por tipo de dato (lotes, alimentación, compras, sanidad, pesadas y bajas) y formatos de moneda y fecha.
- Los reportes solo están disponibles para el administrador.

**RF14: Aplicación instalable y carga sin conexión**

- El sistema debe poder instalarse en el celular o la PC desde el navegador y abrirse como una aplicación.
- Si no hay conexión al confirmar una carga de alimentación, el sistema debe guardarla en el dispositivo con la **fecha de captura** e informarlo al encargado.
- Al recuperar la conexión, el sistema debe enviar automáticamente las cargas guardadas, en orden.
- Un reenvío de la misma carga no debe duplicar registros ni descontar el stock dos veces.
- Si una carga guardada es rechazada (por ejemplo, por falta de stock), el sistema debe mostrarla con el motivo y permitir descartarla.

# 11. Requerimientos no funcionales

**RNF01: Usabilidad**

- La interfaz del encargado debe permitir registrar la alimentación diaria de un lote en no más de 3 clics desde el inicio de sesión. Con la ración definida, el registro se hace con **un toque** por lote.
- El sistema debe ser operable desde celulares y tablets mediante el navegador (diseño responsive).
- La pantalla del encargado tiene botones de al menos 48 px de alto y un "modo pleno sol" de alto contraste y letra grande.
- El sistema no debe requerir capacitación técnica previa para ser usado por el encargado o el veterinario.

**RNF02: Disponibilidad**

- El sistema debe estar disponible al menos el 95 % del tiempo (considerando la dependencia de la conexión Starlink del establecimiento).
- El sistema se publica en plataformas gestionadas con HTTPS (Vercel, Render y Neon).
- Si falla un servicio externo (clima), el resto del sistema sigue funcionando.

**RNF03: Rendimiento**

- El dashboard principal debe cargar en menos de 3 segundos con hasta 20 lotes activos simultáneos.
- El registro de alimentación diaria debe confirmarse en menos de 2 segundos.

**RNF04: Seguridad**

- Las contraseñas deben almacenarse hasheadas (bcrypt).
- La sesión debe expirar automáticamente tras 8 horas de inactividad.
- Cada rol debe tener acceso estrictamente restringido a sus funcionalidades (RBAC), verificado en el servidor.
- La comunicación entre cliente y servidor debe realizarse mediante HTTPS.

**RNF05: Escalabilidad**

- La arquitectura debe permitir agregar nuevos módulos (ciclo completo, identificación individual) sin rediseñar la base existente.
- La base de datos debe soportar el crecimiento histórico de lotes y registros sin degradación del rendimiento.

**RNF06: Mantenibilidad**

- El código debe estar documentado con comentarios en las funciones de cálculo (GDP, costo por kg, proyecciones).
- El proyecto debe seguir una estructura de carpetas estándar para React (frontend) y Node.js (backend).

**RNF07: Operación sin conexión**

- La carga de la alimentación diaria debe poder realizarse sin conexión a internet y enviarse automáticamente al recuperarla.
- El envío posterior debe ser **idempotente**: si se repite, no debe generar registros duplicados.

# 12. Modelos de procesos de negocio (BPMN)

## 12.1 Procesos automatizados

La tabla resume qué procesos del establecimiento automatiza el sistema, cómo se hacían antes según el relevamiento y qué modelo BPMN los representa. Los procesos 1 a 4 ya estaban modelados en la 1ª entrega; en la versión final se amplían y se suman los procesos 5 y 6.

***Tabla 5.*** *Procesos automatizados*

| # | Proceso | Antes (relevamiento) | Con GanadoTrack | RF / CU | Modelo BPMN |
|---|---|---|---|---|---|
| 1 | Ingreso de un lote | Datos anotados en papel; el calendario sanitario dependía de la memoria y de mensajes con el veterinario | Alta del lote con validación de pesos. El protocolo sanitario genera los eventos del calendario en la misma operación | RF01, RF10 · CU03, CU17 | Proceso 1, ampliado con la tarea "Generar eventos del protocolo" |
| 2 | Alimentación diaria | Kilos anotados en papel o informados de palabra; stock y costo sin calcular | Ración precargada que el encargado confirma en un toque. El sistema descuenta el stock y calcula el costo del día. Sin señal, la carga queda en el teléfono y se envía sola | RF02, RF03, RF09, RF14 · CU04, CU05, CU16, CU21 | Proceso 2, ampliado con la compuerta "¿Hay señal?" |
| 3 | Pesada mensual | Pesadas anotadas sin análisis | GDP, proyección al peso objetivo, conversión alimenticia y alerta de bajo rendimiento | RF04, RF09 · CU07 | Proceso 3 |
| 4 | Control sanitario | Coordinación por WhatsApp, sin avisos previos (pérdida de 26 vacas por anaplasmosis) | Alertas 7 días antes y de eventos vencidos, registro del evento, nuevo control de garrapata a los días del tratamiento y alerta de riesgo de garrapata según el clima | RF05, RF12 · CU08, CU09, CU14, CU19 | Proceso 4 |
| 5 | Abastecimiento de insumos | Compras sin anticipación; faltantes en épocas de sequía | Proyección de consumo, días de cobertura, fecha de quiebre, compra sugerida y alerta anticipada | RF03, RF11 · CU06, CU18 | Proceso 5 (nuevo) |
| 6 | Cierre del lote e informes | El productor no conocía su costo por kg ni el margen del lote | Cierre con resumen final, costo de engorde por kg y margen, informe PDF y planilla Excel para el contador | RF01, RF06, RF13 · CU11, CU13, CU20 | Proceso 6 (nuevo) |

## 12.2 Modelos de los procesos ampliados y nuevos

Los procesos 3 (pesada mensual) y 4 (evento sanitario y alerta) se mantienen como en la 1ª entrega. Los procesos 1 y 2 se amplían y los procesos 5 y 6 son nuevos. Se representan con carriles (*lanes*) por participante, en notación BPMN simplificada: círculo verde = inicio, rectángulo = tarea, rombo = compuerta exclusiva, círculo rojo = fin.

**Proceso 1 – Ingreso de un lote (ampliado):** después de guardar el lote, si se eligió un protocolo sanitario, el sistema genera los eventos de su calendario en la misma operación. Si el lote se carga con una fecha de ingreso pasada, se pueden omitir los eventos ya vencidos.

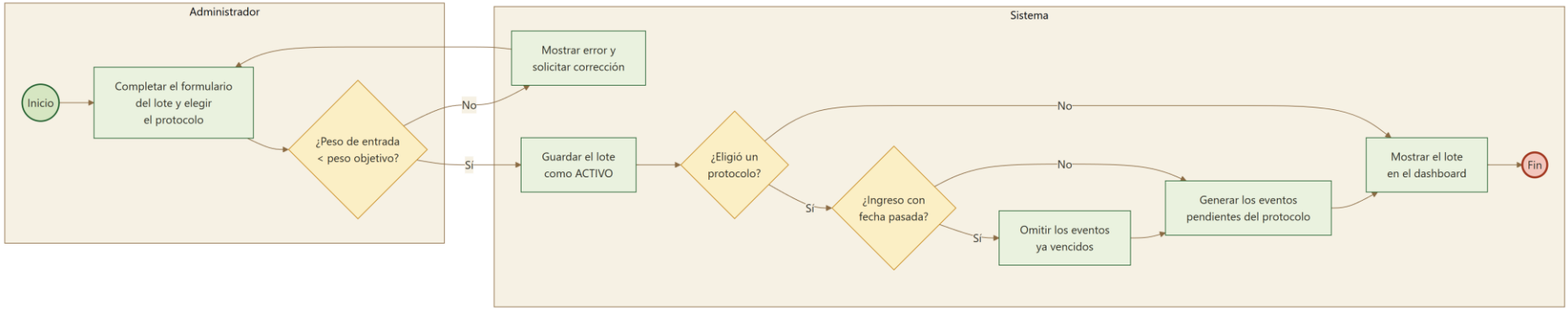



***Figura 2.*** *Proceso 1 (ampliado): ingreso de un lote con su protocolo sanitario*

**Proceso 2 – Alimentación diaria (ampliado):** el encargado ve la ración del día precargada y la confirma con un toque, o informa otros kilos. Si no hay señal, la aplicación guarda la carga en el teléfono y la envía sola al volver la conexión. El servidor la registra una sola vez (idempotencia).

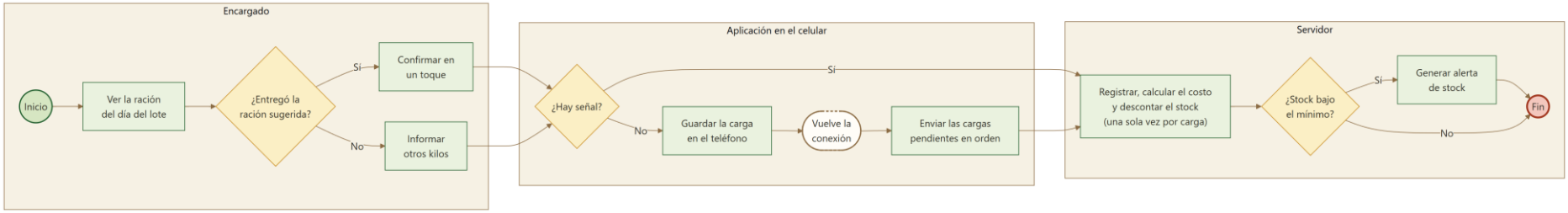



***Figura 3.*** *Proceso 2 (ampliado): alimentación diaria con ración en un toque y carga sin conexión*

**Proceso 5 – Abastecimiento de insumos (nuevo):** cada vez que se consulta el stock o el dashboard, el sistema proyecta el consumo y avisa con anticipación cuándo comprar y cuánto.

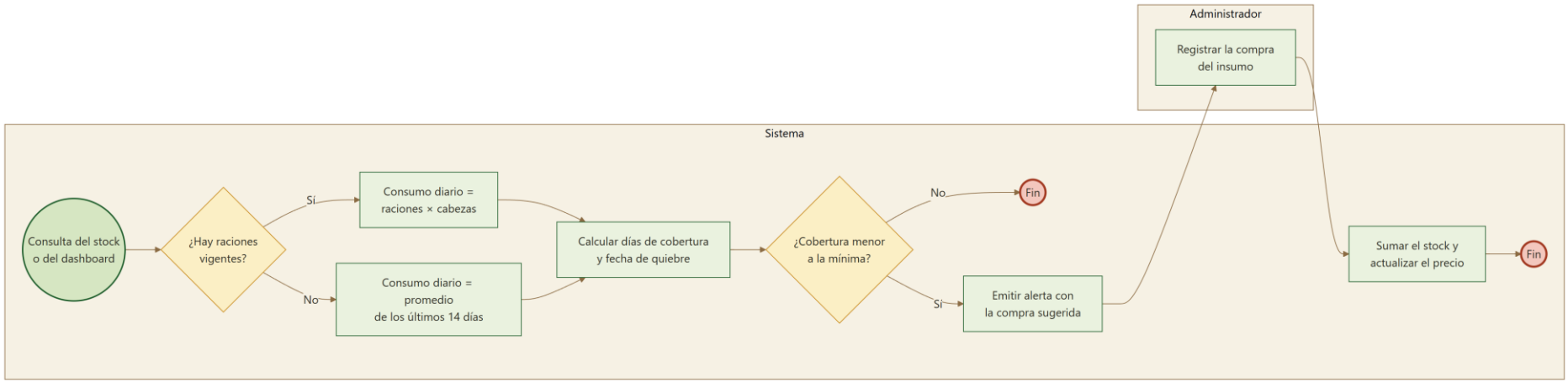



***Figura 4.*** *Proceso 5 (nuevo): abastecimiento de insumos*

**Proceso 6 – Cierre del lote (nuevo):** cuando una pesada indica que el lote alcanzó el peso objetivo, el administrador simula el cierre, revisa el resumen y lo confirma. El lote cerrado queda bloqueado para nuevas cargas y su informe funciona como informe de cierre.

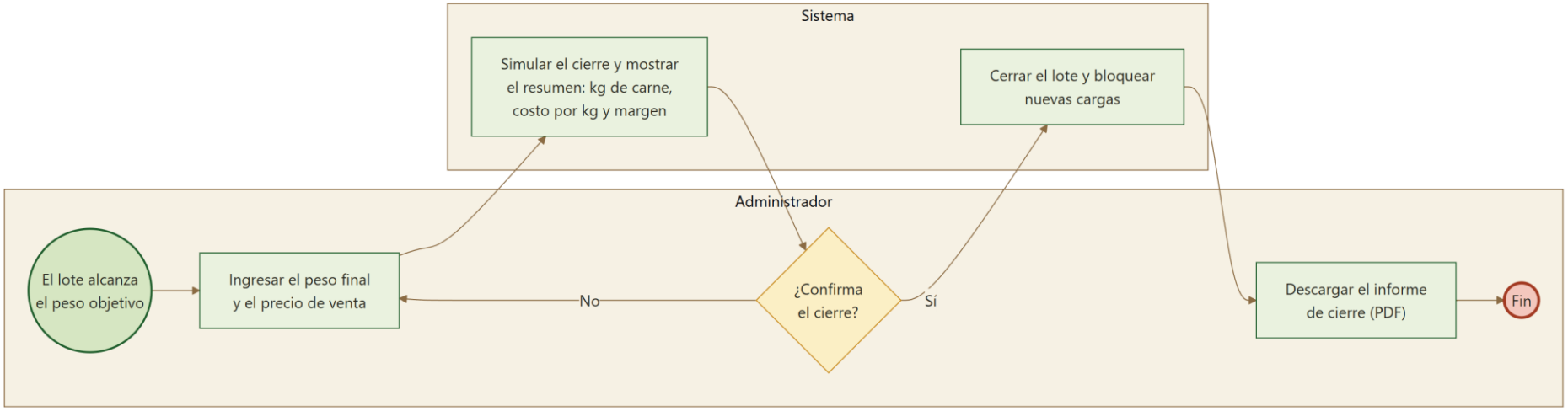



***Figura 5.*** *Proceso 6 (nuevo): cierre del lote*

## 12.3 Procesos que se mantienen de la 1ª entrega

**Proceso 3 – Pesada mensual y evaluación de progreso:** el administrador registra la fecha y el peso promedio; el sistema calcula la GDP, actualiza la proyección y, si la GDP está por debajo del objetivo, genera una alerta de bajo rendimiento.

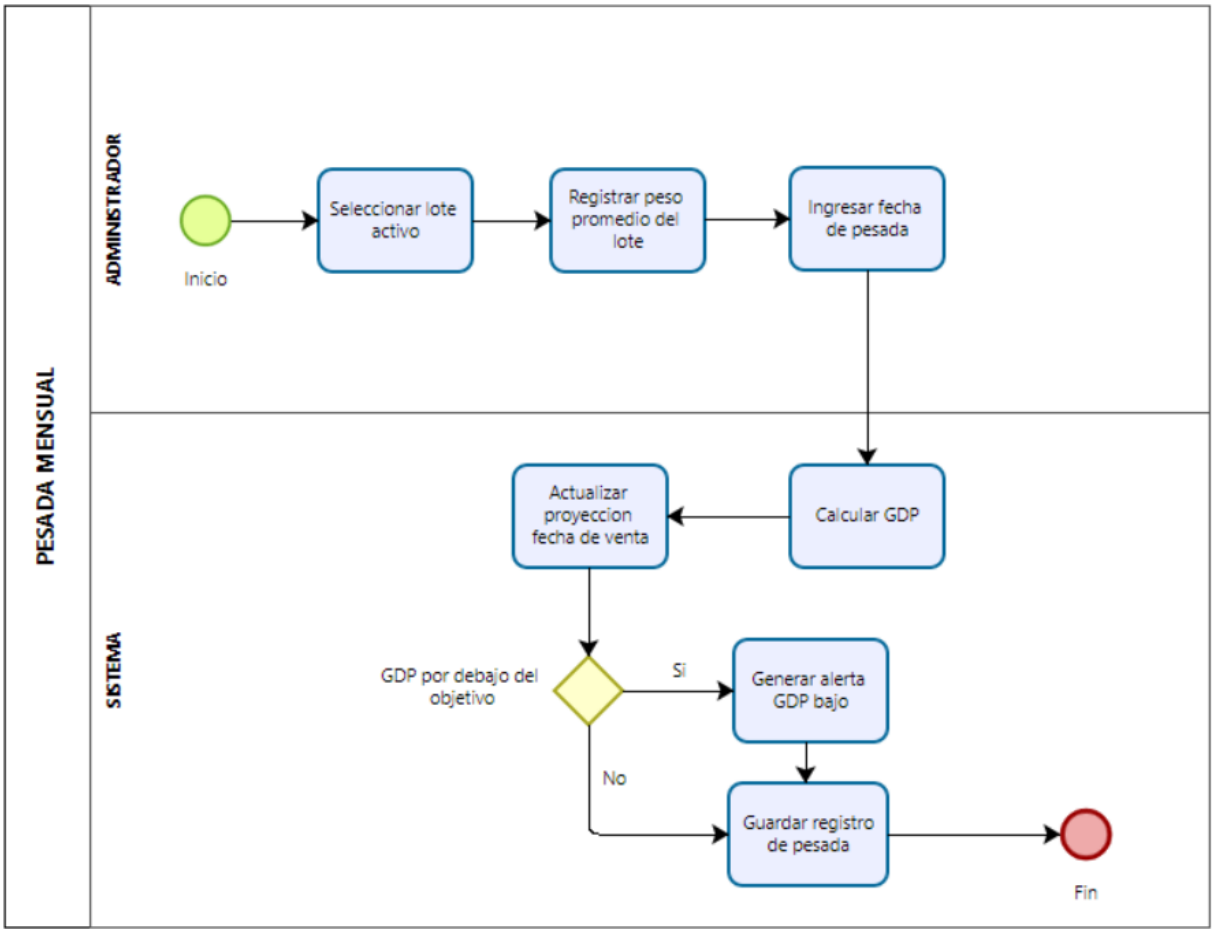



*Figura 6. Proceso 3: pesada mensual y evaluación de progreso (rol Administrador)*

**Proceso 4 – Registro de evento sanitario y alerta:** el sistema avisa los eventos que vencen en los próximos 7 días; el veterinario registra la ejecución y, si es un tratamiento de garrapata, el sistema programa el próximo control.

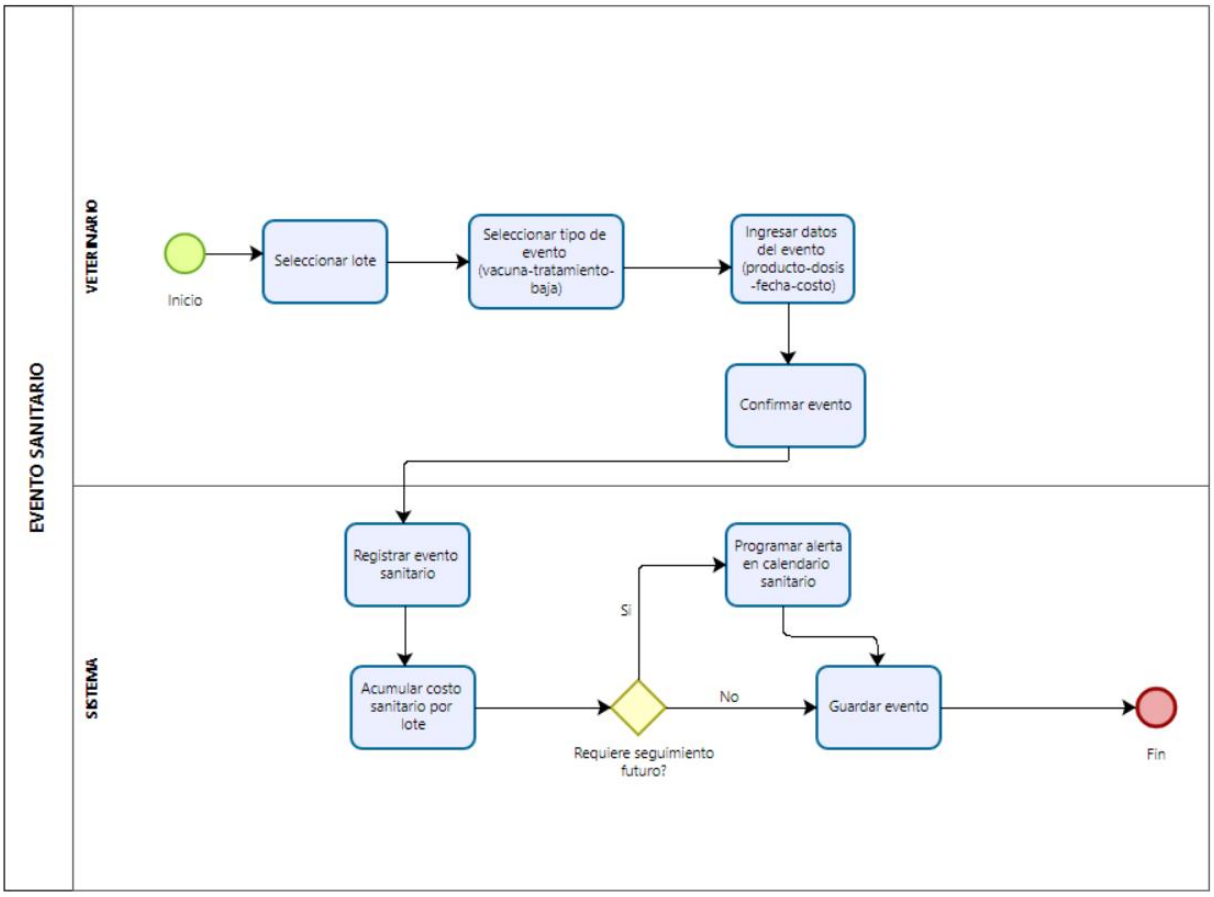



***Figura 7.*** *Proceso 4: registro de evento sanitario y alerta (rol Veterinario)*

# 13. Casos de uso

## 13.1 Actores

***Tabla 6.*** *Actores*

| Actor | Tipo | Descripción | Permisos |
|---|---|---|---|
| Administrador (productor) | Primario | Dueño del establecimiento; toma las decisiones productivas y comerciales | Acceso completo a todos los módulos |
| Encargado | Primario | Empleado que prepara y entrega la ración diaria | Solo registro de alimentación diaria (también sin conexión) y su perfil |
| Veterinario | Primario | Profesional de cabecera; define el plan sanitario y la dieta | Sanidad (eventos, tratamientos, bajas), protocolos sanitarios, raciones, alertas sanitarias y su perfil |
| Sistema (tiempo) | Secundario | Proceso automático que evalúa fechas, stock, GDP y clima | Genera alertas |
| Servicio de clima (Open-Meteo) | Secundario (externo) | Servicio público de pronóstico meteorológico | Provee el clima y el pronóstico de la ubicación configurada |

## 13.2 Lista de casos de uso

***Tabla 7.*** *Lista de casos de uso*

| ID | Caso de uso | Actor principal | RF |
|---|---|---|---|
| CU01 | Iniciar sesión | Administrador, Encargado, Veterinario | RF08 |
| CU02 | Gestionar usuarios | Administrador | RF08 |
| CU03 | Registrar lote | Administrador | RF01 |
| CU04 | Registrar alimentación diaria | Encargado | RF02, RF03 |
| CU05 | Actualizar stock de insumos («include») | — | RF03 |
| CU06 | Registrar compra de insumo | Administrador | RF03 |
| CU07 | Registrar pesada mensual | Administrador | RF04 |
| CU08 | Programar evento sanitario | Veterinario | RF05 |
| CU09 | Registrar evento sanitario realizado | Veterinario | RF05 |
| CU10 | Registrar baja de animales | Veterinario | RF05 |
| CU11 | Consultar costos y rentabilidad del lote | Administrador | RF06 |
| CU12 | Ver dashboard y alertas | Administrador | RF07 |
| CU13 | Cerrar lote | Administrador | RF01, RF07 |
| CU14 | Generar alertas automáticas | Sistema | RF03, RF04, RF05, RF11, RF12 |
| CU15 | Definir ración del lote | Veterinario, Administrador | RF09 |
| CU16 | Confirmar la ración del día en un toque («extend» CU04) | Encargado | RF09, RNF01 |
| CU17 | Gestionar y aplicar protocolos sanitarios («extend» CU03) | Veterinario, Administrador | RF10 |
| CU18 | Consultar proyección de stock | Administrador | RF11 |
| CU19 | Consultar clima y riesgos | Administrador (los demás roles ven el clima actual) | RF12 |
| CU20 | Descargar reportes | Administrador | RF13 |
| CU21 | Cargar alimentación sin conexión («extend» CU04) | Encargado | RF14, RNF07 |

| ID | Caso de uso | Actor principal | RF |
|---|---|---|---|
| CU22 | Actualizar perfil propio | Administrador, Encargado, Veterinario | RF08 |

Los casos CU15 a CU22 se agregaron en la versión final (sistema terminado). Las correcciones de datos (eliminar la última pesada o una baja, reprogramar un evento, corregir una compra) se modelan como flujos alternativos de CU07, CU10, CU08 y CU06.

## 13.3 Diagrama de casos de uso

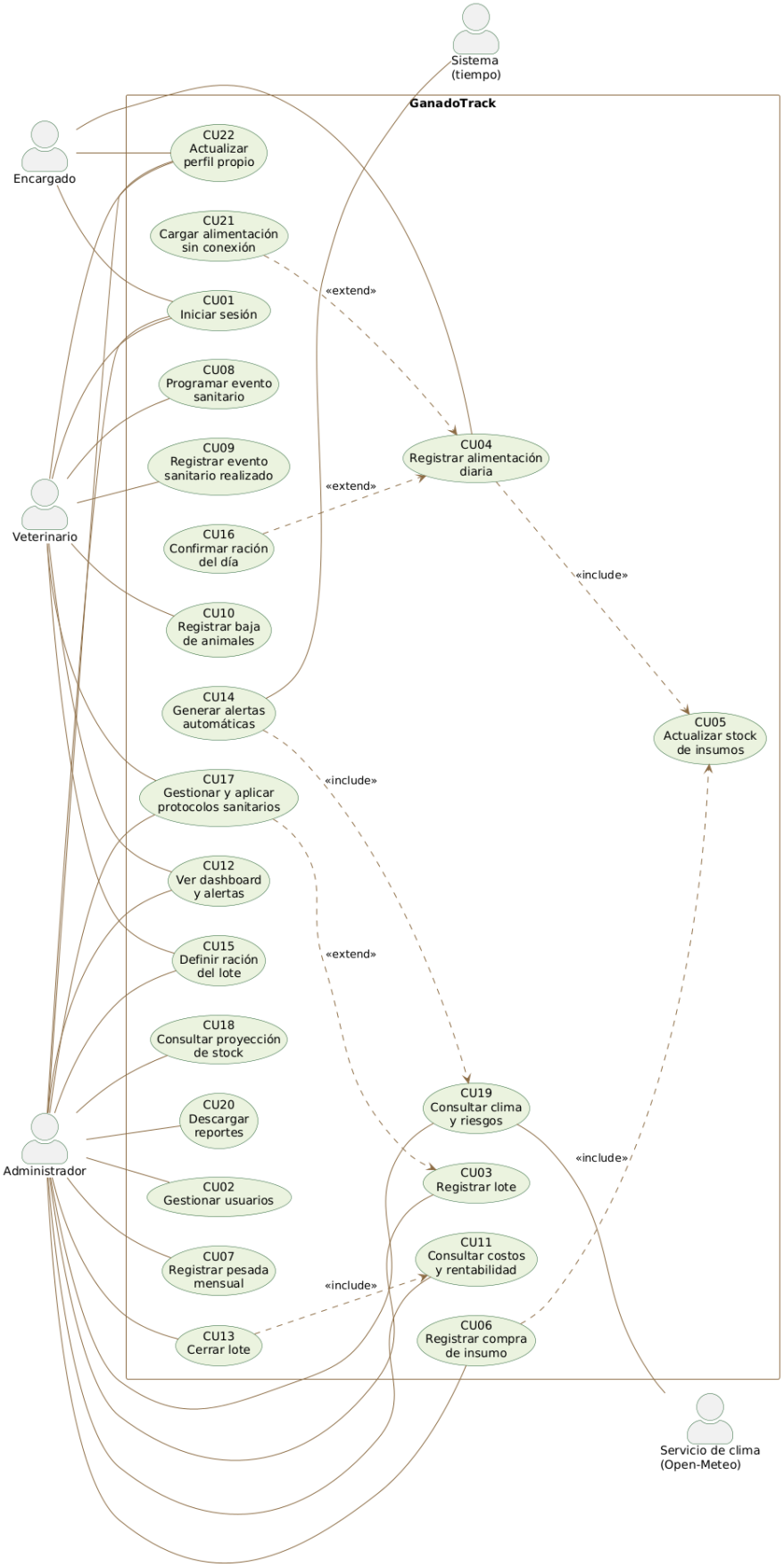


*Figura 8. Diagrama de casos de uso de GanadoTrack*

> *Notas:*

- *El veterinario accede a CU12 solo para ver las alertas **sanitarias**, incluidas las de clima que afectan la sanidad (restricción RF08).*
- *En «extend» la flecha va desde el caso que extiende hacia el caso base: CU16 y CU21 son variantes de CU04 que se activan cuando el lote tiene ración definida o cuando no hay conexión.*
- *CU17 «extend» CU03: si al crear el lote se elige un protocolo sanitario, se generan sus eventos (punto de extensión "protocolo elegido"); con "Ninguno", el lote se crea sin calendario.*

## 13.4 Fichas de casos de uso

### CU01 – Iniciar sesión

*Tabla 8. CU01 – Iniciar sesión*

| Campo | Descripción |
|---|---|
| Actores | Administrador, Encargado, Veterinario |
| Precondición | El usuario existe y está activo. |
| Postcondición | El usuario queda autenticado por 8 h y accede a la pantalla de su rol. |
| Flujo principal | 1. El usuario ingresa email y contraseña. 2. El sistema valida las credenciales. 3. El sistema inicia la sesión. 4. El sistema muestra la pantalla de inicio según el rol: Administrador → Dashboard; Encargado → Cargar alimentación; Veterinario → Sanidad. |
| Flujo alternativo | 3a. La sesión expira tras 8 h sin actividad → el sistema solicita volver a ingresar. 3b. Se abre la app sin conexión en un dispositivo donde el usuario ya había ingresado → se usa la sesión guardada y el encargado puede seguir cargando (CU21). |
| Excepción | 2a. Credenciales incorrectas o usuario inactivo → mensaje "Email o contraseña incorrectos"; no se indica cuál de los dos falló. |

### CU02 – Gestionar usuarios

*Tabla 9. CU02 – Gestionar usuarios*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | Sesión iniciada como Administrador. |
| Postcondición | El usuario queda creado, modificado o desactivado. |
| Flujo principal | 1. El administrador abre "Usuarios". 2. Selecciona "Nuevo usuario". 3. Ingresa nombre, email, rol y contraseña inicial. 4. Confirma. 5. El sistema guarda la contraseña hasheada y muestra el usuario en la lista. |
| Flujo alternativo | 2a. Edita un usuario existente (nombre, rol) o lo desactiva. Un usuario desactivado no puede iniciar sesión. |
| Excepción | 4a. El email ya existe → "Ya existe un usuario con ese email". 4b. El administrador intenta desactivarse a sí mismo → el sistema lo impide. |

### CU03 – Registrar lote

*Tabla 10. CU03 – Registrar lote*

| Campo | Descripción |
|---|---|
| Actor | Administrador |

| Campo | Descripción |
|---|---|
| Precondición | Sesión iniciada como Administrador. |
| Postcondición | El lote queda en estado ACTIVO y aparece en el dashboard. |
| Flujo principal | 1. El administrador selecciona "Nuevo lote". 2. Ingresa nombre, fecha de ingreso, cantidad de animales, peso promedio de entrada, peso objetivo y GDP objetivo. El sistema propone el protocolo sanitario marcado por defecto. 3. Confirma. 4. El sistema valida los datos y crea el lote. 5. El sistema genera los eventos sanitarios del protocolo en la misma operación (CU17). 6. El sistema confirma "Lote creado" con la cantidad de eventos programados. |
| Flujo alternativo | 2a. El administrador elige otro protocolo o "Ninguno (cargar eventos a mano)" → en el segundo caso no se generan eventos (CU08). 2b. La fecha de ingreso es anterior a hoy → el sistema ofrece omitir los eventos con fecha ya pasada (opción activada por defecto). |
| Excepción | 4a. El nombre ya existe → error. 4b. El peso objetivo es menor o igual al de entrada, la cantidad es ≤ 0 o la fecha es futura → error de validación; no se guarda. 5a. El protocolo elegido no existe → no se crea ni el lote ni sus eventos (transacción). |

## CU04 – Registrar alimentación diaria

***Tabla 11.*** *CU04 – Registrar alimentación diaria*

| Campo | Descripción |
|---|---|
| Actor | Encargado (también puede hacerlo el Administrador) |
| Precondición | Sesión iniciada. Existe al menos un lote activo. |
| Postcondición | Los registros de alimentación quedan guardados con su costo. El stock de insumos se descuenta (CU05). |
| Flujo principal | 1. El encargado selecciona el lote. 2. Ingresa los kg de suplemento entregados. 3. Ingresa los kg de silaje entregados. 4. Confirma. 5. El sistema calcula el costo del día (kg × precio/kg de cada insumo) y lo suma al acumulado del lote. 6. El sistema descuenta los kg del stock (CU05). 7. El sistema muestra la confirmación con el costo del día. |
| Flujo alternativo | 1a. El lote tiene ración definida → el sistema muestra la ración del día y el encargado puede confirmarla en un toque (CU16). 2a/3a. Si se informa un solo insumo (el otro vacío o en 0), se registra solo ese. 4c. No hay conexión al confirmar → la carga se guarda en el teléfono (CU21). 6a. El stock de un insumo queda por debajo del mínimo → el sistema genera una alerta de stock para el Administrador (CU14). |
| Excepción | 4a. Ambos valores están vacíos o en 0 → "Ingresá al menos un valor"; no se guarda. 4b. Ya existe un registro de ese insumo para ese lote en la fecha → "Ya se registró la alimentación de hoy para este lote". 6b. Los kg superan el stock disponible → "Stock insuficiente"; no se guarda nada. |

## CU05 – Actualizar stock de insumos (incluido)

***Tabla 12.*** *CU05 – Actualizar stock de insumos (incluido)*

| Campo | Descripción |
|---|---|
| Invocado por | CU04 (descuenta) y CU06 (suma) |
| Precondición | El insumo existe. |
| Postcondición | El stock del insumo queda actualizado en la misma transacción que el registro que lo originó. |
| Flujo principal | 1. El sistema recibe el insumo, la cantidad en kg y la operación (sumar o restar). 2. Actualiza el stock. 3. Si el stock resultante es menor al mínimo, marca la condición de alerta (CU14). |
| Excepción | 2a. Resta mayor al stock disponible → se cancela toda la operación (rollback). |

## CU06 – Registrar compra de insumo

***Tabla 13.*** *CU06 – Registrar compra de insumo*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | Sesión iniciada como Administrador. |
| Postcondición | La compra queda registrada, el stock aumenta (CU05) y el precio por kg del insumo se actualiza. |
| Flujo principal | 1. El administrador abre "Insumos". 2. Selecciona el insumo (suplemento o silaje). 3. Ingresa fecha, cantidad en kg y precio total. 4. Confirma. 5. El sistema suma la cantidad al stock (CU05) y actualiza el precio/kg = precio total / cantidad. |
| Flujo alternativo | 3a. Silaje de producción propia: el administrador ingresa el costo estimado por tonelada producida. 3b. Desde la proyección de stock, "Comprar N kg" precarga la cantidad sugerida (CU18). 6a. **Corregir una compra:** el administrador edita la cantidad o el precio de una compra registrada; el sistema ajusta el stock por la diferencia y recalcula el precio vigente en la misma transacción. |
| Excepción | 4a. Cantidad o precio ≤ 0 → error de validación. 6b. La corrección dejaría el stock negativo (el insumo ya se consumió) → error; no se modifica nada. |

## CU07 – Registrar pesada mensual

***Tabla 14.*** *CU07 – Registrar pesada mensual*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | El lote está ACTIVO. |
| Postcondición | La pesada queda guardada, la GDP calculada y la proyección de llegada al objetivo actualizada. |
| Flujo principal | 1. El administrador selecciona el lote. 2. Ingresa la fecha y el peso promedio. 3. El sistema calcula la GDP = (peso actual − peso anterior) / días transcurridos. Si es la primera pesada, el peso anterior es el peso de ingreso. 4. El sistema proyecta la fecha de llegada al peso objetivo. 5. El sistema muestra GDP actual, días restantes y costo acumulado. |
| Flujo alternativo A | 3a. GDP calculada < GDP objetivo del lote → alerta de bajo rendimiento en el dashboard (CU14). |
| Flujo alternativo B | 3b. Peso ≥ peso objetivo → el sistema informa que el lote alcanzó el objetivo y sugiere cerrarlo (CU13). |
| Flujo alternativo C | 5a. El sistema muestra la conversión alimenticia del período (kg de materia seca por kg ganado, RF09). 6a. **Pesada cargada por error:** el administrador elimina la última pesada del lote, previa confirmación; las pesadas anteriores y su GDP no cambian. |
| Excepción | 2a. Fecha anterior o igual a la última pesada, o anterior al ingreso → error. 2b. Peso ≤ 0 → error de validación. |

## CU08 – Programar evento sanitario

***Tabla 15.*** *CU08 – Programar evento sanitario*

| Campo | Descripción |
|---|---|
| Actor | Veterinario (también el Administrador) |
| Precondición | El lote está ACTIVO. |
| Postcondición | El evento queda en el calendario sanitario del lote en estado PENDIENTE. |
| Flujo principal | 1. El veterinario selecciona el lote. 2. Selecciona "Programar evento". 3. Ingresa el tipo (vacuna, antiparasitario, tratamiento de garrapata, otro), el nombre, el producto previsto y la fecha programada. 4. Confirma. 5. El sistema agrega el evento al calendario. |

| Campo | Descripción |
|---|---|
| Flujo alternativo | 1a. Los eventos también se generan automáticamente al crear el lote con un protocolo (CU17). 2a. **Reprogramar:** el veterinario modifica la fecha, el nombre o el producto de un evento pendiente. 2b. Elimina un evento pendiente que ya no corresponde. |
| Excepción | 3a. Fecha programada anterior a hoy → error. 2c. El evento ya fue realizado → no se puede modificar ni eliminar (409). |

## CU09 – Registrar evento sanitario realizado

***Tabla 16.*** *CU09 – Registrar evento sanitario realizado*

| Campo | Descripción |
|---|---|
| Actor | Veterinario (también el Administrador) |
| Precondición | Sesión iniciada. El lote tiene un evento pendiente, o se registra uno no programado. |
| Postcondición | El evento queda REALIZADO y su costo se suma al acumulado sanitario del lote. |
| Flujo principal | 1. El veterinario selecciona el lote. 2. Selecciona el evento pendiente (o "Nuevo evento no programado"). 3. Ingresa fecha real, producto utilizado, dosis y costo total. 4. Confirma. 5. El sistema marca el evento como realizado y suma el costo al acumulado sanitario. |
| Flujo alternativo A | 3a. El evento es un tratamiento de garrapata → el sistema solicita además la duración del tratamiento (días) y programa automáticamente un nuevo evento de control en fecha real + duración (CU08). |
| Flujo alternativo B | El veterinario necesita registrar muertes → deriva a CU10. |
| Excepción | 3b. No se ingresa el costo → el sistema permite guardarlo en $0 mostrando la advertencia "El costo sanitario del lote quedará subestimado". |

## CU10 – Registrar baja de animales

***Tabla 17.*** *CU10 – Registrar baja de animales*

| Campo | Descripción |
|---|---|
| Actor | Veterinario (también el Administrador) |
| Precondición | El lote está ACTIVO. |
| Postcondición | La baja queda registrada y las cabezas actuales del lote disminuyen. |
| Flujo principal | 1. El veterinario selecciona el lote. 2. Selecciona "Registrar baja". 3. Ingresa fecha, cantidad de animales y causa (enfermedad, accidente, sequía, otra) con una observación opcional. 4. Confirma. 5. El sistema actualiza las cabezas actuales y recalcula los costos por kg. |
| Flujo alternativo | 5a. **Baja cargada por error:** el administrador la elimina, previa confirmación; las cabezas del lote se restituyen y los costos por kg se recalculan. |
| Excepción | 3a. Cantidad mayor a las cabezas actuales del lote o ≤ 0 → error. 5b. El veterinario intenta eliminar una baja → no tiene permiso (solo el administrador). |

## CU11 – Consultar costos y rentabilidad del lote

***Tabla 18.*** *CU11 – Consultar costos y rentabilidad del lote*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | El lote existe. |
| Postcondición | — (consulta) |
| Flujo principal | 1. El administrador abre el detalle del lote, pestaña "Costos". 2. El sistema muestra: costo de alimentación, costo sanitario, costo total, cabezas actuales, kg de carne estimados (peso × cabezas × 57 %), costo por kg de carne y margen bruto estimado. |

| Campo | Descripción |
|---|---|
| Flujo alternativo | 2a. El precio de venta no está cargado → el sistema muestra los costos y solicita el precio de venta por kg para calcular el margen. El administrador lo ingresa y el sistema recalcula. |
| Excepción | 2b. El lote no tiene pesadas → se usa el peso de ingreso y se indica que el cálculo es preliminar. |

## CU12 – Ver dashboard y alertas

***Tabla 19.*** *CU12 – Ver dashboard y alertas*

| Campo | Descripción |
|---|---|
| Actor | Administrador (el Veterinario ve solo las alertas sanitarias) |
| Precondición | Sesión iniciada. |
| Postcondición | — (consulta) |
| Flujo principal | 1. El administrador inicia sesión o abre "Dashboard". 2. El sistema muestra, por cada lote activo: días en feedlot, cabezas, peso promedio actual, última GDP, costo acumulado y cantidad de alertas. 3. El sistema muestra el panel de alertas activas (sanitarias, de stock, de stock proyectado, de GDP y de clima) ordenadas por urgencia. 4. El sistema muestra el clima del campo y sus riesgos (CU19) y el stock de insumos. 5. El administrador selecciona un lote o una alerta para ver el detalle. |
| Flujo alternativo | 2a. No hay lotes activos → el sistema invita a crear el primero (CU03). |

## CU13 – Cerrar lote

***Tabla 20.*** *CU13 – Cerrar lote*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | El lote está ACTIVO. |
| Postcondición | El lote pasa a CERRADO, deja de admitir registros y queda en el historial con su resumen financiero final. |
| Flujo principal | 1. El administrador abre el lote y selecciona "Cerrar lote". 2. Ingresa la fecha de cierre (venta/faena), el peso final promedio y el precio de venta por kg. 3. El sistema muestra el resumen final (CU11): costo total, kg de carne, costo por kg y margen. 4. El administrador confirma. 5. El sistema cierra el lote. |
| Excepción | 2a. Fecha de cierre anterior al ingreso → error. 4a. El lote tiene eventos sanitarios pendientes → advertencia; el administrador puede confirmar igual. |

## CU14 – Generar alertas automáticas

***Tabla 21.*** *CU14 – Generar alertas automáticas*

| Campo | Descripción |
|---|---|
| Actor | Sistema |
| Disparador | Cada vez que se consulta el dashboard o las alertas (evaluación en tiempo real sobre los datos). |
| Postcondición | Las alertas vigentes quedan visibles para el rol correspondiente. |

| Campo | Descripción |
|---|---|
| Flujo principal | 1. El sistema revisa los eventos sanitarios PENDIENTES: si faltan 7 días o menos → alerta "Próximo evento"; si la fecha ya pasó → alerta "Evento vencido". 2. Revisa los insumos: stock < mínimo → alerta de stock. 3. Proyecta el consumo (CU18): si el stock alcanza para menos días que la cobertura mínima → alerta de stock proyectado. 4. Revisa la última pesada de cada lote activo: GDP < GDP objetivo → alerta de bajo rendimiento. 5. Consulta el clima (CU19): ITH en nivel de peligro o emergencia → alerta de estrés térmico; riesgo alto de garrapata → alerta por cada lote sin tratamiento de garrapata en los últimos 21 días; riesgo alto de sequía → alerta con los días que alcanza el silaje. 6. Filtra según el rol (el veterinario recibe solo las sanitarias y las de clima sanitario) y devuelve las alertas ordenadas por severidad. |
| Flujo alternativo | 5a. El servicio de clima no responde o la ubicación no está configurada → se omiten las alertas de clima y el resto se genera normalmente. |
| Nota | Los 7 días de anticipación y la cobertura mínima (7 días) son parámetros configurables. |

## CU15 – Definir ración del lote

***Tabla 22.*** *CU15 – Definir ración del lote*

| Campo | Descripción |
|---|---|
| Actor | Veterinario (también el Administrador) |
| Precondición | Sesión iniciada. El lote está ACTIVO. |
| Postcondición | La ración queda registrada con su fecha de vigencia; desde esa fecha, el encargado ve los kg del día calculados. |
| Flujo principal | 1. El veterinario abre "Raciones". 2. El sistema muestra cada lote activo con sus cabezas, la ración vigente y los kg del día. 3. El veterinario edita la ración de un lote: kg por cabeza por día de cada insumo y la fecha desde la que rige. 4. Confirma. 5. El sistema guarda la ración (conservando las anteriores como historial) y recalcula los kg del día = kg por cabeza × cabezas actuales. |
| Flujo alternativo | 3a. Ingresa 0 kg en un insumo → ese insumo deja de formar parte de la ración. 3b. La fecha de vigencia es futura → la ración actual sigue rigiendo hasta esa fecha. |
| Excepción | 3c. Kg negativos o mayores a 80 por cabeza, fecha anterior al ingreso del lote o lote cerrado → error; no se guarda. 3d. El encargado intenta modificar una ración → no tiene permiso (403). |

## CU16 – Confirmar la ración del día en un toque

***Tabla 23.*** *CU16 – Confirmar la ración del día en un toque*

| Campo | Descripción |
|---|---|
| Actor | Encargado (también el Administrador) |
| Extiende a | CU04, cuando el lote tiene ración definida (punto de extensión: paso 1) |
| Precondición | Sesión iniciada. El lote tiene ración vigente y no se cargó su alimentación hoy. |
| Postcondición | Igual que CU04: registros guardados con su costo y stock descontado (CU05). |
| Flujo principal | 1. El encargado abre "Comida de hoy". 2. El sistema muestra cada lote con su estado (Pendiente, Carga parcial, Cargado hoy) y la "Ración de hoy" con los kg de cada insumo. 3. El encargado toca "Confirmar ración de hoy". 4. El sistema registra los kg sugeridos (continúa en el paso 5 de CU04). 5. La tarjeta pasa a "Cargado hoy" con los kg entregados. |
| Flujo alternativo | 3a. Entregó otra cantidad → toca "Di otros kilos"; el sistema abre el formulario con los kg sugeridos precargados para modificarlos (paso 2 de CU04). 3b. Activa "Modo pleno sol" → letra más grande y alto contraste. |
| Excepción | Las de CU04 (stock insuficiente, registro duplicado). |

## CU17 – Gestionar y aplicar protocolos sanitarios

***Tabla 24.*** *CU17 – Gestionar y aplicar protocolos sanitarios*

| Campo | Descripción |
|---|---|
| Actor | Veterinario (también el Administrador) |
| Extiende a | CU03, cuando al crear el lote se elige un protocolo (punto de extensión: paso 5) |
| Precondición | Sesión iniciada como Veterinario o Administrador. |
| Postcondición | El protocolo queda guardado y disponible al crear lotes; si se aplicó a un lote, sus eventos quedan PENDIENTES en el calendario. |
| Flujo principal | 1. El veterinario abre "Protocolos". 2. Selecciona "Nuevo protocolo". 3. Ingresa nombre y descripción, y agrega los eventos: tipo, nombre, producto, día desde el ingreso y, si se repite, cada cuántos días y cuántas aplicaciones. 4. Opcionalmente lo marca "Usar por defecto al crear lotes". 5. Confirma. 6. El sistema guarda el protocolo y muestra su línea de tiempo. |
| Flujo alternativo | 2a. Edita o elimina un protocolo existente (los lotes ya creados no cambian). 4a. Marca otro protocolo como "por defecto" → el anterior deja de serlo. 7a. **Aplicar a un lote existente:** desde el calendario sanitario del lote, "Aplicar protocolo" genera los eventos con fecha = ingreso del lote + día de cada evento, con la opción de omitir los ya pasados. |
| Excepción | 3a. Evento repetido sin frecuencia, día fuera de 0–365 o nombre duplicado → error de validación. 3b. El encargado intenta acceder → 403. |

## CU18 – Consultar proyección de stock

***Tabla 25.*** *CU18 – Consultar proyección de stock*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | Sesión iniciada como Administrador. |
| Postcondición | — (consulta). |
| Flujo principal | 1. El administrador abre "Insumos y stock". 2. El sistema calcula, para cada insumo, el consumo diario previsto (raciones vigentes × cabezas). 3. Muestra los días de cobertura ("Alcanza para N días, hasta dd/mm"), el consumo previsto y el gráfico de stock proyectado a 30 días con la línea del mínimo. 4. Si la cobertura es menor a la mínima, muestra la compra sugerida ("Comprar N kg"). 5. El administrador puede registrar la compra con la cantidad precargada (CU06). |
| Flujo alternativo | 2a. No hay raciones definidas → el sistema usa el consumo real promedio de los últimos 14 días. 2b. No hay consumo previsto ni histórico → informa "Sin consumo previsto: definí las raciones de los lotes". |

## CU19 – Consultar clima y riesgos

***Tabla 26.*** *CU19 – Consultar clima y riesgos*

| Campo | Descripción |
|---|---|
| Actores | Administrador; Encargado y Veterinario (solo el clima actual en la barra superior). Secundario: Servicio de clima |
| Precondición | Sesión iniciada. La ubicación del establecimiento está configurada. |
| Postcondición | — (consulta). |
| Flujo principal | 1. El administrador abre el Dashboard. 2. El sistema solicita al servicio de clima el estado actual, el pronóstico de 7 días y la lluvia de los últimos 30 días de la ubicación configurada. 3. Calcula el ITH (estrés térmico), el riesgo de garrapata y el riesgo de sequía. 4. Muestra el panel "Clima del campo" con el pronóstico y los "Riesgos (orientativos)". |
| Flujo alternativo | 2a. Hay un dato consultado hace menos de 30 minutos → se usa sin volver a consultar. 0a. **Configurar la ubicación:** en "Configuración", el administrador busca la localidad por nombre, elige un resultado y lo guarda. |

| Campo | Descripción |
|---|---|
| Excepción | 2b. El servicio no responde → "El servicio de clima no responde en este momento"; el resto del sistema sigue funcionando. 0b. Ubicación sin configurar → el panel invita a configurarla. |

## CU20 – Descargar reportes

***Tabla 27.*** *CU20 – Descargar reportes*

| Campo | Descripción |
|---|---|
| Actor | Administrador |
| Precondición | Sesión iniciada como Administrador. |
| Postcondición | El archivo queda descargado en el dispositivo. |
| Flujo principal | 1. El administrador abre "Reportes". 2. Elige el reporte: **Informe de lote** (elige el lote), **Reporte mensual** (elige el mes) o **Planilla para el contador** (elige el rango de fechas). 3. Presiona "Descargar". 4. El sistema genera el PDF o el Excel con los datos al momento y lo descarga. |
| Flujo alternativo | 1a. Desde el detalle de un lote, "Informe PDF" (o "Informe de cierre" si el lote está cerrado) descarga directamente el informe de ese lote. |
| Excepción | 2a. Mes o rango de fechas inválido → error de validación. 3a. Otro rol intenta descargar un reporte → 403. |

## CU21 – Cargar alimentación sin conexión

***Tabla 28.*** *CU21 – Cargar alimentación sin conexión*

| Campo | Descripción |
|---|---|
| Actor | Encargado |
| Extiende a | CU04/CU16, cuando no hay conexión al confirmar (punto de extensión: paso 4) |
| Precondición | La app se abrió al menos una vez con conexión en ese dispositivo (datos del día y sesión guardados). |
| Postcondición | La carga queda guardada en el teléfono y, al volver la señal, registrada en el sistema una sola vez, con la fecha en que se capturó. |
| Flujo principal | 1. El sistema detecta que no hay conexión y muestra el aviso "Sin conexión: podés seguir cargando". 2. El encargado confirma la ración (CU16) o informa los kilos (CU04). 3. El sistema guarda la carga en el teléfono con un identificador único y la fecha de captura, y marca el lote "En el teléfono". 4. Al volver la conexión (o al tocar "Enviar ahora"), el sistema envía las cargas guardadas, en orden. 5. El servidor registra cada carga y descuenta el stock. 6. El aviso de cargas pendientes desaparece. |
| Flujo alternativo | 5a. La misma carga llega dos veces (por ejemplo, se cortó la señal durante el envío) → el servidor reconoce el identificador y no la registra de nuevo. |
| Excepción | 5b. El servidor rechaza la carga (stock insuficiente, ya cargada, lote cerrado) → queda marcada "No se pudo enviar" con el motivo; el encargado puede descartarla y avisar al administrador. |

## CU22 – Actualizar perfil propio

***Tabla 29.*** *CU22 – Actualizar perfil propio*

| Campo | Descripción |
|---|---|
| Actores | Administrador, Encargado, Veterinario |
| Precondición | Sesión iniciada. |
| Postcondición | El nombre o la contraseña del usuario quedan actualizados. |
| Flujo principal | 1. El usuario abre el menú de usuario → "Mi perfil". 2. Modifica su nombre y presiona "Guardar nombre". 3. El sistema actualiza el nombre en la barra superior. |

| Campo | Descripción |
|---|---|
| Flujo alternativo | 2a. **Cambiar contraseña:** ingresa la contraseña actual, la nueva y su repetición → el sistema verifica la actual, guarda la nueva (hash bcrypt) y la sesión sigue activa. |
| Excepción | 2b. La contraseña actual es incorrecta → error; tras varios intentos se aplica el mismo límite que en el login. 2c. La nueva contraseña tiene menos de 8 caracteres o no coincide con la repetición → error de validación. |

# 14. Arquitectura y herramientas de desarrollo

## 14.1 Tipo de arquitectura

GanadoTrack utiliza una **arquitectura cliente-servidor en tres capas**, con el frontend como **aplicación de una sola página (SPA)** y el backend como **API REST organizada en capas y por módulos de negocio**:

1. **Capa de presentación:** aplicación web React que se ejecuta en el navegador (PC, tablet o celular).
2. **Capa de lógica de negocio:** API REST en Node.js/Express, que valida, aplica las reglas del feedlot y calcula los indicadores.
3. **Capa de datos:** base de datos relacional PostgreSQL, a la que se accede mediante el ORM Prisma.

### Justificación

***Tabla 30.*** *Justificación*

| Necesidad del caso | Cómo la resuelve la arquitectura |
|---|---|
| Tres tipos de usuario con dispositivos distintos: el encargado usa el celular en el campo; el productor, PC o tablet (RNF01) | Una sola aplicación web responsive sirve a todos los dispositivos, sin instalar nada ni mantener apps nativas |
| Conexión por Starlink y señal de celular (RNF02) | La SPA descarga la interfaz una vez y después solo intercambia JSON liviano con la API |
| Corrales sin señal (RF14, RNF07) | La SPA es además una **PWA**: un *service worker* guarda la aplicación en el teléfono, y las cargas de alimentación sin conexión quedan en una cola local (IndexedDB) que se envía sola al volver la señal |
| Datos externos poco confiables (clima) | La API actúa como intermediaria (*proxy*) del servicio de clima, con caché y tiempo de espera; si el servicio falla, el sistema sigue funcionando |
| Reglas de negocio críticas: costo por kg, GDP, alertas sanitarias | Se centralizan en el backend: ningún cliente puede saltearlas y se prueban con tests automatizados |
| Restricción estricta por rol (RNF04, RF08) | La autorización se verifica en el servidor en cada petición, no solo ocultando botones en la interfaz |
| Datos relacionados entre sí (lote → pesadas, alimentación, eventos, bajas) e integridad de costos | Una base relacional con claves foráneas, restricciones CHECK y transacciones garantiza la consistencia |
| Crecimiento futuro: ciclo completo, caravanas, SENASA (RNF05) | Cada área es un módulo independiente; agregar un módulo nuevo no requiere modificar los existentes |
| Proyecto de una sola persona con plazo acotado | Un único lenguaje (JavaScript) en frontend y backend reduce la curva de aprendizaje y el cambio de contexto |

Se descartaron otras alternativas:

- **Microservicios:** agregan complejidad operativa (varios despliegues, comunicación entre servicios) que no se justifica para un establecimiento con un solo productor.
- **Aplicación monolítica con vistas generadas en el servidor:** acopla la interfaz a la lógica y dificulta una experiencia móvil fluida.

## 14.2 Diagrama de arquitectura

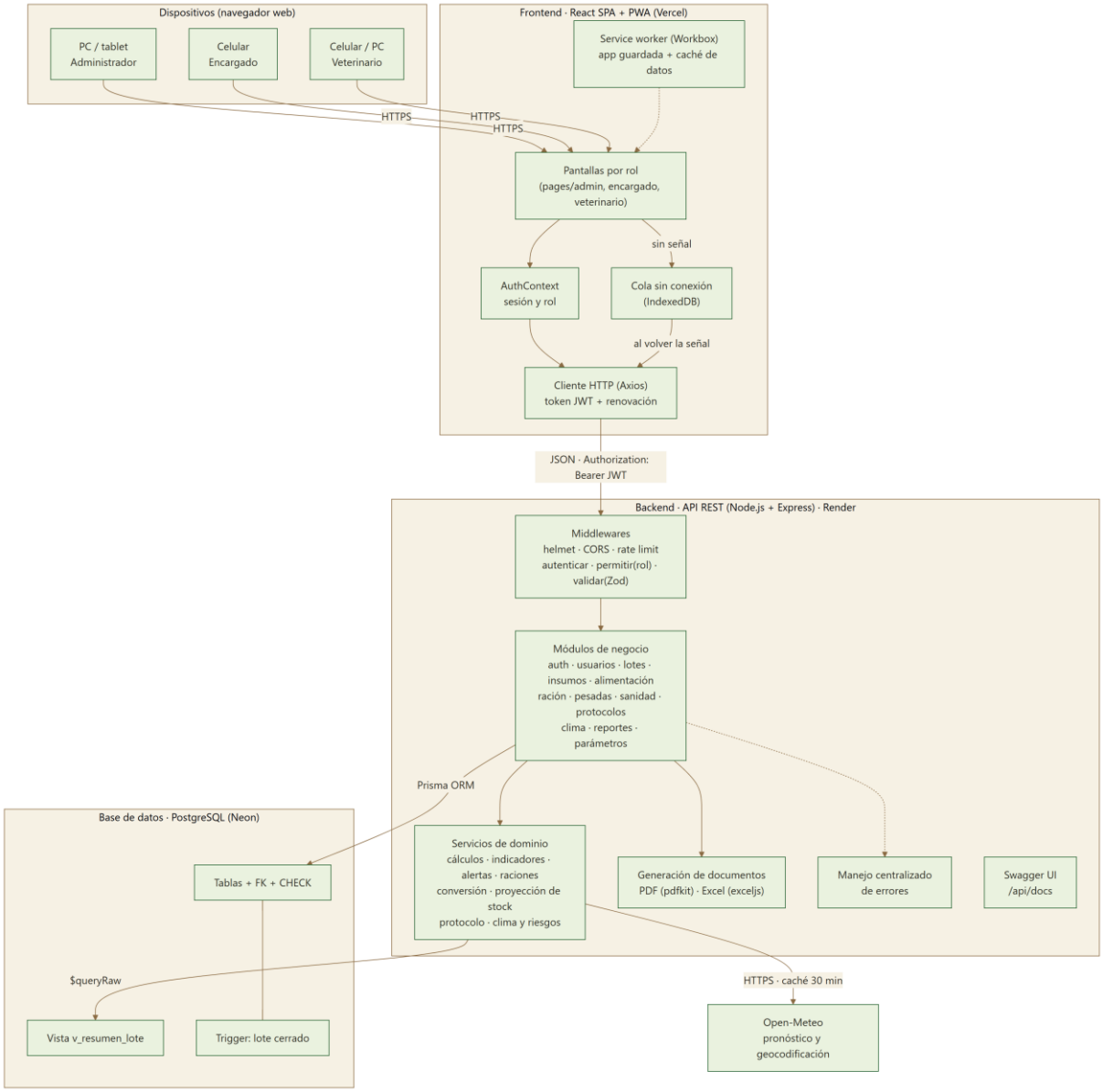



***Figura 9.*** *Arquitectura de GanadoTrack: componentes y comunicación*

## 14.3 Capas y componentes del backend

Cada módulo de `backend/src/modules/` tiene la misma estructura, con una responsabilidad por archivo:

***Tabla 31.*** *Capas y componentes del backend*

| Archivo | Responsabilidad | Ejemplo (módulo alimentación) |
|---|---|---|
| `*.routes.js` | Define las rutas HTTP, los roles permitidos y el esquema de validación | `POST /alimentacion` → `permitir('ENCARGADO','ADMIN')` → `validar(...)` |
| `*.schema.js` | Reglas de validación de los datos de entrada (Zod) | `kg` ≥ 0, al menos un insumo con kg > 0, sin insumos repetidos |
| `*.controller.js` | Traduce HTTP ↔ servicio: lee `req` y responde con el código adecuado | Responde `201` con el resultado |
| `*.service.js` | Lógica de negocio y acceso a datos | Transacción: valida lote, descuenta stock, calcula costo, guarda |

Los **servicios de dominio** compartidos son:

***Tabla 32.*** *Capas y componentes del backend*

| Servicio | Responsabilidad |
|---|---|
| `calculos.js` | Funciones puras documentadas y cubiertas con pruebas unitarias (RNF06): GDP, proyección al objetivo, kg de carne al gancho, costo por kg, margen, kg sugeridos de la ración, materia seca, conversión alimenticia, días de cobertura, compra sugerida y stock proyectado |
| `indicadores.js` | Combina la vista SQL `v_resumen_lote` con los cálculos para obtener los indicadores de cada lote |
| `alertas.js` | Genera las alertas sanitarias, de stock, de stock proyectado, de GDP y de clima, filtradas por rol (CU14) |
| `raciones.js` | Ración vigente de cada lote a una fecha y kg del día según las cabezas actuales (RF09) |
| `conversion.js` | Conversión alimenticia del lote entre pesadas y acumulada (RF09) |
| `proyeccionStock.js` | Consumo previsto, cobertura, fecha de quiebre y compra sugerida de cada insumo (RF11) |
| `protocolo.js` | Planifica las fechas de los eventos de un protocolo y los crea dentro de la transacción del lote (RF10) |
| `clima.js` y `riesgosClima.js` | Consulta a Open-Meteo con caché de 30 minutos y tiempo de espera de 8 s; cálculo puro del ITH y de los riesgos de garrapata y sequía (RF12) |
| `parametros.js` | Lectura de los parámetros configurables con sus valores por defecto |

El módulo `reportes/` incluye además `pdf.js`, con los bloques reutilizables del PDF (encabezado con el logo, secciones, tablas, indicadores y gráfico de línea), y `documentos.js`, que arma el informe de lote, el reporte mensual y la planilla Excel.

## Frontend

***Tabla 33.*** *Frontend*

| Carpeta | Responsabilidad |
|---|---|
| `pages/` | Una pantalla por función, agrupadas por rol (14 pantallas, cargadas bajo demanda) |
| `components/` | Layout con menú por rol, componentes comunes (indicadores, estados vacíos, diálogos de confirmación), ilustraciones SVG propias y panel de clima |
| `api/` y `context/` | Cliente HTTP con el token y su renovación; sesión del usuario (guardada para abrir la app sin conexión) |
| `offline/` | Cola de cargas sin conexión en IndexedDB, detección de conexión y botón de instalación |
| `theme.js` | Tema "Campo cálido": paleta del logo, tipografías Bitter y Nunito Sans, formas y estados |

## 14.4 Comunicación entre frontend, backend y base de datos

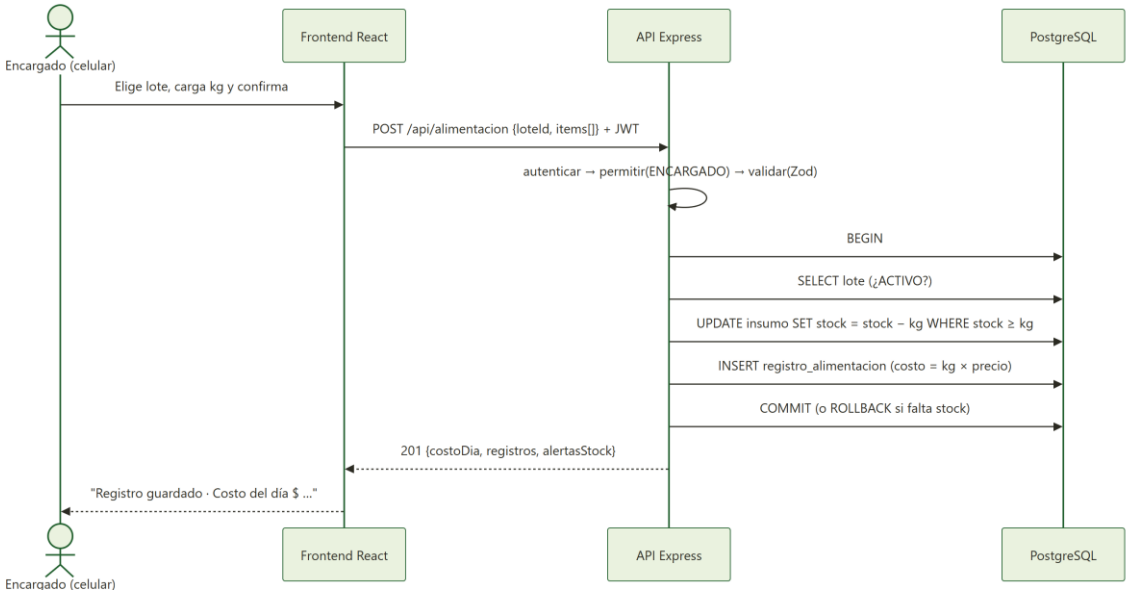



***Figura 10.*** *Secuencia del registro de alimentación diaria (CU04)*

- **Protocolo:** HTTP/HTTPS con cuerpos JSON. Las fechas viajan como `AAAA-MM-DD` y los importes como números.
- **Acceso a datos:** Prisma traduce las operaciones a SQL parametrizado (previene inyección SQL). Las consultas agregadas usan la vista `v_resumen_lote` mediante `$queryRaw` con parámetros.
- **Transacciones:** las operaciones que modifican varias tablas se ejecutan en una única transacción: o se guarda todo o no se guarda nada. Son las siguientes:
    - Alimentación + stock.
    - Compra o corrección de compra + stock.
    - Evento de garrapata + nuevo control.
    - Baja + validación de cabezas.
    - Lote + eventos del protocolo.
    - Cambio del protocolo por defecto.

### Carga sin conexión y sincronización (RF14)

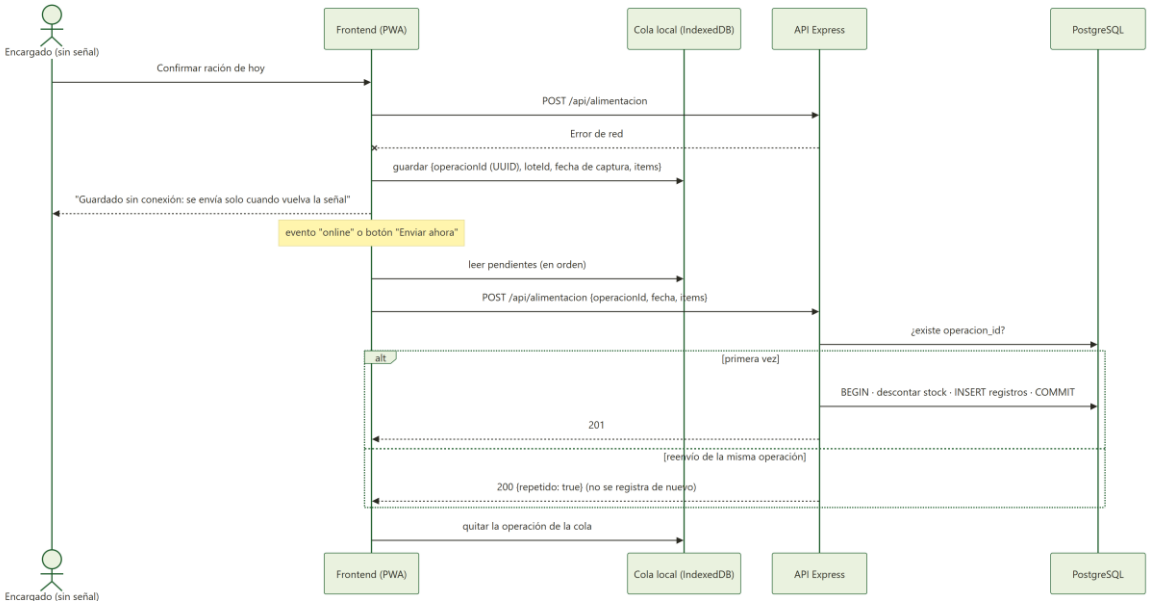



***Figura 11.*** *Secuencia de la carga sin conexión y su sincronización (CU21)*

- **Idempotencia:** el celular genera un identificador único (UUID) por carga. La base tiene un índice único (`operacion_id`, `insumo_id`), así que, aunque la misma carga llegue dos veces (por ejemplo, porque se cortó la señal durante el envío), se registra una sola vez.
- **Fecha de captura:** la carga se registra con la fecha en que se hizo en el corral, no con la del envío.
- **Errores de negocio:** si el servidor rechaza la carga (por ejemplo, por stock insuficiente), queda marcada con el motivo para que el encargado la descarte y avise al administrador. No se reintenta indefinidamente.
- **Service worker:** guarda la aplicación (*app shell*, fuentes e íconos, ≈ 1,9 MB) y aplica la estrategia *NetworkFirst* a las consultas que necesita el encargado: lotes y ración del día (`/alimentacion/opciones`), usuario (`/auth/me`) y clima. Si hay red, usa siempre datos frescos; si no, usa los últimos guardados.

## 14.5 Diseño de la API

API REST con prefijo `/api`, recursos en plural, verbos HTTP estándar y códigos de estado semánticos. El detalle completo está en api.md y en Swagger UI (`/api/docs`).

***Tabla 34.*** *Diseño de la API*

| Recurso | Operaciones | Roles |
|---|---|---|
| `/auth` | `POST /login`, `GET /me`, `PATCH /perfil`, `PATCH /password` | Público / autenticado |
| `/usuarios` | listar, crear, modificar | ADMIN |
| `/lotes` | listar, crear (con protocolo), detalle, editar, `POST /:id/cierre` | ADMIN |
| `/raciones`, `/lotes/:id/racion` | raciones de los lotes activos; consultar y definir la ración de un lote | VETERINARIO, ADMIN |
| `/protocolos`, `/lotes/:id/aplicar-protocolo` | CRUD de protocolos; aplicar uno a un lote | VETERINARIO, ADMIN |
| `/insumos` | listar con proyección, configurar, compras y corrección de compras | ADMIN |

| Recurso | Operaciones | Roles |
|---|---|---|
| `/alimentacion` | `GET /opciones` (con ración sugerida), registrar (idempotente), historial, eliminar | ENCARGADO (registrar), ADMIN |
| `/pesadas` | listar (con conversión), registrar, eliminar la última | ADMIN |
| `/sanidad` | eventos (programar, realizar, no programado, reprogramar), bajas | VETERINARIO, ADMIN (eliminar baja: solo ADMIN) |
| `/clima` | clima y riesgos; consultar, buscar y guardar la ubicación | Todos (ubicación: ADMIN) |
| `/dashboard`, `/costos/:id`, `/alertas` | consultas | ADMIN (alertas también VETERINARIO) |
| `/reportes` | informe de lote (PDF), reporte mensual (PDF), planilla (Excel) | ADMIN |
| `/parametros` | consultar, modificar | ADMIN |

En total: **54 operaciones** sobre **41 rutas**, todas documentadas en OpenAPI.

## 14.6 Autenticación y autorización

- **Autenticación:** el usuario envía email y contraseña a `POST /api/auth/login`. La contraseña se compara con el hash **bcrypt** guardado (RNF04) y, si es correcta, la API emite un **token JWT** firmado (HS256) que incluye el id y el rol del usuario.
- **Expiración por inactividad (RNF04):** el token vence a las **8 horas**. Mientras el usuario sigue trabajando, cada respuesta incluye un token renovado (encabezado `X-Token-Renovado`) que el frontend guarda automáticamente. Si pasan 8 horas sin actividad, el token vence y el sistema pide iniciar sesión nuevamente.
- **Autorización por rol (RBAC, RF08):** cada ruta declara los roles permitidos con el middleware `permitir(...)`. El frontend también oculta las opciones no permitidas, pero la restricción real está en el servidor: un encargado que intente llamar a `/api/lotes` recibe `403`.
- **Usuario desactivado:** en cada petición se verifica que el usuario siga activo, de modo que desactivarlo corta el acceso de inmediato aunque tenga un token vigente.

## 14.7 Consideraciones de seguridad

***Tabla 35.*** *Consideraciones de seguridad*

| Riesgo | Medida aplicada |
|---|---|
| Robo de contraseñas desde la base | Hash bcrypt (10 rondas); nunca se devuelven en la API |
| Fuerza bruta en el login | Límite de 20 intentos cada 15 minutos por IP (`express-rate-limit`); mensaje genérico que no revela si el email existe |
| Acceso indebido entre roles | RBAC en el servidor en cada ruta; pruebas automáticas CP-05 a CP-08 |
| Inyección SQL | Consultas parametrizadas de Prisma; `$queryRaw` con plantillas parametrizadas |
| Datos inválidos o maliciosos | Validación de todo dato de entrada con Zod; límite de 100 kB por petición |
| Cabeceras HTTP inseguras | `helmet` (CSP, X-Frame-Options, HSTS, etc.) |
| Peticiones desde otros sitios | CORS restringido al origen del frontend |
| Exposición de credenciales | Secretos en variables de entorno (`.env`, excluido de Git); `.env.example` sin valores reales |

| Riesgo | Medida aplicada |
|---|---|
| Intercepción de datos en tránsito | HTTPS obligatorio en producción, provisto por la plataforma de despliegue (RNF04) |
| Integridad ante errores de la aplicación | Restricciones CHECK, claves foráneas y trigger en la base de datos como segunda barrera |
| Cambio de contraseña por un tercero con la sesión abierta | Se exige la contraseña actual, con el mismo límite de intentos que el login |
| Reenvíos duplicados desde el celular | Identificador de operación único (idempotencia) |
| Límite de intentos evadido detrás del proxy de la plataforma | `trust proxy` configurado para que el límite se aplique a la IP real del cliente |
| Exposición de la ubicación del establecimiento a terceros | El navegador no consulta el servicio de clima directamente: lo hace la API, que solo envía coordenadas |
| Datos guardados en el celular | La cola sin conexión guarda solo las cargas pendientes (lote, fecha y kg) y las quita al sincronizarlas. Al cerrar sesión se borran del dispositivo el token y los datos del usuario |

## 14.8 Manejo de errores y validación de datos

- **Validación en dos niveles:**
    1. En el **frontend**, campos obligatorios y tipos de entrada numéricos, como ayuda al usuario.
    2. En el **backend**, con esquemas Zod. Es la validación que realmente cuenta.

  Las reglas de negocio que dependen de los datos guardados se validan en los servicios: stock suficiente, fecha posterior a la última pesada, cantidad de bajas ≤ cabezas actuales, lote activo.
- **Errores tipados:** los servicios lanzan `AppError(status, mensaje)` y un middleware central (`manejarErrores.js`) los transforma en respuestas uniformes:

```
{ "error": "Datos inválidos",
  "detalles": [{ "campo": "pesoObjetivo", "mensaje": "El peso objetivo debe ser mayor al peso de ingreso" }] }
```

*Tabla 36. Manejo de errores y validación de datos*

| Código | Cuándo |
|---|---|
| 400 | Datos inválidos o regla de negocio violada (fecha futura, baja mayor a las cabezas) |
| 401 | Sin sesión, token vencido o usuario desactivado |
| 403 | El rol no tiene permiso |
| 404 | Recurso inexistente |
| 409 | Conflicto: stock insuficiente, registro duplicado, lote cerrado |
| 500 | Error inesperado (se registra en el log del servidor; al cliente se le muestra un mensaje genérico) |

- **En la interfaz:** el texto del error se muestra junto al formulario. Si se pierde la conexión, se informa "No se pudo conectar con el servidor", y si la sesión expira, se redirige al login.

## 14.9 Escalabilidad y mantenimiento

- **Nuevos módulos (RNF05):** agregar, por ejemplo, el módulo "cría" o "identificación por caravana" implica crear una carpeta en `modules/`, sus modelos en `schema.prisma` y una migración. Los módulos existentes no se modifican.

- **Crecimiento de datos:** índices en las columnas de búsqueda frecuente (`lote.estado`, `registro_alimentacion(lote_id, fecha)`, `evento_sanitario(estado, fecha_programada)`). Con ~5 lotes activos y 2 registros diarios, el volumen anual ronda los 4.000 registros, muy por debajo de la capacidad de PostgreSQL.
- **Rendimiento (RNF03):** el dashboard se resuelve con una única consulta agregada a la vista SQL. Medido en el recorrido de prueba del 15/10/2026: **≈ 1,1 s** de carga completa del dashboard (incluido el clima) y **≈ 170 ms** para confirmar la ración del día. Ambos valores están dentro de lo exigido (3 s y 2 s, respectivamente). El frontend divide el código por pantalla (*lazy loading*): el código común pesa ≈ 117 kB comprimidos y la pantalla del encargado agrega solo 3,4 kB. Una vez instalada, la app abre desde el teléfono sin volver a descargarse.
- **Servicios externos:** el clima se guarda en caché 30 minutos por ubicación. Así, con cualquier cantidad de usuarios, se hacen como máximo unas 50 consultas diarias a Open-Meteo, muy por debajo de su límite gratuito.
- **Escalado horizontal:** la API no guarda estado de sesión (JWT), por lo que pueden ejecutarse varias instancias detrás de un balanceador si fuera necesario.
- **Mantenibilidad (RNF06):**
  - Estructura de carpetas estándar.
  - Funciones de cálculo comentadas.
  - ESLint y Prettier.
  - Pruebas automatizadas que se ejecutan en cada cambio (CI).
  - Migraciones versionadas.

## 14.10 Atributos de calidad

***Tabla 37.*** *Atributos de calidad*

| Atributo | Objetivo | Cómo lo logra la arquitectura | Verificación |
|---|---|---|---|
| Seguridad | Acceso restringido por rol, credenciales protegidas | JWT + RBAC en el servidor, bcrypt, helmet, rate limit, validación | CP-01 a CP-09 |
| Integridad de datos | Costos y stock siempre consistentes | Transacciones, FK, CHECK, trigger, unicidad por lote/insumo/día | CP-13, CP-14, CP-37 |
| Usabilidad | Encargado: ≤ 3 clics desde el login, uso en el celular y a pleno sol | Pantalla única por rol, ración precargada (1 toque), botones de 48 px o más, modo pleno sol, diseño responsive | Recorrido E2E en 390 px |
| Rendimiento | Dashboard < 3 s, registro < 2 s | Vista agregada, índices, *code splitting*, caché del clima | Medición E2E: 1,1 s / 0,17 s |
| Mantenibilidad | Fácil de modificar y extender | Módulos con capas, funciones puras documentadas, ESLint, CI con cobertura | 118 pruebas automáticas; 88 % de cobertura de líneas en el backend |
| Disponibilidad | ≥ 95 %; carga sin señal | API sin estado en plataforma gestionada, PWA con cola sin conexión, degradación elegante si falla el clima | E2E sin conexión; CP-71, CP-76, CP-77 |
| Escalabilidad | Agregar módulos sin rediseño | Modularidad por dominio, migraciones aditivas | En la versión final se agregaron 3 módulos (ración, protocolos, clima) sin cambiar la arquitectura ni las capas |

## 14.11 Herramientas y tecnologías

***Tabla 38.*** *Herramientas y tecnologías*

| Tecnología | Finalidad en el proyecto | Justificación de la elección | Componente | Beneficios |
|---|---|---|---|---|
| **React 19** | Construir la interfaz de usuario | Biblioteca de UI más difundida, basada en componentes reutilizables; mucha documentación y comunidad | Frontend | Reutilización de componentes (diálogos, tarjetas), interfaz reactiva sin recargar la página |
| **Vite** | Servidor de desarrollo y empaquetado | Arranque e inicio casi instantáneos; configuración mínima | Frontend | Recarga en caliente, *build* optimizado con división de código |
| **Material UI (MUI) 7** | Componentes visuales y diseño responsive | Componentes accesibles y probados (formularios, tablas, diálogos); sistema de grillas para celular | Frontend | Interfaz profesional y consistente sin diseñar cada componente desde cero |
| **React Router** | Navegación entre pantallas | Estándar de facto para SPA en React | Frontend | Rutas protegidas por rol |
| **Axios** | Comunicación HTTP con la API | Interceptores para agregar el token y renovarlo automáticamente | Frontend | Manejo centralizado de sesión y errores |
| **Recharts** | Gráficos de evolución de peso y de stock proyectado | Gráficos declarativos en React, livianos | Frontend | Visualización de la GDP frente al objetivo y del quiebre de stock |
| **vite-plugin-pwa (Workbox)** | Aplicación instalable y *service worker* | Genera el manifiesto y el *service worker* a partir de la configuración de Vite, con estrategias de caché probadas | Frontend | La app se instala en el celular y abre sin conexión |
| **idb-keyval** | Cola de cargas sin conexión | Interfaz mínima sobre IndexedDB, el almacenamiento persistente del navegador (≈ 600 bytes) | Frontend | Las cargas sobreviven al cierre de la app o del teléfono |
| **@fontsource (Bitter, Nunito Sans)** | Tipografías del estilo "Campo cálido" | Fuentes incluidas en la app, sin depender de servicios externos | Frontend | Funcionan sin conexión y sin rastreo de terceros |
| **sharp** | Generación de los íconos de la app a partir del logo | Procesamiento de imágenes rápido desde un script de Node | Frontend (herramienta) | Íconos de 64 a 512 px y versión *maskable* consistentes con el logo |
| **Node.js** | Entorno de ejecución del backend | Mismo lenguaje (JavaScript) que el frontend; ideal para APIs de E/S | Backend | Un solo lenguaje en todo el proyecto |
| **Express 5** | Framework de la API REST | Minimalista y muy documentado; v5 maneja errores asíncronos automáticamente | Backend | Middlewares para seguridad, validación y errores |
| **Zod** | Validación de datos de entrada | Esquemas declarativos con mensajes en español | Backend | Datos garantizados antes de llegar a la lógica de negocio |
| **jsonwebtoken + bcryptjs** | Autenticación y hash de contraseñas | Estándares de la industria (JWT, bcrypt) exigidos por RNF04 | Backend | Sesiones sin estado, contraseñas protegidas |
| **helmet, cors, express-rate-limit** | Seguridad HTTP | Mitigan ataques comunes con configuración mínima | Backend | Cumplimiento de buenas prácticas OWASP |

| Tecnología | Finalidad en el proyecto | Justificación de la elección | Componente | Beneficios |
|---|---|---|---|---|
| **pdfkit** | Informes en PDF | Genera el PDF en el servidor sin navegador ni servicios externos; permite dibujar tablas y gráficos | Backend | Informes de lote, de cierre y mensual con el estilo del sistema |
| **exceljs** | Planilla Excel para el contador | Crea archivos .xlsx reales con formatos de moneda y fecha, y varias hojas | Backend | El contador abre los datos directamente en Excel |
| **Open-Meteo** | Pronóstico, lluvia histórica y búsqueda de localidades | Servicio público gratuito y sin clave de acceso, con datos para Argentina | Backend (servicio externo) | Clima y riesgos sin costo ni registro |
| **morgan** | Registro de peticiones | Formato estándar (*combined*) en producción | Backend | Diagnóstico de errores en el servidor publicado |
| **PostgreSQL 17** | Persistencia de datos | Base relacional robusta y gratuita; soporta CHECK, vistas, triggers y transacciones | Base de datos | Integridad y consistencia de la información |
| **Prisma ORM 6** | Acceso a datos y migraciones | Define el modelo en un archivo, genera las migraciones SQL versionadas y consultas seguras | Backend / BD | Menos código repetido, esquema versionado en Git, prevención de inyección SQL |
| **Docker / Docker Compose** | Ejecutar PostgreSQL en desarrollo | Misma base de datos en cualquier PC con un solo comando | Infraestructura | Entorno reproducible, sin instalar Postgres manualmente |
| **Git + GitHub** | Control de versiones | Estándar de la industria; ramas, *pull requests*, *tags* de versión | Todo el proyecto | Historial de cambios, trazabilidad commit ↔ issue |
| **GitHub Projects** | Gestión y seguimiento de tareas | Integrado con el repositorio: issues, tablero Kanban, *milestones* | Gestión | Trazabilidad requisito → tarea → commit (ver sección 19) |
| **Vitest + Supertest** | Pruebas unitarias y de integración | Rápido, compatible con módulos ES; Supertest prueba la API real contra una base de pruebas | Backend y frontend | 118 pruebas automáticas que verifican los RF, con reporte de cobertura (v8) |
| **fake-indexeddb** | Pruebas de la cola sin conexión | Simula IndexedDB en Node | Frontend (pruebas) | La cola se prueba sin navegador |
| **Playwright** | Pruebas end-to-end en el navegador | Automatiza el recorrido completo con los tres roles, simula la pérdida de conexión, descarga los reportes y genera capturas y videos | Frontend + Backend | Verificación de la integración completa, del modo sin conexión y del diseño responsive |
| **ESLint + Prettier** | Calidad y formato del código | Detectan errores y unifican el estilo | Todo el código | Código legible y homogéneo |
| **GitHub Actions** | Integración continua y mantenimiento | Ejecuta lint, pruebas con cobertura y *build* en cada *push* y *pull request*; un flujo programado mantiene activa la API en horario de uso | Repositorio | Detecta errores antes de integrarlos a `main` |
| **Swagger UI (OpenAPI 3)** | Documentación de la API | Estándar para describir APIs REST; permite probar los endpoints desde el navegador | Backend | API autodocumentada en `/api/docs` |
| **Markdown + Mermaid** | Documentación del proyecto y diagramas | Texto plano versionable junto al código; diagramas como código | Documentación | Documentación siempre sincronizada con el sistema |

| Tecnología | Finalidad en el proyecto | Justificación de la elección | Componente | Beneficios |
|---|---|---|---|---|
| **VS Code** | Entorno de desarrollo | Extensiones para JavaScript, Prisma, ESLint y Git | Desarrollo | Productividad |
| **Vercel** | Publicación del frontend | Especializado en aplicaciones estáticas; CDN global, HTTPS y despliegue automático desde GitHub en el plan gratuito | Producción | La app carga rápido desde el campo y se actualiza sola con cada versión |
| **Render** | Publicación de la API | Ejecuta Node.js desde el repositorio con una configuración versionada (`render.yaml`); plan gratuito con HTTPS | Producción | Infraestructura como código: la configuración del servidor queda en Git |
| **Neon** | Base de datos PostgreSQL gestionada | PostgreSQL estándar (el mismo que en desarrollo), plan gratuito con historial de restauración | Producción | Sin administrar un servidor de base de datos |

### Integración de las herramientas en el proceso de desarrollo

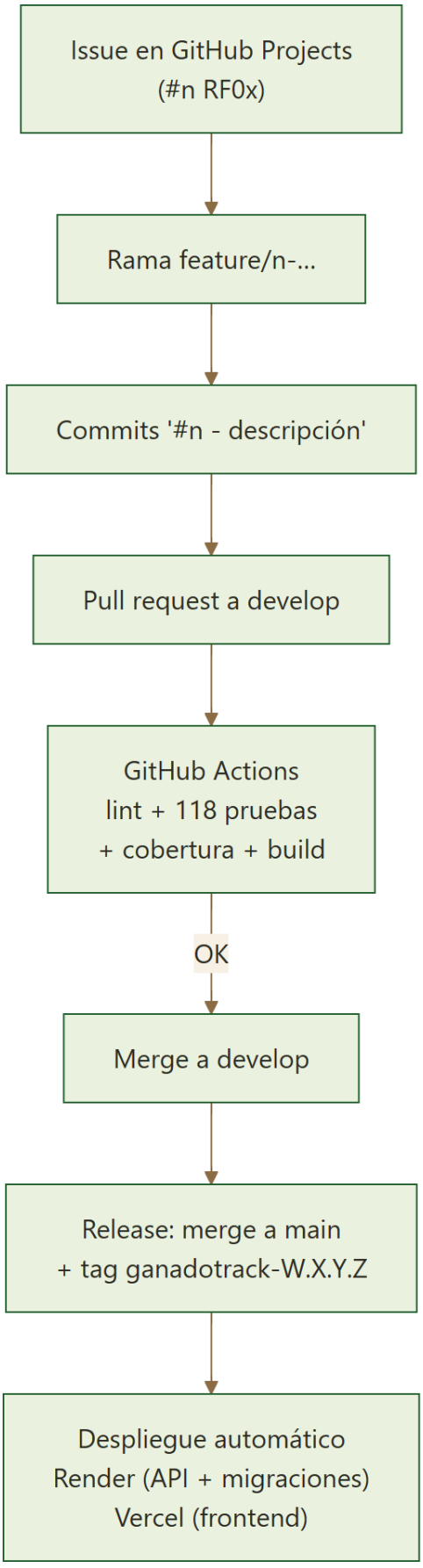


***Figura 12.*** *Integración de las herramientas en el proceso de desarrollo*

## 14.12 Despliegue

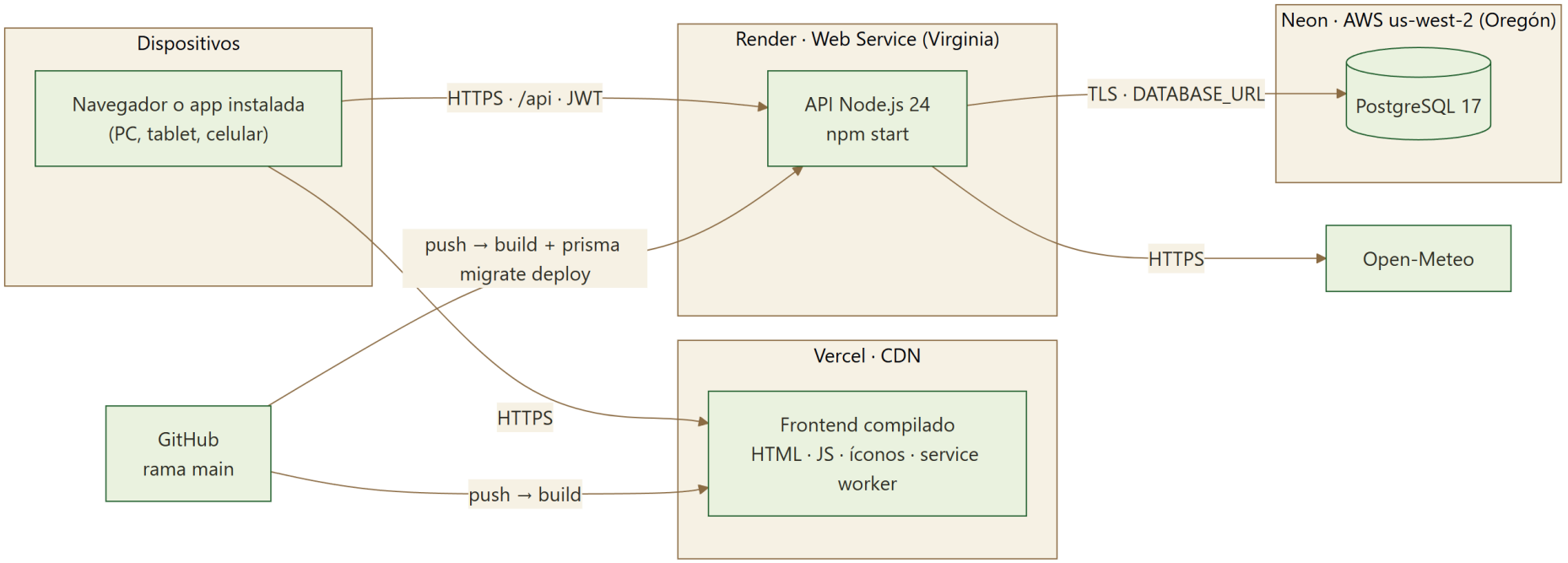



***Figura 13.*** *Diagrama de despliegue en producción*

***Tabla 39.*** *Despliegue*

| Componente | Plataforma | Configuración versionada | Variables (cargadas en la plataforma, nunca en Git) |
|---|---|---|---|
| Frontend | Vercel | `frontend/vercel.json`: *rewrite* de todas las rutas a la SPA y caché del *service worker* | `VITE_API_URL` |
| API | Render | `render.yaml`: Node 24, *build* con migraciones, *health check* en `/api/health`, región Virginia | `DATABASE_URL`, `CORS_ORIGIN`, `SEED_PASSWORD` (`JWT_SECRET` lo genera Render) |
| Base de datos | Neon | Migraciones de Prisma | — |

- La API (Virginia) y la base (Oregón) quedaron en regiones distintas de EE. UU.; cada consulta suma unos 65 ms de ida y vuelta. Medido en producción, el dashboard responde en ≈ 1,25 s, dentro del límite de RNF03 (3 s). Si hiciera falta, se puede mover la API a Oregón cambiando `region` en `render.yaml` y volviendo a crear el servicio.
- El plan gratuito de Render "duerme" la API tras 15 minutos sin uso. La primera visita tarda unos 50 s en despertarla. Para evitarlo en horario de trabajo, el flujo `mantener-activa.yml` de GitHub Actions consulta `/api/health` cada 14 minutos en días hábiles.
- Cada *push* a `main` vuelve a publicar ambos componentes y aplica las migraciones pendientes.

# 15. Modelo de datos

El modelo se presenta en tres niveles: **conceptual** (entidades del dominio, §1), **lógico** (diagrama entidad-relación con atributos, claves y cardinalidades, §2 a §4) y **físico** (implementación en PostgreSQL, §5).

## 15.1 Modelo conceptual: entidades principales

***Tabla 40.*** *Modelo conceptual: entidades principales*

| Entidad | Representa | Requerimientos |
|---|---|---|
| USUARIO | Persona que usa el sistema, con su rol | RF08 |
| LOTE | Tropa de animales que ingresa al feedlot | RF01 |
| INSUMO | Suplemento proteico o silaje de sorgo, con su stock | RF03 |
| COMPRA_INSUMO | Compra (o carga de silaje propio) que suma stock | RF03 |
| REGISTRO_ALIMENTACION | Kg de un insumo entregados a un lote en un día | RF02 |
| PESADA | Pesada mensual del lote con la GDP calculada | RF04 |
| EVENTO_SANITARIO | Evento del calendario sanitario: programado o realizado | RF05 |
| BAJA | Muerte de animales del lote, con su causa | RF05 |
| PARAMETRO | Valores configurables (rendimiento al gancho, días de aviso, cobertura de stock, ubicación) | RF04, RF05, RF06, RF11, RF12 |
| RACION | Kg de un insumo por cabeza y por día para un lote, desde una fecha (historial) | RF09 |
| PROTOCOLO_SANITARIO | Plan sanitario tipo que se aplica al crear un lote | RF10 |
| PROTOCOLO_ITEM | Evento del protocolo: qué se aplica, a los cuántos días del ingreso y cada cuánto se repite | RF10 |

Las tres últimas entidades se agregaron en la versión final. Además, INSUMO incorporó el **porcentaje de materia seca** (RF09) y REGISTRO_ALIMENTACION, el **identificador de operación** para la carga sin conexión (RF14).

## 15.2 Modelo lógico: diagrama entidad-relación

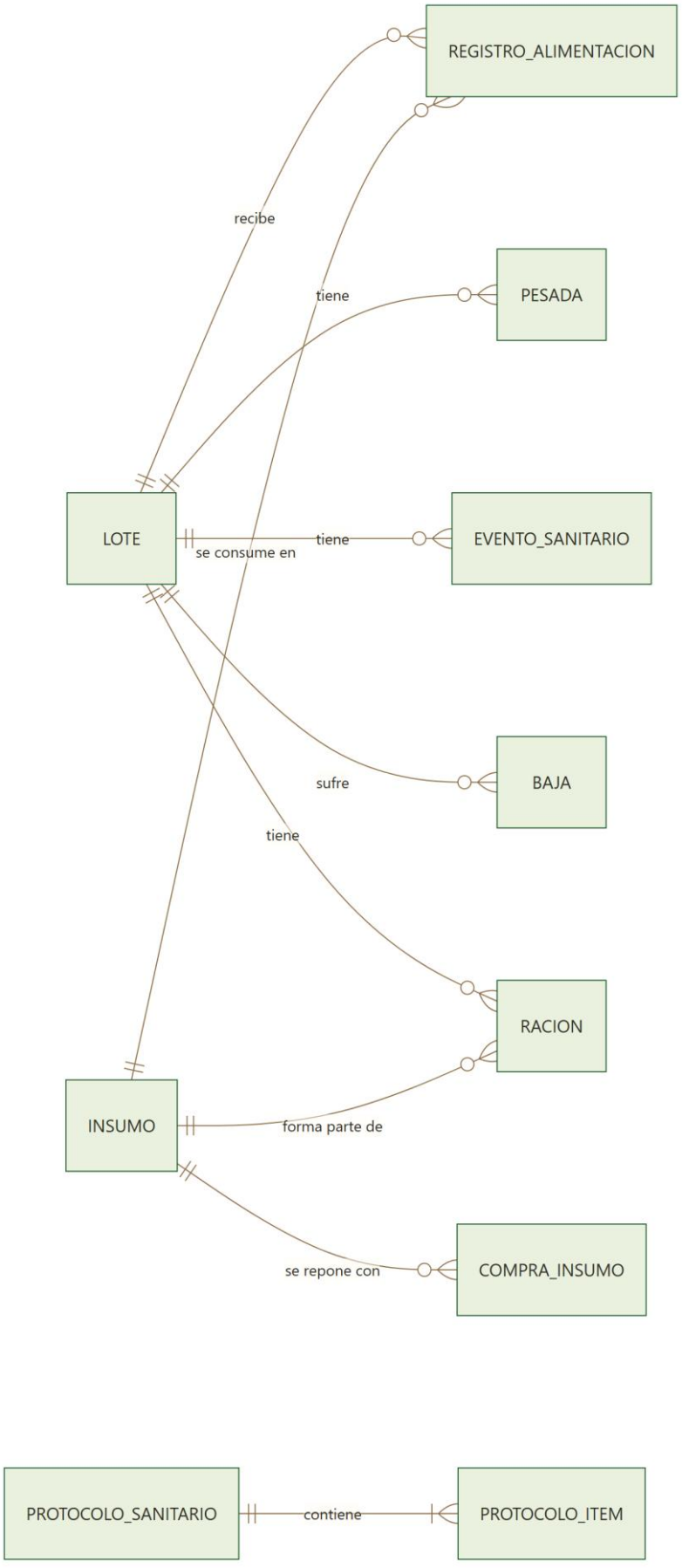


***Figura 14.*** *Diagrama entidad-relación: entidades y relaciones de negocio*

> *PROTOCOLO_SANITARIO no tiene una relación con LOTE ni con EVENTO_SANITARIO. Al aplicarlo, el sistema **copia** sus ítems como eventos pendientes del lote. Así, si el veterinario modifica el protocolo después, los calendarios ya generados no cambian, y cada evento se puede reprogramar en forma individual.*

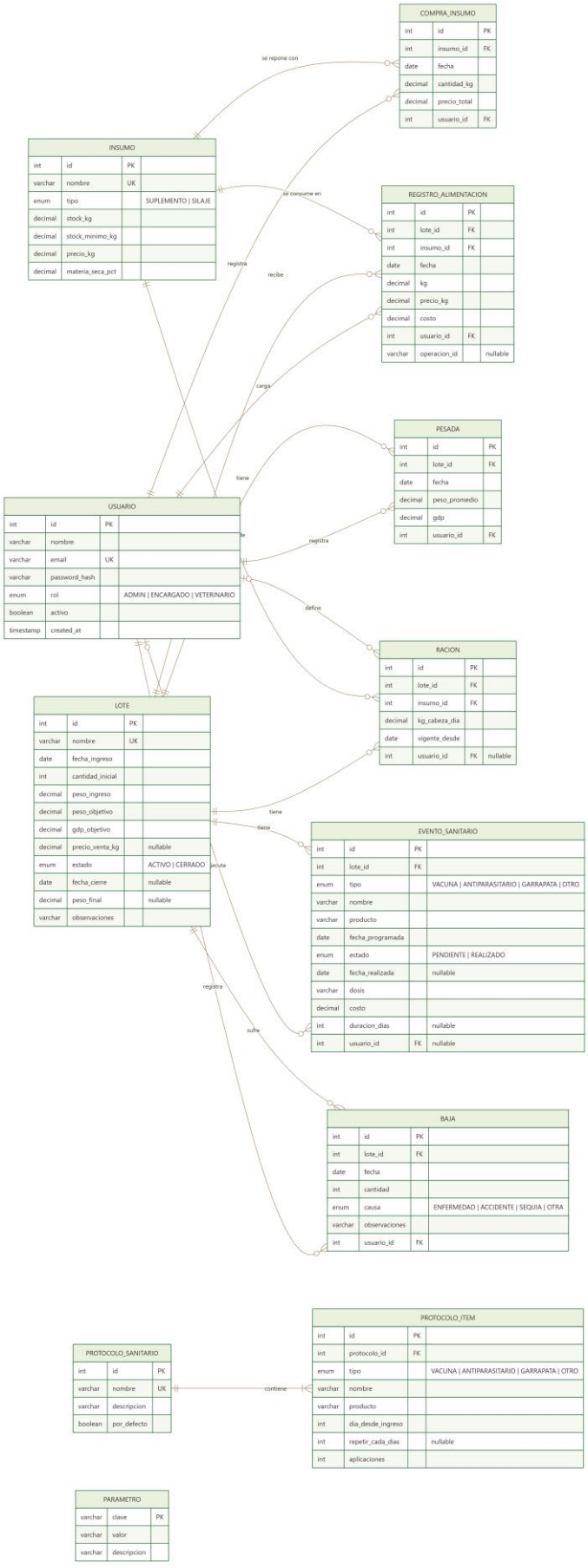



***Figura 15.*** *Diagrama entidad-relación completo, con los atributos de cada entidad*

### Cardinalidades

*Tabla 41. Cardinalidades*

| Relación | Cardinalidad | Lectura |
|---|---|---|
| LOTE – REGISTRO_ALIMENTACION | 1 : N | Un lote recibe muchos registros diarios; cada registro es de un lote |
| INSUMO – REGISTRO_ALIMENTACION | 1 : N | Cada registro corresponde a un insumo |
| LOTE – PESADA | 1 : N | Un lote tiene una pesada mensual (0 al ingresar) |
| LOTE – EVENTO_SANITARIO | 1 : N | Calendario sanitario del lote |
| LOTE – BAJA | 1 : N | Un lote puede tener varias bajas |
| INSUMO – COMPRA_INSUMO | 1 : N | Historial de compras del insumo |
| USUARIO – registros | 1 : N | Auditoría: quién cargó cada dato. En EVENTO_SANITARIO es opcional (0..1) porque un evento programado todavía no tiene ejecutor |
| LOTE – RACION | 1 : N | Un lote tiene varias raciones a lo largo del tiempo (una vigente por insumo) |
| INSUMO – RACION | 1 : N | Un insumo forma parte de la ración de varios lotes |
| PROTOCOLO_SANITARIO – PROTOCOLO_ITEM | 1 : N (al menos 1) | Un protocolo contiene uno o más eventos; si se elimina el protocolo, se eliminan sus ítems (cascada) |

**Relaciones N:M resueltas:** LOTE e INSUMO tienen dos relaciones muchos a muchos (un lote consume varios insumos y un insumo alimenta varios lotes):

- **REGISTRO_ALIMENTACION** registra lo que **se entregó**: fecha, kg, precio aplicado y costo.
- **RACION** registra lo que **se planifica entregar**: kg por cabeza por día y fecha de vigencia.

## 15.3 Normalización

El modelo está en **tercera forma normal (3FN)**:

- **1FN:** todos los atributos son atómicos; no hay listas dentro de un campo. Por ejemplo, cada insumo entregado en el día es una fila distinta.
- **2FN:** las tablas usan una clave primaria simple (`id`), por lo que no hay dependencias parciales.
- **3FN:** no hay dependencias transitivas. Los datos derivados (cabezas actuales, costo acumulado, días en feedlot, fecha proyectada) **no se guardan**: se calculan a partir de los datos base.

**Desnormalizaciones deliberadas:**

*Tabla 42. Normalización*

| Campo | Motivo |
|---|---|
| `registro_alimentacion.precio_kg` y `costo` | Guardan el precio **vigente al momento de la carga**. Si el precio del insumo cambia después, el costo histórico no debe cambiar |
| `pesada.gdp` | Se guarda calculada al registrar la pesada. Es un dato histórico que no debe variar si se corrige una pesada posterior |
| `insumo.stock_kg` | Saldo actual, mantenido en las mismas transacciones que lo modifican. Evita recalcular "compras − consumos" en cada consulta |
| Eventos generados por un protocolo | Copian el tipo, el nombre y el producto del ítem: el calendario del lote es independiente de cambios posteriores al protocolo |

Los datos derivados de la versión final tampoco se guardan: los kg del día de la ración, la conversión alimenticia, los días de cobertura y la compra sugerida se calculan en cada consulta.

## 15.4 Diccionario de datos

Convenciones: claves `SERIAL`; importes en **ARS** (`NUMERIC(14,2)`); pesos y cantidades en **kg**; fechas de negocio `DATE`.

### USUARIO (`usuario`)

***Tabla 43.*** *USUARIO (usuario)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | Identificador |
| nombre | VARCHAR(100) | No | | Nombre visible |
| email | VARCHAR(150) | No | UNIQUE | Usuario de inicio de sesión (minúsculas) |
| password_hash | VARCHAR(100) | No | | Hash bcrypt de la contraseña; nunca se guarda en texto plano |
| rol | ENUM Rol | No | ADMIN, ENCARGADO, VETERINARIO | Define los permisos (RF08) |
| activo | BOOLEAN | No | DEFAULT true | Un usuario inactivo no puede iniciar sesión |
| created_at | TIMESTAMP | No | DEFAULT now() | Alta del registro |

### LOTE (`lote`)

***Tabla 44.*** *LOTE (lote)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| nombre | VARCHAR(60) | No | UNIQUE | Nombre o número de tropa |
| fecha_ingreso | DATE | No | | Ingreso al feedlot |
| cantidad_inicial | INT | No | CHECK > 0 | Animales al ingreso |
| peso_ingreso | NUMERIC(7,2) | No | CHECK > 0 | Peso promedio de entrada (kg) |
| peso_objetivo | NUMERIC(7,2) | No | CHECK > peso_ingreso | Peso promedio de salida esperado (kg) |
| gdp_objetivo | NUMERIC(5,3) | No | CHECK > 0 | GDP esperada (kg/día); por debajo genera alerta |
| precio_venta_kg | NUMERIC(12,2) | Sí | CHECK > 0 | Precio de venta por kg al gancho, para el margen |
| estado | ENUM | No | ACTIVO, CERRADO · DEFAULT ACTIVO · índice | |
| fecha_cierre | DATE | Sí | CHECK ≥ fecha_ingreso; obligatorio si CERRADO | Venta/faena |
| peso_final | NUMERIC(7,2) | Sí | | Peso promedio al cierre |
| observaciones | VARCHAR(500) | Sí | | |

## INSUMO (`insumo`)

***Tabla 45.*** *INSUMO (insumo)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| nombre | VARCHAR(80) | No | UNIQUE | |
| tipo | ENUM | No | SUPLEMENTO, SILAJE | |
| stock_kg | NUMERIC(12,2) | No | CHECK ≥ 0 | Stock actual (el silaje se muestra en toneladas) |
| stock_minimo_kg | NUMERIC(12,2) | No | CHECK ≥ 0 | Por debajo genera alerta |
| precio_kg | NUMERIC(12,2) | No | CHECK ≥ 0 | Precio vigente (última compra o costo de producción) |
| materia_seca_pct | NUMERIC(5,2) | No | CHECK > 0 y ≤ 100 · DEFAULT 100 | % de materia seca, para la conversión alimenticia (suplemento ≈ 90, silaje ≈ 35) |

## COMPRA_INSUMO (`compra_insumo`)

***Tabla 46.*** *COMPRA_INSUMO (compra_insumo)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| insumo_id | INT | No | FK → insumo | |
| fecha | DATE | No | índice (insumo_id, fecha) | |
| cantidad_kg | NUMERIC(12,2) | No | CHECK > 0 | |
| precio_total | NUMERIC(14,2) | No | CHECK ≥ 0 | |
| usuario_id | INT | No | FK → usuario | Quién la registró |

## REGISTRO_ALIMENTACION (`registro_alimentacion`)

***Tabla 47.*** *REGISTRO_ALIMENTACION (registro_alimentacion)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| lote_id | INT | No | FK → lote | |
| insumo_id | INT | No | FK → insumo | |
| fecha | DATE | No | UNIQUE (lote_id, insumo_id, fecha) | Un registro por insumo, lote y día |
| kg | NUMERIC(10,2) | No | CHECK > 0 | Kg entregados |
| precio_kg | NUMERIC(12,2) | No | | Precio aplicado (histórico) |
| costo | NUMERIC(14,2) | No | CHECK ≥ 0 | kg × precio_kg |
| usuario_id | INT | No | FK → usuario | Encargado que cargó |
| operacion_id | VARCHAR(36) | Sí | UNIQUE (operacion_id, insumo_id) | UUID generado en el celular para cada carga; evita duplicados al reenviar una carga hecha sin conexión |

## PESADA (`pesada`)

***Tabla 48.*** *PESADA (pesada)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| lote_id | INT | No | FK → lote | |
| fecha | DATE | No | UNIQUE (lote_id, fecha) | |
| peso_promedio | NUMERIC(7,2) | No | CHECK > 0 | |
| gdp | NUMERIC(6,3) | No | | (peso − peso anterior) / días; puede ser negativa |
| usuario_id | INT | No | FK → usuario | |

## EVENTO_SANITARIO (`evento_sanitario`)

***Tabla 49.*** *EVENTO_SANITARIO (evento_sanitario)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| lote_id | INT | No | FK → lote · índice | |
| tipo | ENUM | No | VACUNA, ANTIPARASITARIO, GARRAPATA, OTRO | |
| nombre | VARCHAR(100) | No | | Ej.: "Vacuna clostridial" |
| producto | VARCHAR(100) | Sí | | |
| fecha_programada | DATE | No | índice (estado, fecha_programada) | Base de las alertas |
| estado | ENUM | No | PENDIENTE, REALIZADO | |
| fecha_realizada | DATE | Sí | Obligatoria si REALIZADO | |
| dosis | VARCHAR(60) | Sí | | |
| costo | NUMERIC(14,2) | No | CHECK ≥ 0 · DEFAULT 0 | Suma al costo sanitario |
| duracion_dias | INT | Sí | CHECK > 0 | Duración del tratamiento de garrapata |
| observaciones | VARCHAR(500) | Sí | | |
| usuario_id | INT | Sí | FK → usuario | Quién lo ejecutó |

## BAJA (`baja`)

***Tabla 50.*** *BAJA (baja)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| lote_id | INT | No | FK → lote | |
| fecha | DATE | No | | |
| cantidad | INT | No | CHECK > 0; ≤ cabezas actuales (validado en la API) | |
| causa | ENUM | No | ENFERMEDAD, ACCIDENTE, SEQUIA, OTRA | |
| observaciones | VARCHAR(500) | Sí | | |
| usuario_id | INT | No | FK → usuario | |

## RACION (`racion`)

*Tabla 51. RACION (racion)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| lote_id | INT | No | FK → lote | |
| insumo_id | INT | No | FK → insumo | |
| kg_cabeza_dia | NUMERIC(8,3) | No | CHECK ≥ 0 (la API limita a 80) | Kg por cabeza por día; 0 quita el insumo de la ración desde esa fecha |
| vigente_desde | DATE | No | UNIQUE (lote_id, insumo_id, vigente_desde) | La ración vigente de un insumo es la de mayor `vigente_desde` que no supere la fecha consultada |
| usuario_id | INT | Sí | FK → usuario | Quién la definió |
| created_at | TIMESTAMP | No | DEFAULT now() | |

## PROTOCOLO_SANITARIO (`protocolo_sanitario`)

*Tabla 52. PROTOCOLO_SANITARIO (protocolo_sanitario)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| nombre | VARCHAR(100) | No | UNIQUE | Ej.: "Protocolo feedlot estándar" |
| descripcion | VARCHAR(500) | Sí | | |
| por_defecto | BOOLEAN | No | DEFAULT false · índice único parcial `WHERE por_defecto` | Se propone al crear un lote; solo uno puede estar marcado |
| created_at | TIMESTAMP | No | DEFAULT now() | |

## PROTOCOLO_ITEM (`protocolo_item`)

*Tabla 53. PROTOCOLO_ITEM (protocolo_item)*

| Campo | Tipo | Nulo | Restricciones | Descripción |
|---|---|---|---|---|
| id | SERIAL | No | PK | |
| protocolo_id | INT | No | FK → protocolo_sanitario (ON DELETE CASCADE) · índice | |
| tipo | ENUM | No | VACUNA, ANTIPARASITARIO, GARRAPATA, OTRO | Mismo tipo que EVENTO_SANITARIO |
| nombre | VARCHAR(100) | No | | Nombre del evento a generar |
| producto | VARCHAR(100) | Sí | | Producto previsto |
| dia_desde_ingreso | INT | No | CHECK entre 0 y 365 | Primera aplicación: fecha de ingreso + días |
| repetir_cada_dias | INT | Sí | CHECK > 0 | Frecuencia de repetición |
| aplicaciones | INT | No | DEFAULT 1 · CHECK entre 1 y 52; si es mayor a 1, `repetir_cada_dias` es obligatorio | Cantidad total de aplicaciones |

**Ejemplo:** el ítem "Control de garrapata", día 21, cada 21 días, 4 aplicaciones, genera eventos en los días 21, 42, 63 y 84 desde el ingreso.

### PARAMETRO (`parametro`)

*Tabla 54. PARAMETRO (parametro)*

| Clave | Valor inicial | Uso |
|---|---|---|
| rendimiento_gancho | 0.57 | kg de carne = peso × cabezas × rendimiento (RF06) |
| dias_aviso_sanitario | 7 | Anticipación de las alertas sanitarias (RF05) |
| dias_cobertura_minima | 7 | Por debajo, alerta de stock proyectado (RF11) |
| horizonte_compra_dias | 14 | Días de consumo que cubre la compra sugerida (RF11) |
| ubicacion_nombre, ubicacion_latitud, ubicacion_longitud | Pergamino, Buenos Aires (−33,89; −60,57) en los datos de demostración | Localidad del establecimiento para el clima (RF12) |

## 15.5 Modelo físico: implementación en PostgreSQL

El esquema se define en `backend/prisma/schema.prisma` y se crea con migraciones SQL versionadas en `backend/prisma/migrations/`:

*Tabla 55. Modelo físico: implementación en PostgreSQL*

| Migración | Contenido |
|---|---|
| `…_init` | Tablas, tipos ENUM, claves primarias y foráneas, índices y restricciones UNIQUE (generada por Prisma) |
| `…_checks_y_vistas` | **Escrita a mano:** restricciones CHECK, vista `v_resumen_lote` y trigger de lote cerrado |
| `…_vista_resumen_tipos` | Ajusta los tipos de la vista (las sumas de enteros devuelven `bigint`) |
| `…_racion_y_materia_seca` | Tabla `racion` y campo `insumo.materia_seca_pct`, con sus CHECK (RF09) |
| `…_protocolos_sanitarios` | Tablas `protocolo_sanitario` y `protocolo_item`, CHECK de días y repeticiones, e índice único parcial `protocolo_unico_por_defecto` (RF10) |
| `…_operacion_offline` | **Escrita a mano:** campo `registro_alimentacion.operacion_id` e índice único (operacion_id, insumo_id) para la sincronización sin duplicados (RF14) |

Todas las migraciones son **aditivas**: agregan tablas, columnas o restricciones sin borrar datos. Por eso la base de producción se actualiza con `prisma migrate deploy` al publicar cada versión.

### Índice único parcial del protocolo por defecto

```
CREATE UNIQUE INDEX protocolo_unico_por_defecto ON protocolo_sanitario (por_defecto) WHERE
por_defecto;
```

Solo indexa las filas con `por_defecto = true`, de modo que la base rechaza un segundo protocolo marcado por defecto. La API, al marcar uno nuevo, desmarca el anterior en la misma transacción.

### Vista `v_resumen_lote`

Concentra los acumulados de cada lote que usan el dashboard y el módulo de costos: cabezas actuales, peso de la última pesada, última GDP, costo de alimentación, costo sanitario y costo total. Evita repetir estas consultas en distintos lugares del código.

### Trigger `fn_validar_lote_activo`

Antes de insertar en `registro_alimentacion`, `pesada` o `baja`, verifica que el lote no esté CERRADO. La API ya valida esta regla; el trigger es una segunda barrera por si se intenta cargar datos por otra vía (verificado por la prueba CP-37).

### Seguridad de la información

- Contraseñas almacenadas solo como hash bcrypt.
- Las credenciales de la base de datos están en variables de entorno, no en el código.
- Se registra el usuario que realizó cada operación (auditoría).
- **Respaldo:** en producción se usan los respaldos automáticos de la plataforma. En local, el respaldo se hace con `docker exec ganadotrack-db pg_dump -U ganadotrack ganadotrack > respaldo.sql`.

## 15.6 Consultas de ejemplo

**a) Resumen de los lotes activos** (lo que muestra el dashboard):

```
SELECT nombre, cabezas_actuales, peso_actual, ultima_gdp, costo_total
FROM v_resumen_lote
WHERE estado = 'ACTIVO'
ORDER BY fecha_ingreso;
```

**b) Costo por kg de carne y margen de cada lote** (RF06):

```
SELECT r.nombre,
       r.costo_total,
       ROUND(r.peso_actual * r.cabezas_actuales * 0.57, 2)                    AS kg_carne,
       ROUND(r.costo_total / NULLIF(r.peso_actual * r.cabezas_actuales * 0.57, 0), 2) AS
costo_por_kg,
       ROUND(l.precio_venta_kg * r.peso_actual * r.cabezas_actuales * 0.57 - r.costo_total, 2) AS
margen
FROM v_resumen_lote r
JOIN lote l ON l.id = r.lote_id;
```

**c) Eventos sanitarios que vencen en los próximos 7 días o ya vencidos** (alertas, RF05):

```
SELECT l.nombre AS lote, e.nombre AS evento, e.fecha_programada,
       e.fecha_programada - CURRENT_DATE AS dias_restantes
FROM evento_sanitario e
JOIN lote l ON l.id = e.lote_id
WHERE e.estado = 'PENDIENTE' AND l.estado = 'ACTIVO'
  AND e.fecha_programada <= CURRENT_DATE + 7
ORDER BY e.fecha_programada;
```

**d) Insumos por debajo del stock mínimo** (RF03):

```
SELECT nombre, stock_kg, stock_minimo_kg FROM insumo WHERE stock_kg < stock_minimo_kg;
```

**e) Consumo y costo de alimentación por lote e insumo en el último mes:**

```
SELECT l.nombre, i.nombre AS insumo, SUM(r.kg) AS kg, SUM(r.costo) AS costo
FROM registro_alimentacion r
JOIN lote l ON l.id = r.lote_id
JOIN insumo i ON i.id = r.insumo_id
WHERE r.fecha >= CURRENT_DATE - 30
GROUP BY l.nombre, i.nombre
ORDER BY l.nombre, i.nombre;
```

**f) Bajas por causa** (relacionado con el caso de anaplasmosis del relevamiento):

```
SELECT causa, SUM(cantidad) AS animales FROM baja GROUP BY causa ORDER BY animales DESC;
```

**g) Ración vigente hoy y kg del día por lote** (RF09):

```
SELECT DISTINCT ON (r.lote_id, r.insumo_id)
       v.nombre AS lote, i.nombre AS insumo, r.kg_cabeza_dia,
       ROUND(r.kg_cabeza_dia * v.cabezas_actuales) AS kg_del_dia
FROM racion r
JOIN v_resumen_lote v ON v.lote_id = r.lote_id AND v.estado = 'ACTIVO'
JOIN insumo i ON i.id = r.insumo_id
WHERE r.vigente_desde <= CURRENT_DATE
ORDER BY r.lote_id, r.insumo_id, r.vigente_desde DESC;
```

*(Las filas con 0 kg indican que el insumo se quitó de la ración.)*

**h) Cargas sincronizadas desde el celular** (RF14):

```
SELECT fecha, operacion_id, COUNT(*) AS insumos, SUM(costo) AS costo
FROM registro_alimentacion
WHERE operacion_id IS NOT NULL
GROUP BY fecha, operacion_id
ORDER BY fecha DESC;
```

Estas consultas se pueden ejecutar con `docker exec -it ganadotrack-db psql -U ganadotrack` o desde Prisma Studio (`npm run db:studio`).

# 16. Módulos implementados

El sistema se organiza en módulos funcionales. Cada uno responde a uno o más requerimientos y existe en las tres capas de la arquitectura: una o más pantallas en el frontend, un módulo en `backend/src/modules/` (rutas → controlador → servicio → esquema) y sus tablas en la base de datos. Los cálculos que comparten varios módulos están en los servicios de dominio de `backend/src/services/` (sección 14.3).

***Tabla 56.*** *Módulos implementados, requerimientos que cubren y componentes en cada capa*

| Módulo | Qué hace | RF | Pantallas (rol) | Backend | Tablas |
|---|---|---|---|---|---|
| Autenticación y usuarios | Inicio de sesión con JWT, roles, alta y baja de usuarios, perfil propio | RF08 | Login, Usuarios (admin), Mi perfil (todos) | `auth`, `usuarios` | `usuario` |
| Lotes | Alta con protocolo, indicadores (días, cabezas, GDP, proyección), edición, cierre con resumen | RF01 | Lotes y Detalle del lote (admin) | `lotes` | `lote` |
| Alimentación | Registro diario con descuento de stock y costo en una transacción; carga sin conexión idempotente | RF02, RF14 | Comida de hoy (encargado, admin) | `alimentacion` | `registro_alimentacion` |
| Ración y conversión | Ración por lote con historial, kg del día sugeridos, conversión alimenticia | RF09 | Raciones (veterinario, admin), Comida de hoy | `racion` | `racion` |
| Insumos y proyección | Stock, compras y su corrección, días de cobertura, fecha de quiebre, compra sugerida | RF03, RF11 | Insumos y stock (admin) | `insumos` | `insumo`, `compra_insumo` |
| Pesadas | Registro de pesadas, GDP, proyección al peso objetivo, eliminación de la última | RF04 | Detalle del lote (admin) | `pesadas` | `pesada` |
| Sanidad | Calendario, eventos realizados, garrapata con control automático, bajas | RF05 | Sanidad (veterinario, admin) | `sanidad` | `evento_sanitario`, `baja` |
| Protocolos sanitarios | Plantillas de calendario y su aplicación a lotes | RF10 | Protocolos (veterinario, admin), Nuevo lote | `protocolos` | `protocolo_sanitario`, `protocolo_item` |
| Costos, dashboard y alertas | Costo por kg, margen, dashboard, alertas por rol | RF06, RF07 | Dashboard y Detalle del lote (admin), Sanidad (alertas del veterinario) | `reportes` (`/dashboard`, `/costos`, `/alertas`) | vista `v_resumen_lote` |
| Clima y riesgos | Clima, pronóstico, ITH y riesgos de garrapata y sequía, con caché | RF12 | Dashboard, barra superior, Configuración | `clima` | `parametro` (ubicación) |
| Reportes | Informe de lote y de cierre en PDF, reporte mensual, planilla Excel | RF13 | Reportes y Detalle del lote (admin) | `reportes` | todas (solo lectura) |

| Módulo | Qué hace | RF | Pantallas (rol) | Backend | Tablas |
|---|---|---|---|---|---|
| Configuración | Parámetros: rendimiento al gancho, días de aviso, cobertura mínima, ubicación | RF04–RF06, RF11, RF12 | Configuración (admin) | `parametros` | `parametro` |

## 16.1 Integración entre módulos

Los módulos no trabajan aislados: varias operaciones atraviesan más de uno, siempre dentro de una transacción para que los datos queden consistentes.

- **Alimentación → Insumos:** cada registro descuenta el stock y toma el precio vigente para el costo del día. Si un insumo no alcanza, se revierte todo el registro (CP-14).
- **Ración → Alimentación:** la pantalla del encargado propone los kg del día calculados con la ración vigente y las cabezas actuales (CP-53).
- **Lotes → Protocolos → Sanidad:** al crear un lote con protocolo, sus eventos se generan en la misma transacción; si algo falla, no se crea el lote (CP-60 a CP-62).
- **Sanidad → Lotes y Costos:** las bajas reducen las cabezas actuales y los eventos realizados suman el costo sanitario del lote.
- **Clima → Alertas:** el riesgo alto de garrapata genera alertas solo para los lotes sin tratamiento en los últimos 21 días (CP-70).
- **Pesadas → Alertas y Cierre:** una GDP baja genera una alerta; una pesada que alcanza el peso objetivo sugiere el cierre (CP-24 y CP-25).

## 16.2 Reglas de negocio relevantes

***Tabla 57.*** *Reglas de negocio y dónde se garantizan*

| Regla | Dónde se garantiza |
|---|---|
| El stock nunca queda negativo | Servicio (transacción) + CHECK en la base |
| Un solo registro por lote, insumo y día | Índice único en la base |
| No se cargan datos en un lote cerrado | Servicio + trigger `fn_validar_lote_activo` |
| La fecha de una pesada debe ser posterior a la anterior | Servicio (validación) |
| El peso objetivo debe superar al peso de entrada | Esquema Zod + CHECK |
| Solo un protocolo puede ser el protocolo por defecto | Índice único parcial |
| Una carga sin conexión reenviada no se duplica | Índice único (`operacion_id`, `insumo_id`) |
| Un evento realizado no se puede modificar | Servicio |
| Cada rol accede solo a sus funciones | Middleware `permitir(...)` en cada ruta |

# 17. Descripción de la API

- **Base:** `/api` (en local: `http://localhost:4000/api`; en producción: `https://ganadotrack-api.onrender.com/api`)
- **Formato:** JSON · fechas `AAAA-MM-DD` · pesos en kg · importes en ARS. Los reportes devuelven archivos PDF o Excel.
- **Documentación interactiva:** `/api/docs` (Swagger UI). La especificación OpenAPI 3 está en `backend/src/docs/openapi.yaml`: 41 rutas y 54 operaciones.
- **Autenticación:** encabezado `Authorization: Bearer <token>`, obtenido con `POST /auth/login`. El token vence a las 8 h sin actividad; las respuestas pueden traer un token renovado en `X-Token-Renovado`.

## 17.1 Endpoints

Los marcados con ★ se agregaron en la versión final.

***Tabla 58.*** *Endpoints*

| Método | Ruta | Rol | Descripción | CU |
|---|---|---|---|---|
| POST | `/auth/login` | Público | Inicia sesión; devuelve el token y el usuario | CU01 |
| GET | `/auth/me` | Todos | Usuario de la sesión | CU01 |
| PATCH | `/auth/perfil` ★ | Todos | Cambia el nombre propio | CU22 |
| PATCH | `/auth/password` ★ | Todos | Cambia la contraseña propia (exige la actual) | CU22 |
| GET | `/usuarios` | ADMIN | Lista los usuarios | CU02 |
| POST | `/usuarios` | ADMIN | Crea un usuario | CU02 |
| PATCH | `/usuarios/:id` | ADMIN | Modifica nombre, rol, estado o contraseña | CU02 |
| GET | `/lotes?estado=` | ADMIN | Lista los lotes con indicadores | CU12 |
| POST | `/lotes` | ADMIN | Crea un lote y aplica el protocolo elegido ★ (`protocoloId`, `omitirEventosPasados`) | CU03, CU17 |
| GET | `/lotes/:id` | ADMIN | Detalle del lote con indicadores | CU11 |
| PATCH | `/lotes/:id` | ADMIN | Edita el lote o carga el precio de venta | CU11 |
| POST | `/lotes/:id/cierre` | ADMIN | Cierra el lote (`simular: true` = vista previa) | CU13 |
| GET | `/raciones` ★ | VETERINARIO, ADMIN | Lotes activos con su ración vigente y los kg del día | CU15 |
| GET | `/lotes/:loteId/racion` ★ | VETERINARIO, ADMIN | Ración vigente e historial de un lote | CU15 |
| PUT | `/lotes/:loteId/racion` ★ | VETERINARIO, ADMIN | Define la ración desde una fecha (0 kg quita el insumo) | CU15 |
| GET | `/protocolos` ★ | VETERINARIO, ADMIN | Lista los protocolos con sus eventos | CU17 |
| POST | `/protocolos` ★ | VETERINARIO, ADMIN | Crea un protocolo | CU17 |
| PUT | `/protocolos/:id` ★ | VETERINARIO, ADMIN | Modifica un protocolo y sus eventos | CU17 |
| DELETE | `/protocolos/:id` ★ | VETERINARIO, ADMIN | Elimina un protocolo (los lotes ya creados no cambian) | CU17 |

| Método | Ruta | Rol | Descripción | CU |
|---|---|---|---|---|
| POST | `/lotes/:loteId/aplicar-protocolo` ★ | VETERINARIO, ADMIN | Genera los eventos de un protocolo en un lote existente | CU17 |
| GET | `/insumos` | ADMIN | Stock de insumos con su proyección ★ (cobertura, quiebre, compra sugerida, curva a 30 días) | CU05, CU18 |
| PATCH | `/insumos/:id` | ADMIN | Configura el stock mínimo, el precio y el % de materia seca ★ | CU06 |
| GET / POST | `/insumos/:id/compras` | ADMIN | Historial / nueva compra o carga | CU06 |
| PUT | `/insumos/:id/compras/:compraId` ★ | ADMIN | Corrige una compra y ajusta el stock | CU06 |
| GET | `/alimentacion/opciones` | ENCARGADO, ADMIN | Lotes activos, insumos, lo ya cargado hoy y la ración sugerida ★ | CU04, CU16 |
| POST | `/alimentacion` | ENCARGADO, ADMIN | Registra la alimentación y descuenta el stock; idempotente con `operacionId` ★ | CU04, CU16, CU21 |
| GET | `/alimentacion?loteId=&desde=&hasta=` | ADMIN | Historial | — |
| DELETE | `/alimentacion/:id` | ADMIN | Elimina un registro erróneo y devuelve el stock | — |
| GET | `/pesadas?loteId=` | ADMIN | Pesadas del lote con la conversión alimenticia ★ | CU07 |
| POST | `/pesadas` | ADMIN | Registra una pesada, calcula la GDP y la proyección | CU07 |
| DELETE | `/pesadas/:id` ★ | ADMIN | Elimina la última pesada del lote | CU07 |
| GET | `/sanidad/lotes` | VETERINARIO, ADMIN | Lotes activos con cabezas actuales | — |
| GET | `/sanidad/eventos?loteId=&estado=` | VETERINARIO, ADMIN | Calendario sanitario | CU08 |
| POST | `/sanidad/eventos` | VETERINARIO, ADMIN | Programa un evento | CU08 |
| PATCH | `/sanidad/eventos/:id` ★ | VETERINARIO, ADMIN | Reprograma o corrige un evento pendiente | CU08 |
| POST | `/sanidad/eventos/:id/realizar` | VETERINARIO, ADMIN | Registra un evento programado como realizado | CU09 |
| POST | `/sanidad/eventos/realizados` | VETERINARIO, ADMIN | Registra un evento no programado | CU09 |
| DELETE | `/sanidad/eventos/:id` | VETERINARIO, ADMIN | Elimina un evento pendiente | CU08 |
| GET / POST | `/sanidad/bajas` | VETERINARIO, ADMIN | Lista / registra bajas | CU10 |
| DELETE | `/sanidad/bajas/:id` ★ | ADMIN | Elimina una baja cargada por error | CU10 |
| GET | `/clima` ★ | Todos | Clima actual, pronóstico de 7 días, lluvia de 30 días y riesgos | CU19 |
| GET / PUT | `/clima/ubicacion` ★ | ADMIN | Consulta / guarda la ubicación del establecimiento | CU19 |
| GET | `/clima/buscar?q=` ★ | ADMIN | Busca localidades por nombre | CU19 |
| GET | `/dashboard` | ADMIN | Resumen, lotes activos, alertas y stock | CU12 |
| GET | `/costos/:id` | ADMIN | Costos con desglose, conversión ★ y rentabilidad | CU11 |

| Método | Ruta | Rol | Descripción | CU |
|---|---|---|---|---|
| GET | `/alertas` | ADMIN, VETERINARIO | Alertas vigentes (el veterinario solo ve las sanitarias y las de clima sanitario) | CU14 |
| GET | `/reportes/lotes/:id/informe.pdf` ★ | ADMIN | Informe del lote (de cierre si está cerrado) | CU20 |
| GET | `/reportes/mensual.pdf?mes=AAAA-MM` ★ | ADMIN | Reporte mensual | CU20 |
| GET | `/reportes/planilla.xlsx?desde=&hasta=` ★ | ADMIN | Planilla Excel para el contador | CU20 |
| GET | `/parametros` · PUT `/parametros/:clave` | ADMIN | Rendimiento, días de aviso, cobertura mínima ★ y horizonte de compra ★ | — |
| GET | `/health` | Público | Estado de la API y de la base de datos | — |

## 17.2 Ejemplo: ración sugerida para el encargado (CU16)

`GET /api/alimentacion/opciones` (fragmento):

```
{
  "fecha": "2026-10-15T00:00:00.000Z",
  "insumos": [
    { "id": 55, "nombre": "Suplemento proteico 18%", "tipo": "SUPLEMENTO", "stockKg": 1500,
"precioKg": 480 },
    { "id": 56, "nombre": "Silaje de sorgo (propio)", "tipo": "SILAJE", "stockKg": 180000,
"precioKg": 45 }
  ],
  "lotes": [{
    "id": 124, "nombre": "Lote 01 - Angus novillos", "cabezasActuales": 118,
    "racionSugerida": [
      { "insumoId": 55, "kgCabezaDia": 1.5, "vigenteDesde": "2026-08-17T00:00:00.000Z", "kgDia":
177 },
      { "insumoId": 56, "kgCabezaDia": 15,  "vigenteDesde": "2026-08-17T00:00:00.000Z", "kgDia":
1770 }
    ],
    "cargadoHoy": []
  }]
}
```

`kgDia` = `kgCabezaDia` × `cabezasActuales` (1,5 × 118 = 177). El botón "Confirmar ración de hoy" envía esos kg en un `POST /alimentacion`.

## 17.3 Ejemplo: registrar alimentación (CU04 / CU21)

```
POST /api/alimentacion
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{ "loteId": 124, "items": [ { "insumoId": 55, "kg": 177 }, { "insumoId": 56, "kg": 1770 } ],
  "operacionId": "5f0c2a8e-3b7d-4c1e-9a52-1d2e3f4a5b6c", "fecha": "2026-10-15" }
```

`operacionId` y `fecha` son opcionales. Los envía la app cuando sincroniza una carga hecha sin conexión: la fecha es la de captura.

Respuesta `201 Created`:

```
{
  "lote": { "id": 124, "nombre": "Lote 01 - Angus novillos" },
  "fecha": "2026-10-15T00:00:00.000Z",
  "registros": [
    { "id": 331, "insumoId": 55, "kg": 177, "precioKg": 480, "costo": 84960, "insumo": {
"nombre": "Suplemento proteico 18%" } },
    { "id": 332, "insumoId": 56, "kg": 1770, "precioKg": 45, "costo": 79650, "insumo": {
"nombre": "Silaje de sorgo (propio)" } }
```

```
  ],
  "costoDia": 164610,
  "alertasStock": ["Suplemento proteico 18% quedó por debajo del stock mínimo (1.323 kg)"]
}
```

Si se reenvía la misma `operacionId`, la respuesta es `200 OK` con `"repetido": true` y los registros ya guardados: no se duplica nada ni se descuenta el stock otra vez.

Respuesta `409 Conflict` (stock insuficiente; no se guarda nada):

```
{ "error": "Stock insuficiente de Suplemento proteico 18%: hay 1.323 kg disponibles" }
```

## 17.4 Ejemplo: clima y riesgos (CU19)

`GET /api/clima` (fragmento; el pronóstico trae 7 días):

```
{
  "configurado": true, "disponible": true,
  "ubicacion": { "nombre": "Pergamino, Buenos Aires", "latitud": -33.89101, "longitud": -60.57462
},
  "actual": { "temperatura": 13, "humedad": 93, "viento": 12.3, "lluvia": 0, "texto":
"Despejado",
              "ith": 55.5, "estresTermico": "normal" },
  "pronostico": [{ "fecha": "2026-10-15", "maxima": 19.3, "minima": 11.3, "lluvia": 4.4,
                   "texto": "Lluvia", "ith": 66.1, "estresTermico": "normal" }],
  "riesgos": {
    "estresTermico": { "nivel": "normal", "dia": "2026-10-15" },
    "garrapata": { "nivel": "bajo", "temperaturaMedia7": 13.5, "humedadMedia7": 81 },
    "sequia": { "nivel": "bajo", "lluvia30": 72.2, "lluviaPronostico7": 14.1 }
  }
}
```

Si el servicio externo no responde, la respuesta es `200` con `"disponible": false`, y el resto del sistema sigue funcionando.

## 17.5 Ejemplo: indicadores de un lote (`GET /api/lotes/2`)

```
{
  "id": 2, "nombre": "Lote 01 - Angus novillos", "estado": "ACTIVO",
  "fechaIngreso": "2026-08-10T00:00:00.000Z", "cantidadInicial": 120,
  "pesoIngreso": 380, "pesoObjetivo": 420, "gdpObjetivo": 0.45, "precioVentaKg": 9800,
  "indicadores": {
    "diasEnFeedlot": 59, "cabezasActuales": 118, "bajas": 2,
    "pesoActual": 405, "ultimaGdp": 0.423, "bajoRendimiento": true,
    "diasRestantes": 36, "fechaEstimadaObjetivo": "2026-11-11T00:00:00.000Z",
    "costoAlimentacion": 9876600, "costoSanitario": 750000, "costoTotal": 10626600,
    "rendimientoGancho": 0.57, "kgCarneEstimados": 27240.3,
    "costoPorKgCarne": 390.11, "margenBruto": 256328340, "calculoPreliminar": false
  }
}
```

## 17.6 Códigos de error

***Tabla 59.*** *Códigos de error*

| Código | Significado |
|---|---|
| 400 | Datos inválidos (con `detalles` por campo), regla de negocio violada o contraseña actual incorrecta |
| 401 | Sin sesión, token vencido o usuario desactivado |
| 403 | El rol no tiene permiso para la operación |
| 404 | Recurso o ruta inexistente |
| 409 | Conflicto: stock insuficiente, duplicado, lote cerrado, evento ya realizado, solo se puede eliminar la última pesada |
| 429 | Demasiados intentos de login o de cambio de contraseña |
| 500 | Error interno |

# 18. Estrategia de pruebas y validación

## 18.1 Estrategia

***Tabla 60.*** *Estrategia*

| Tipo de prueba | Qué verifica | Herramienta | Cantidad | Ubicación |
|---|---|---|---|---|
| **De componente (unitarias)** | Funciones puras del backend: GDP, proyección, cabezas, kg de carne, costo por kg, margen, ración, materia seca, conversión, cobertura y compra sugerida; alertas; fechas de los protocolos; índices de clima | Vitest | 42 | `backend/tests/unit/` |
| **De componente (frontend)** | Cola de cargas sin conexión: guardado, orden de envío, corte de señal, rechazos y concurrencia | Vitest + fake-indexeddb | 6 | `frontend/src/offline/` |
| **De integración** | API + reglas de negocio + base de datos real (PostgreSQL de prueba): autenticación, roles, transacciones, triggers, idempotencia, reportes; el servicio de clima se simula | Vitest + Supertest | 70 | `backend/tests/integration/` |
| **Funcionales end-to-end** | Recorrido completo en el navegador con los 3 roles, en escritorio y celular, con descarga de reportes; pérdida y recuperación de la conexión | Playwright (Microsoft Edge) | 15 + 3 pasos | `e2e/recorrido.mjs`, `e2e/offline.mjs` |
| **De aceptación** | Criterios de aceptación de cada rol, verificados con los escenarios E2E; la confirmación del productor queda pendiente | Escenarios E2E + videos + cuestionario | 5 criterios | Sección 5 |

**Proceso:**

1. Las pruebas unitarias y de integración se ejecutan con `npm test` en `backend/` y en `frontend/`; `npm run test:coverage` agrega el reporte de cobertura.
2. **GitHub Actions** ejecuta lint, pruebas con cobertura y *build* en cada *push* y *pull request*. Una rama no se integra a `develop` si alguna prueba falla (por ejemplo, el PR #30 se integró recién con la CI en verde).
3. Las pruebas de integración usan una base separada (`ganadotrack_test`) que se vacía antes de cada prueba, de modo que son repetibles y no afectan los datos de desarrollo.
4. Las pruebas no dependen de internet: la respuesta de Open-Meteo se simula con datos fijos (`vi.stubGlobal('fetch')`), así los riesgos de clima se prueban con valores conocidos.
5. Los recorridos E2E se ejecutan antes de cada entrega y generan las capturas del manual de usuario (`docs/capturas/`).

**Resultado de la última ejecución (15/10/2026):** 118 de 118 pruebas automatizadas aprobadas (112 del backend + 6 del frontend) · recorridos E2E completos sin errores de consola ni de servidor.

```
backend   Test Files  14 passed (14)      Tests  112 passed (112)
frontend  Test Files   1 passed (1)       Tests    6 passed (6)
```

**Cobertura del backend** (Vitest + v8):

*Tabla 61. Estrategia*

| Alcance | Sentencias | Ramas | Funciones | Líneas |
|---|---|---|---|---|
| Todo el backend | 85,2 % | 72,3 % | 80,0 % | **88,4 %** |
| Servicios de dominio (`src/services`) | — | 86,7 % | 95,1 % | **97,9 %** |

Las líneas sin cubrir corresponden sobre todo a ramas de error poco probables (por ejemplo, fallas de red en el servicio de clima ya cubiertas por el caso general) y a controladores triviales.

## 18.2 Casos de prueba

Los identificadores **CP-xx** coinciden con el nombre de cada prueba automatizada en el código (`it('CP-10 …')`), lo que permite rastrear cada caso hasta su implementación. La columna **Observaciones** registra las incidencias detectadas: cuando un caso se relaciona con un error encontrado durante el desarrollo, remite a su número en la sección 18.3.

Condición inicial común a CP-01 a CP-37: base de pruebas vacía, con un usuario por rol, dos insumos (suplemento: 1.000 kg a $ 400/kg, mínimo 500 kg; silaje: 20.000 kg a $ 40/kg) y un lote de 100 animales ingresado hace 30 días con 380 kg (objetivo 420 kg, GDP objetivo 0,45).

### Autenticación y roles (RF08, RNF04)

*Tabla 62. Autenticación y roles (RF08, RNF04)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-01 | Login (CU01) | Usuario admin activo | Email y contraseña correctos | 200, token JWT y rol ADMIN; no se devuelve el hash | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-02 | Login inválido | Ídem | Contraseña incorrecta | 401 "Email o contraseña incorrectos" | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-03 | Acceso sin sesión | — | Petición sin token / con token falso | 401 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-04 | Contraseñas hasheadas | Usuario creado | — | El campo guardado es un hash bcrypt (`$2…`), distinto a la contraseña | Según lo esperado | ☑ Aprobado | El hash nunca se devuelve en las respuestas de la API |
| CP-05 | RBAC encargado | Sesión de encargado | GET /lotes, /dashboard, /sanidad | 403 en todas; 200 en /alimentacion/opciones | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-06 | RBAC veterinario | Sesión de veterinario | POST /alimentacion, GET /insumos | 403; 200 en /sanidad/eventos | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-07 | Alertas por rol | Stock bajo y evento hoy | GET /alertas con cada rol | Admin: STOCK y SANITARIA; veterinario: solo SANITARIA | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-08 | Usuario desactivado | Encargado con token vigente | El admin lo desactiva | El token deja de funcionar (401) | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-09 | Autodesactivación | Sesión admin | PATCH a sí mismo `activo: false` | 400, no se desactiva | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Alimentación y stock (RF02, RF03 · CU04, CU05, CU06)

***Tabla 63.*** *Alimentación y stock (RF02, RF03 · CU04, CU05, CU06)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-10 | Registrar alimentación | Común | 150 kg suplemento + 1.500 kg silaje | 201; costo del día $ 120.000; stock 850 kg y 18.500 kg | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-11 | Un solo insumo | Común | Suplemento 0, silaje 1.000 | Se guarda 1 registro | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-12 | Sin valores (excepción CU04) | Común | Ambos en 0 | 400, no se guarda nada | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-13 | Registro duplicado del día | Silaje ya cargado hoy | Silaje 1.000 kg otra vez | 409; el stock no cambia | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-14 | Stock insuficiente (atomicidad) | Suplemento 1.000 kg | Silaje 1.000 + suplemento 5.000 | 409 "Stock insuficiente"; **tampoco** se descuenta el silaje | Según lo esperado | ☑ Aprobado | Regla crítica: la transacción revierte también el silaje |
| CP-15 | Alerta de stock mínimo | Mínimo 500 kg | 600 kg de suplemento | 201 con aviso; aparece alerta STOCK | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-16 | Validación de fechas | Común | Fecha futura / anterior al ingreso | 400 en ambos casos | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-17 | Compra de insumo (CU06) | Común | 2.000 kg por $ 1.000.000 | Stock 3.000 kg; precio $ 500/kg | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-18 | Corrección de un registro | Registro de 100 kg | DELETE como admin | 200; el stock vuelve a 1.000 kg | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Lotes, pesadas y costos (RF01, RF04, RF06 · CU03, CU07, CU11)

***Tabla 64.*** *Lotes, pesadas y costos (RF01, RF04, RF06 · CU03, CU07, CU11)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-19 | Validación de lote | — | Peso objetivo 380 < ingreso 400 | 400, error en el campo `pesoObjetivo` | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-20 | Nombre duplicado | Lote existente | Mismo nombre | 409 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-21 | Días en feedlot | Ingreso hace 30 días | GET /lotes/:id | `diasEnFeedlot = 30`; cálculo preliminar | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-22 | Primera pesada | Sin pesadas | 395 kg hoy | GDP 0,5; 50 días restantes; fecha = hoy + 50 | Según lo esperado | ☑ Aprobado | Sin incidencias |

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-23 | Pesada sucesiva | Pesada de 390 kg hace 10 días | 396 kg hoy | GDP 0,6 (calculada desde la anterior) | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-24 | GDP bajo objetivo (alt. A CU07) | Común | 386 kg hoy (0,2 kg/día) | Aviso y alerta GDP en el dashboard | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-25 | Objetivo alcanzado (alt. B CU07) | Común | 421 kg | `alcanzoObjetivo`, sugiere cierre | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-26 | Fecha de pesada inválida | Pesada hace 5 días | Otra pesada en la misma fecha | 400 "posterior a la última pesada" | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-27 | Costos y margen (RF06) | $ 40.000 de alimentación, $ 10.000 de sanidad, 10 bajas, pesada de 400 kg, precio $ 9.000 | GET /costos/:id | Costo $ 50.000; 90 cab.; 20.520 kg carne; $ 2,44/kg; margen $ 184.630.000 | Según lo esperado | ☑ Aprobado | El margen no incluye el valor del animal al ingreso (límite del alcance, §7) |

## Sanidad, bajas y cierre (RF05, RF01, RF07 · CU08–CU10, CU13, CU14)

***Tabla 65.*** *Sanidad, bajas y cierre (RF05, RF01, RF07 · CU08–CU10, CU13, CU14)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-28 | Alerta 7 días antes (RF05) | Eventos a 7 y 8 días | GET /alertas | Solo alerta el de 7 días | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-29 | Programar en el pasado | Común | Fecha de ayer | 400 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-30 | Registrar evento realizado | Evento pendiente | Fecha, dosis, $ 25.000 | REALIZADO; costo sanitario del lote $ 25.000 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-31 | Garrapata (alt. A CU09) | Evento GARRAPATA | Sin duración → con 21 días | 400 sin duración; con duración se programa un control a +21 días | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-32 | Costo omitido (excepción CU09) | Evento pendiente | Sin costo | Guarda $ 0 con advertencia | Según lo esperado | ☑ Aprobado | La advertencia es el comportamiento previsto en la excepción del CU09 |
| CP-33 | Doble registro | Evento ya realizado | Registrar otra vez | 409 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-34 | Baja (CU10) | 100 cabezas | Baja de 95; luego de 6 | Quedan 5; la segunda: 400 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-35 | Simulación de cierre | Lote activo | 421 kg, $ 9.500, `simular` | Resumen (23.997 kg carne); el lote sigue ACTIVO | Según lo esperado | ☑ Aprobado | Sin incidencias |

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-36 | Cierre de lote (CU13) | Evento pendiente | Cierre real | CERRADO + advertencia; alimentación posterior: 409; sin alertas del lote | Según lo esperado | ☑ Aprobado | La advertencia informa el evento pendiente que queda sin realizar |
| CP-37 | Trigger de la base | Lote cerrado | INSERT directo de una pesada (sin la API) | La base rechaza la operación | Según lo esperado | ☑ Aprobado | Segunda barrera: la regla se cumple aunque se saltee la API |

## Pruebas end-to-end y no funcionales (navegador)

***Tabla 66.*** *Pruebas end-to-end y no funcionales (navegador)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-38 | Login inválido en la interfaz | Datos de demo | Contraseña errónea | Mensaje de error visible | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-39 | Carga del encargado en celular (RNF01) | Pantalla de 390 px | Lote 01, 180 kg / 1.800 kg | Confirmación con costo del día; 2 clics (lote + Confirmar) más el tipeo de los kg | 2 clics; confirmación en **127 ms** (RNF03 < 2 s) | ☑ Aprobado | Relacionada con la incidencia 5 (orden de los insumos), corregida |
| CP-40 | Restricción de pantallas | Sesión de encargado | Navegar a /dashboard | Redirige a su pantalla | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-41 | Garrapata desde la interfaz | Sesión de veterinario | Registrar control con 21 días | Aviso "Se programó un control…" | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-42 | Tiempo de carga del dashboard (RNF03) | 3 lotes activos, ~330 registros | Abrir el dashboard | < 3 s | **≈ 970 ms** | ☑ Aprobado | Relacionada con las incidencias 1 (tipos de la vista) y 6 (tamaño del paquete), corregidas |
| CP-43 | Alta, pesada y cierre desde la interfaz | Sesión admin | Lote nuevo; pesada; cierre con resumen | Cada acción confirma y actualiza los indicadores | Según lo esperado | ☑ Aprobado | Relacionada con la incidencia 4 (eje del gráfico de peso), corregida |
| CP-44 | Diseño responsive | Pantalla de 390 px | Dashboard y menú | Sin desplazamiento horizontal; menú desplegable | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Perfil y corrección de datos (RF08, RF03–RF05 · CU22, CU06–CU10)

***Tabla 67.*** *Perfil y corrección de datos (RF08, RF03–RF05 · CU22, CU06–CU10)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-45 | Cambiar el nombre propio (CU22) | Sesión de encargado | Nombre "Juan Encargado" | 200 con el nombre nuevo | Según lo esperado | ☑ Aprobado | Sin incidencias |

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-46 | Cambiar la contraseña (CU22) | Sesión de veterinario | Actual incorrecta; luego la correcta | 400 con la incorrecta; con la correcta, 200 y la nueva sirve para el login | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-47 | Eliminar la última pesada (CU07) | Pesadas hace 10 días y hoy | DELETE de la primera y de la segunda | 409 la primera; 200 la última; queda 1 pesada | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-48 | Eliminar una baja (CU10) | Baja de 3 animales | DELETE como veterinario y como admin | 403 / 200; las cabezas vuelven a 100 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-49 | Reprogramar un evento (CU08) | Evento pendiente a 3 días | Nueva fecha a 10 días; luego editar el realizado | 200 con la fecha nueva; 409 si ya está realizado | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-50 | Corregir una compra (CU06) | Compra de 2.000 kg por $ 1.000.000 | 2.500 kg; luego 100 kg con el stock ya consumido | Stock 3.500 kg y $ 400/kg; la segunda: 409 (no deja stock negativo) | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-51 | Estado del servicio | API y base en funcionamiento | GET /health | `{estado: ok, baseDeDatos: ok}` | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Ración y conversión alimenticia (RF09 · CU15, CU16)

Condición inicial: la común (lote de 100 cabezas; suplemento a $ 400/kg y silaje a $ 40/kg).

***Tabla 68.*** *Ración y conversión alimenticia (RF09 · CU15, CU16)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-52 | Definir la ración (CU15) | Común | 1,5 kg de suplemento y 15 kg de silaje por cabeza | 200; kg del día: 150 y 1.500 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-53 | Ración sugerida y carga en un envío (CU16) | Ración definida | El encargado envía la sugerida | Opciones con 100 cabezas y 2 insumos; 201 con costo $ 120.000 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-54 | Historial de la ración | Ración de 1,5 kg vigente | Cambio a 2 kg desde mañana; luego 0 kg | Hoy rige 1,5 kg con 2 registros de historial; con 0 kg el insumo sale de la ración | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-55 | Permisos de la ración | Sesión de encargado | PUT de la ración | 403 | Según lo esperado | ☑ Aprobado | Sin incidencias |

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-56 | Conversión alimenticia | Silaje con 35 % MS | 2 días de 100 kg de silaje; pesada +1 kg | 70 kg MS / 100 kg ganados = 0,70 (por pesada y acumulada) | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Protocolos sanitarios (RF10 · CU17, CU03)

Protocolo de prueba: clostridial (día 0), refuerzo (día 21) y control de garrapata (día 21, cada 21 días, 3 aplicaciones).

***Tabla 69.*** *Protocolos sanitarios (RF10 · CU17, CU03)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-57 | Crear un protocolo | Sesión de veterinario | Protocolo de prueba; luego como encargado | 201 con 3 ítems; el encargado: 403 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-58 | Único protocolo por defecto | Protocolo por defecto existente | Otro protocolo marcado por defecto | Solo el último queda por defecto | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-59 | Validación de repeticiones | — | Ítem con 3 aplicaciones sin frecuencia | 400 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-60 | Alta de lote con protocolo (CU03 + CU17) | Protocolo de prueba | Lote que ingresa hoy | 5 eventos PENDIENTES en los días 0, 21, 21, 42 y 63 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-61 | Lote cargado con atraso | Protocolo de prueba | Lote que ingresó hace 30 días | Se omiten los eventos pasados: quedan 2 (días 42 y 63) | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-62 | Atomicidad del alta | — | Lote con un protocolo inexistente | 404 y no se crea el lote | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-63 | Aplicar a un lote existente | Lote sin eventos | Aplicar el protocolo de prueba | 201 con 5 eventos; los de hoy aparecen en las alertas | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Proyección de stock (RF11 · CU18)

Condición inicial: la común (suplemento: 1.000 kg, mínimo 500 kg; silaje: 20.000 kg, mínimo 1.000 kg).

*Tabla 70. Proyección de stock (RF11 · CU18)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-64 | Proyección con ración | Ración de 1,5 kg de suplemento (150 kg/día) | GET /insumos | Cobertura 6 días; quiebre hoy + 6; compra sugerida 1.600 kg (150 × 14 + 500 − 1.000); alerta STOCK_PROYECTADO | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-65 | Proyección sin ración | 2 cargas de 700 kg de silaje en los últimos 14 días | GET /insumos | Consumo 100 kg/día (1.400 / 14); cobertura 186 días; sin alerta | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-66 | Cobertura mínima configurable | Ración de CP-64 | Cobertura mínima = 3 días | Sin cobertura baja ni alerta | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Clima y riesgos (RF12 · CU19, CU14)

Clima simulado: máxima 34 °C, media 24 °C, humedad 80 % y sin lluvias en 30 días ni en el pronóstico.

*Tabla 71. Clima y riesgos (RF12 · CU19, CU14)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-67 | Sin ubicación | Ubicación no configurada | GET /clima | `{configurado: false}` | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-68 | Clima y riesgos | Ubicación configurada | GET /clima | Temperatura 28 °C; 7 días de pronóstico; garrapata alto, sequía alto, estrés térmico en emergencia | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-69 | Alertas de clima por rol | Ídem | GET /alertas como admin y veterinario | Admin: estrés, garrapata y sequía; veterinario: estrés y garrapata | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-70 | Garrapata con tratamiento reciente | Tratamiento hace 5 días | GET /alertas | Sin alerta de garrapata para ese lote | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-71 | Servicio de clima caído | El servicio no responde | GET /clima y /alertas | `{disponible: false}`; las alertas responden 200 | Según lo esperado | ☑ Aprobado | Degradación elegante: el resto del sistema sigue funcionando |

## Reportes (RF13 · CU20)

***Tabla 72.*** *Reportes (RF13 · CU20)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-72 | Informe de lote en PDF | "Lote Ñandú 1" con alimentación y pesada | GET informe.pdf | 200, `application/pdf`, archivo `informe-lote-nandu-1.pdf` que empieza con `%PDF` | Según lo esperado | ☑ Aprobado | Relacionada con las incidencias 9 y 10 (filas y pie del PDF), corregidas |
| CP-73 | Reporte mensual | Ídem | Mes actual; mes `2026-13` | PDF válido; 400 para el mes inválido | Según lo esperado | ☑ Aprobado | Relacionada con las incidencias 9 y 10 (filas y pie del PDF), corregidas |
| CP-74 | Planilla Excel | Ídem | GET planilla.xlsx | Hojas Lotes, Alimentación, Compras, Sanidad, Pesadas y Bajas; la fila de alimentación con 500 kg y $ 20.000 | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-75 | Permisos de los reportes | — | Encargado y veterinario | 403 | Según lo esperado | ☑ Aprobado | Relacionada con la incidencia 12 (401 en lugar de 404), corregida |

## Carga sin conexión (RF14, RNF07 · CU21)

***Tabla 73.*** *Carga sin conexión (RF14, RNF07 · CU21)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-76 | Idempotencia del reenvío | Común | La misma `operacionId` dos veces (100 kg + 1.000 kg) | 201 y luego 200 `repetido`; 2 registros; stock de suplemento 900 kg (descontado una sola vez) | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-77 | Fecha de captura | Común | Carga de ayer enviada hoy | Registro con la fecha de ayer | Según lo esperado | ☑ Aprobado | Sin incidencias |

## Pruebas end-to-end de la versión final (navegador)

Ejecutadas el 15/10/2026 sobre los datos de demostración recién cargados.

***Tabla 74.*** *Pruebas end-to-end de la versión final (navegador)*

| ID | Funcionalidad | Condiciones iniciales | Datos de entrada | Resultado esperado | Resultado obtenido | Estado | Observaciones |
|---|---|---|---|---|---|---|---|
| CP-78 | Ración en un toque (CU16, RNF01) | Encargado en 390 px; Lote 01 con ración | "Confirmar ración de hoy" | La tarjeta pasa a "Cargado hoy" en < 2 s | 1 toque; confirmación en **166 ms** | ☑ Aprobado | Sin incidencias |
| CP-79 | Otros kilos (CU16 alt.) | Lote 02 con ración | "Di otros kilos" → modificar → Confirmar | Formulario precargado; registro guardado | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-80 | Alta de lote con protocolo (CU03) | Protocolo por defecto | Nuevo lote desde la interfaz | Aviso "Lote creado · N evento(s) sanitario(s) programado(s)" | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-81 | Eliminar la última pesada (CU07) | Pesada recién registrada | Eliminar y confirmar en el diálogo | Diálogo de confirmación; la pesada desaparece | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-82 | Descarga de reportes (CU20) | Sesión de administrador | Informe de lote y planilla | Descarga de `informe-lote-….pdf` y `ganadotrack-planilla-….xlsx` | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-83 | Perfil y ruta inexistente | Sesión iniciada | "Mi perfil"; URL inexistente | Pantalla de perfil; página 404 con regreso al inicio | Según lo esperado | ☑ Aprobado | Sin incidencias |
| CP-84 | Carga sin conexión de punta a punta (CU21) | App de producción (*preview*) abierta con señal | Cortar la red, recargar, confirmar la ración; restablecer la red | Sin red, la app abre y la carga queda "En el teléfono"; al volver la señal se envía sola | *Service worker* activo; **2 registros** sincronizados, sin duplicados | ☑ Aprobado | Relacionada con las incidencias 7 (rutas del service worker) y 13 (tamaño de la PWA), corregidas |
| CP-85 | Rendimiento del dashboard (RNF03) | 3 lotes activos, clima configurado | Abrir el dashboard | < 3 s | **≈ 1.070 ms** | ☑ Aprobado | Sin incidencias |
| CP-86 | Prueba de humo en producción (`e2e/humo-produccion.mjs`) | App publicada en Vercel + Render + Neon con los datos de demostración | Ingreso con los 3 roles, todas sus pantallas y descarga de PDF y Excel; cualquier escritura en la API se bloquea | Sin errores de consola ni de la API; *service worker* activo; datos sin modificar | Según lo esperado; respuesta del dashboard de la API ≈ 1,25 s | ☑ Aprobado (15/10) | Solo lectura: no modifica los datos de producción |

Las pruebas unitarias (42) verifican con valores calculados a mano cada fórmula documentada. Por ejemplo:

- GDP(380 → 395 kg en 30 días) = 0,5.
- Proyección de 395 a 420 kg con GDP 0,6 = 42 días (se redondea hacia arriba).
- El evento a exactamente 7 días genera alerta y el de 8 días no.

- ITH con 30 °C y 60 % de humedad = 79,8 (nivel peligro).
- La compra sugerida se redondea a múltiplos de 100 kg.
- Un ítem de protocolo "día 21, cada 21 días, 3 aplicaciones" genera los días 21, 42 y 63.

## 18.3 Incidencias detectadas y corregidas durante el desarrollo

***Tabla 75.*** *Incidencias detectadas y corregidas durante el desarrollo*

| # | Incidencia | Detectada por | Corrección |
|---|---|---|---|
| 1 | La vista `v_resumen_lote` devolvía las cabezas como `bigint` (suma de enteros en PostgreSQL), que no se puede convertir a JSON | Prueba manual de la vista | Nueva migración que castea a `int` |
| 2 | Al editar un lote sin enviar "observaciones", el campo se borraba | Revisión de código | La validación distingue "no enviado" de "vacío" |
| 3 | Las reglas de React marcaban `setState` sincrónico en un efecto (riesgo de renders en cascada) | ESLint (react-hooks) | Se reescribió el hook `useCarga` |
| 4 | El gráfico de peso espaciaba las pesadas por igual aunque estuvieran separadas por distinta cantidad de días | Revisión de capturas E2E | Eje X temporal real |
| 5 | En la pantalla del encargado, el silaje aparecía antes que el suplemento, distinto al orden del CU04 | Revisión de capturas E2E | Orden por tipo |
| 6 | El paquete inicial del frontend pesaba más de 1 MB | Build de Vite | Carga diferida por pantalla: el encargado descarga ~180 kB comprimidos |
| 7 | Las reglas de caché del *service worker* no reconocían las rutas de la API: Workbox copia la función de filtro como texto y pierde las constantes definidas fuera de ella | E2E sin conexión (la app no abría los datos sin red) | La lista de rutas se escribió dentro de la propia función |
| 8 | En Windows, regenerar el cliente de Prisma fallaba (EPERM) mientras la API estaba corriendo, porque el archivo del motor estaba en uso | Al agregar la migración de la ración | Procedimiento documentado: detener la API, ejecutar `prisma generate` y volver a iniciarla |
| 9 | En las tablas de los PDF, cada fila ocupaba el doble de alto: pdfkit mueve el cursor al escribir cada celda | Revisión visual de los PDF generados | Las celdas se dibujan con una posición vertical fija por fila |
| 10 | El pie de página del PDF generaba páginas en blanco al escribirse dentro del margen inferior | Revisión visual de los PDF generados | Se anula el margen inferior al dibujar el pie |
| 11 | El recorrido E2E fallaba al buscar un texto que aparecía también en un subtítulo | Recorrido E2E | Búsqueda por texto exacto |
| 12 | Las rutas inexistentes de la API respondían 401 en lugar de 404, porque la autenticación de los reportes se aplicaba a todo `/api` | Revisión de la API | La autenticación se declara en cada ruta de reportes |

| # | Incidencia | Detectada por | Corrección |
|---|---|---|---|
| 13 | El primer *build* de la PWA guardaba 3,1 MB en el teléfono (fuentes de alfabetos que no se usan y el logo en alta resolución) | Build de Vite | Se excluyeron esas fuentes y se generó un logo de 420 px: 1,9 MB |
| 14 | La CI en GitHub fallaba en la instalación: el *lockfile* no era válido para npm 10 (Node 22) y ESLint marcaba un script de Node | Primera ejecución de GitHub Actions | *Lockfile* regenerado para npm 10 y 11, Node 24 en CI y Render, *globals* de Node para los scripts (PR #30) |

## 18.4 Matriz de trazabilidad

***Tabla 76.*** *Matriz de trazabilidad*

| Requerimiento | Casos de uso | Endpoints | Pantalla | Pruebas |
|---|---|---|---|---|
| RF01 Gestión de lotes | CU03, CU13 | `POST/GET/PATCH /lotes`, `POST /lotes/:id/cierre` | Lotes, Detalle del lote | CP-19, 20, 21, 35, 36, 37, 43, 80 |
| RF02 Alimentación diaria | CU04 | `GET /alimentacion/opciones`, `POST /alimentacion` | Comida de hoy | CP-10–14, 16, 18, 39, 79 |
| RF03 Stock de insumos | CU05, CU06 | `GET/PATCH /insumos`, `POST/PUT /insumos/:id/compras` | Insumos y stock | CP-14, 15, 17, 18, 50 |
| RF04 Pesadas y GDP | CU07 | `POST/GET/DELETE /pesadas` | Detalle del lote · Pesadas | CP-22–26, 47, 81, unitarias de GDP/proyección |
| RF05 Sanidad y alertas | CU08, CU09, CU10, CU14 | `/sanidad/eventos…`, `/sanidad/bajas`, `/alertas` | Sanidad | CP-07, 28–34, 41, 48, 49, unitarias de alertas |
| RF06 Costos y rentabilidad | CU11 | `GET /costos/:id`, `PATCH /lotes/:id` | Detalle del lote · Costos | CP-27, 35, unitarias de costos |
| RF07 Dashboard | CU12 | `GET /dashboard` | Dashboard | CP-24, 42, 44, 85 |
| RF08 Usuarios y roles | CU01, CU02, CU22 | `/auth/*`, `/usuarios` | Login, Usuarios, Mi perfil | CP-01–09, 38, 40, 45, 46, 83 |
| RF09 Ración y conversión | CU15, CU16 | `/raciones`, `/lotes/:id/racion`, `GET /alimentacion/opciones`, `GET /pesadas` | Raciones, Comida de hoy, Detalle del lote | CP-52–56, 78, 79, unitarias de ración y conversión |
| RF10 Protocolos sanitarios | CU17, CU03 | `/protocolos`, `POST /lotes`, `/lotes/:id/aplicar-protocolo` | Protocolos, Nuevo lote, Detalle del lote | CP-57–63, 80, unitarias de protocolo |
| RF11 Proyección de stock | CU18, CU14 | `GET /insumos`, `/alertas`, `/parametros` | Insumos y stock | CP-64–66, unitarias de cobertura y compra |
| RF12 Clima y riesgos | CU19, CU14 | `/clima`, `/clima/ubicacion`, `/clima/buscar`, `/alertas` | Dashboard, Configuración, barra superior | CP-67–71, unitarias de riesgos |
| RF13 Reportes | CU20 | `/reportes/*` | Reportes, Detalle del lote | CP-72–75, 82 |
| RF14 App instalable y sin conexión | CU21 | `POST /alimentacion` (`operacionId`, `fecha`) | Comida de hoy | CP-76, 77, 84, pruebas de la cola (frontend) |
| RNF01 Usabilidad | CU04, CU16 | — | Comida de hoy | CP-39, 44, 78 |
| RNF03 Rendimiento | CU04, CU12 | — | — | CP-39, 42, 78, 85 |

| Requerimiento | Casos de uso | Endpoints | Pantalla | Pruebas |
|---|---|---|---|---|
| RNF04 Seguridad | CU01 | todos | — | CP-01–09, 46, 55, 75 |
| RNF07 Operación sin conexión | CU21 | `POST /alimentacion` | Comida de hoy | CP-76, 77, 84 |

## 18.5 Validación de aceptación

Los criterios de aceptación se definieron desde el punto de vista de cada usuario. Se verificaron **internamente**: el alumno ejecutó los escenarios de cada rol con los recorridos E2E sobre los datos de demostración, en escritorio y en un celular de 390 px. La **confirmación del productor** no se completó antes de la entrega, porque depende de su disponibilidad. Para hacerla quedaron preparados tres videos de demostración (encargado en el celular, productor y veterinario), un acceso de prueba a la aplicación publicada y un cuestionario de 8 preguntas (`docs/entregas/validacion-productor.md`).

***Tabla 77.*** *Validación de aceptación*

| Criterio de aceptación | Evidencia | Verificación interna | Confirmación del productor |
|---|---|---|---|
| El encargado carga la alimentación del día desde el celular, sin ayuda | CP-39 y CP-78: un toque por lote en 390 px, confirmación en 166 ms; CP-84: también sin señal | ☑ Cumple | Pendiente |
| El veterinario encuentra los eventos pendientes y registra un tratamiento de garrapata | CP-41: el evento se registra y el sistema programa el próximo control | ☑ Cumple | Pendiente |
| El productor ve el costo por kg de carne y el margen de cada lote | CP-27 (cálculo verificado a mano) y CP-82 (informe PDF del lote) | ☑ Cumple | Pendiente: que confirme que lo entiende |
| Las alertas (evento vencido, stock bajo o por agotarse, GDP baja, clima) le aparecen a quien corresponde | CP-07, CP-24, CP-28, CP-64 y CP-69 | ☑ Cumple | Pendiente: que confirme que le resultan útiles |
| Los valores de referencia son correctos: GDP objetivo, ración, rendimiento 57 %, precios y protocolo sanitario | Tomados del relevamiento (Anexo I); todos son configurables desde el sistema | ⚠ Sin confirmar | Pendiente |

**Conclusión:** el sistema cumple los criterios que se pueden verificar sin el usuario real. Falta que el productor confirme que la información le resulta útil y que valide los valores de referencia, en especial la GDP objetivo: el valor relevado (380 → 420 kg en unos 95 días, ≈ 0,42 kg/día) es bajo para un feedlot. Como la GDP objetivo se carga por lote, corregirla no requiere cambios en el sistema.

## 18.6 Análisis de resultados y mejoras identificadas

- Todas las funcionalidades pasan sus pruebas: 118 automatizadas y 18 pasos E2E. Las reglas críticas están cubiertas por pruebas automáticas y además protegidas en la base de datos: atomicidad del stock, bloqueo de lotes cerrados, permisos por rol, alta de lote con su protocolo e idempotencia de la carga sin conexión.
- Las mejoras identificadas en la 2ª entrega se incorporaron:
  - La cobertura se mide en CI.
  - El frontend tiene sus primeras pruebas automatizadas (la cola sin conexión).

    - El E2E prueba la pérdida de conexión.
- Las pruebas de integración del clima no dependen de internet: los riesgos se verifican con datos simulados. El funcionamiento con el servicio real se comprobó manualmente con la ubicación de demostración.
- **Mejoras pendientes:**
    - Pruebas de componentes React (Testing Library) para las pantallas.
    - Ejecutar los recorridos E2E dentro del pipeline de CI.
    - Pruebas con conexión lenta (simulando la red del campo) y en celulares de gama baja.

# 19. Seguimiento del proyecto (GitHub Projects)

## 19.1 Herramientas

***Tabla 78.*** *Herramientas*

| Necesidad | Herramienta |
|---|---|
| Control de versiones del código y la documentación | **Git**, con el repositorio remoto en **GitHub** |
| Gestión y seguimiento de tareas | **GitHub Projects** (tablero Kanban) + **Issues** |
| Planificación por hitos | **Milestones** de GitHub (uno por fase del proyecto) |
| Verificación automática de cada cambio | **GitHub Actions** (lint + pruebas + build) |
| Identificación de versiones | **Tags** de Git |

Se eligió GitHub Projects porque integra en un mismo lugar el código, las tareas y las versiones. Así, la **trazabilidad** que pide la cátedra (requisito → tarea → código → prueba) se obtiene con enlaces automáticos y sin herramientas externas.

## 19.2 Organización de las tareas

- **Un issue por requerimiento o tarea.** El título lleva el código del requerimiento, por ejemplo: `#7 RF02 – Registro de alimentación diaria`. La descripción incluye los criterios de aceptación tomados del caso de uso.
- **Etiquetas (labels)**, con los colores del logo:
  - Por módulo: `lotes`, `alimentacion`, `insumos`, `pesadas`, `sanidad`, `costos`, `dashboard`, `usuarios`, `clima`, `reportes`, `pwa`, `diseno`, `infra`.
  - Por tipo: `feature`, `bug`, `docs`, `test`.
  - Por prioridad: `prioridad-alta`.
- **Milestones:** uno por fase del proyecto, con su fecha límite:
  - `F0 MVP – ganadotrack-1.0.5.0`
  - `F1 Rediseño y completitud – 1.0.7.0`
  - `F2 Gestión – 1.0.9.0`
  - `F3 Tecnología – 1.1.0.0`
  - `F4 Pruebas, despliegue y validación`
  - `F5 Documentación y entregables`
  - `F6 Cierre – ganadotrack-1.1.1.1`
- **Tablero Kanban** (GitHub Projects): columnas *Backlog* → *Por hacer* → *En progreso* → *En revisión* → *Hecho*. Cuando se integra el *pull request* que dice `Closes #7`, el issue pasa solo a *Hecho*.
- **Numeración:** los issues #1 a #29 se crearon en orden, de modo que el número de cada issue coincide con el que usan los commits. Los números siguientes (#30 en adelante) corresponden a *pull requests*.

### Issues del proyecto

***Tabla 79.*** *Issues del proyecto*

| # | Issue | Milestone | Etiquetas | Estado final |
|---|---|---|---|---|
| 1 | Correcciones a la 1ª entrega | F0 | docs | Cerrado |

| # | Issue | Milestone | Etiquetas | Estado final |
|---|---|---|---|---|
| 2 | Configurar repositorio, CI y entorno Docker | F0 | infra, test | Cerrado |
| 3 | Modelo de datos y migraciones | F0 | feature, infra | Cerrado |
| 4 | RF08 – Autenticación y roles | F0 | usuarios, feature, prioridad-alta | Cerrado |
| 5 | RF01 – Gestión de lotes | F0 | lotes, feature | Cerrado |
| 6 | RF03 – Stock y compras de insumos | F0 | insumos, feature | Cerrado |
| 7 | RF02 – Registro de alimentación diaria | F0 | alimentacion, feature, prioridad-alta | Cerrado |
| 8 | RF04 – Pesadas, GDP y proyección | F0 | pesadas, feature | Cerrado |
| 9 | RF05 – Calendario sanitario, tratamientos y bajas | F0 | sanidad, feature, prioridad-alta | Cerrado |
| 10 | CU14 – Alertas automáticas | F0 | sanidad, feature, prioridad-alta | Cerrado |
| 11 | RF06 – Costos y rentabilidad | F0 | costos, feature, prioridad-alta | Cerrado |
| 12 | RF07 – Dashboard | F0 | dashboard, feature | Cerrado |
| 13 | Pruebas automatizadas y E2E | F0 | test | Cerrado |
| 14 | Validación con el productor | F4 | test | Abierto (trabajo futuro) |
| 15 | Documentación y manuales | F5 | docs | Cerrado |
| 16 | Despliegue | F4 | infra | Cerrado |
| 17 | Presentación final | F5 | docs | Cerrado |
| 18 | Rediseño visual "Campo cálido" | F1 | diseno, feature | Cerrado |
| 19 | Completitud del sistema | F1 | usuarios, feature | Cerrado |
| 20 | RF09 – Ración por lote y carga en 1 toque | F2 | alimentacion, feature, prioridad-alta | Cerrado |
| 21 | RF10 – Protocolos sanitarios automáticos | F2 | sanidad, feature | Cerrado |
| 22 | RF11 – Proyección de stock | F2 | insumos, feature | Cerrado |
| 23 | RF12 – Clima y riesgos | F3 | clima, feature | Cerrado |
| 24 | RF13 – Reportes PDF y Excel | F3 | reportes, feature | Cerrado |
| 25 | RF14 – App instalable y modo sin conexión | F3 | pwa, alimentacion, feature | Cerrado |
| 26 | Preparación de despliegue y validación | F4 | infra, test | Cerrado |
| 27 | Documento final en Word y PDF | F5 | docs, prioridad-alta | Cerrado |
| 28 | Capturas y diagramas finales | F5 | docs | Cerrado |
| 29 | Versión final ganadotrack-1.1.1.1 | F6 | infra | Cerrado |

Cada issue incluye la descripción y sus **criterios de aceptación** como lista de verificación. Al cerrarlo, se deja un comentario con el commit que lo resolvió.

## 19.3 Flujo de ramas

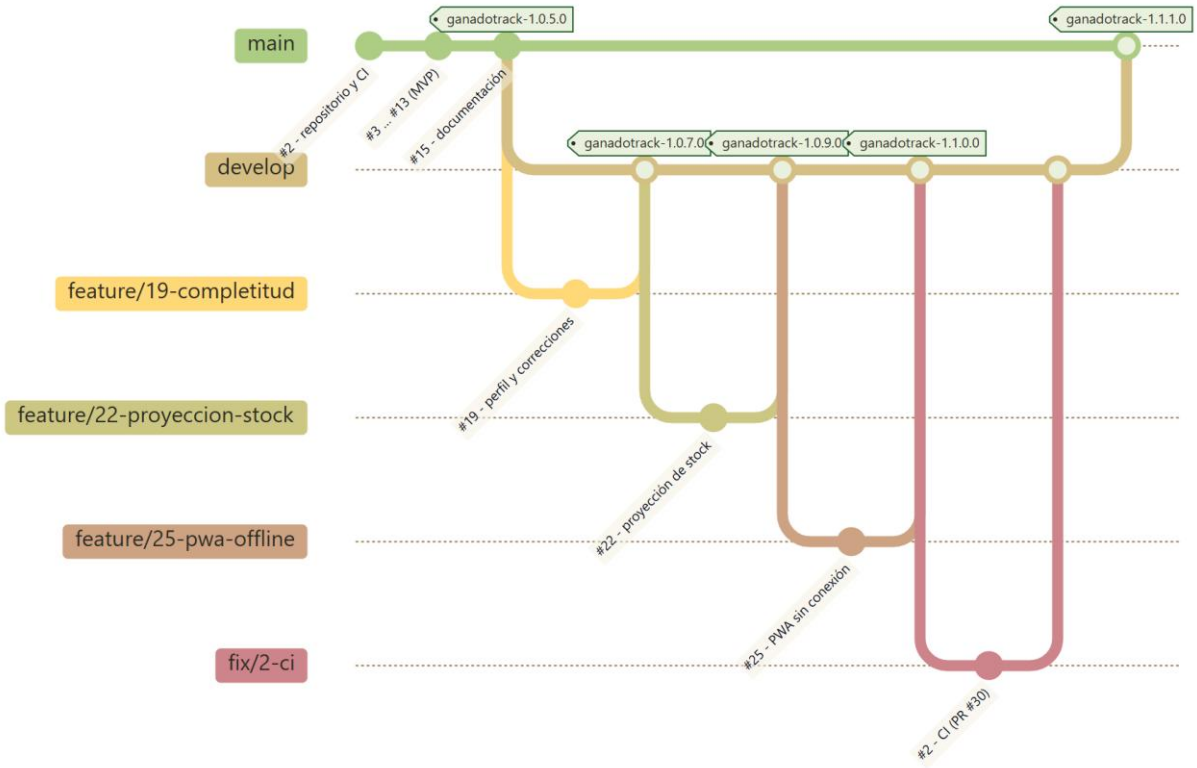



***Figura 16.*** *Flujo de ramas del repositorio*

*El diagrama muestra solo algunas ramas: en el repositorio hay una rama `feature/<n>-…` por cada issue del #18 al #26.*

***Tabla 80.*** *Flujo de ramas*

| Rama | Uso |
|---|---|
| `main` | Solo versiones liberadas. Cada *merge* lleva un tag de versión |
| `develop` | Integración de las funcionalidades terminadas |
| `feature/<n>-descripcion` | Una rama por issue (por ejemplo `feature/21-protocolo-sanitario`); se integra a `develop` con *merge* `--no-ff` o mediante un *pull request* |
| `fix/<n>-descripcion` | Corrección de errores (por ejemplo `fix/2-ci`, PR #30) |

**Reglas:**

- No se hacen commits directos a `main`.
- Todo *pull request* debe pasar el pipeline de GitHub Actions (lint + 118 pruebas con cobertura + build) antes de integrarse.
- Al ser un proyecto individual, la revisión del *pull request* la hace el propio autor con el checklist de la sección 19.6.

## 19.4 Convención de commits (trazabilidad)

Cada commit referencia el issue que resuelve, como se vio en clase:

```
git commit -m "#7 - Registro de alimentación diaria con descuento de stock"
git commit -m "#9 - Reprogramación automática del control de garrapata"
git commit -m "#11 - Fix: el costo por kg descuenta las bajas"
```

GitHub convierte `#7` en un enlace al issue. Desde el issue se ven todos los commits que lo implementaron y desde cada commit se llega al requerimiento. La matriz de trazabilidad completa (RF → CU → endpoint → prueba) está en pruebas.md.

## 19.5 Versionado

Nomenclatura: **`ganadotrack-W.X.Y.Z`**

*Tabla 81. Versionado*

| Dígito | Significado |
|---|---|
| W | Cambio de plataforma o de arquitectura |
| X | Cambios de gran impacto (nuevo módulo mayor) |
| Y | Cambios relevantes para el cliente (nueva funcionalidad visible) |
| Z | Revisiones y correcciones |

Siguiendo la convención de la cátedra, un **valor impar** indica versión liberada (despliegue) y un **valor par**, versión de desarrollo.

*Tabla 82. Versionado*

| Versión | Contenido |
|---|---|
| `ganadotrack-1.0.5.0` | MVP de la 2ª entrega: RF01–RF08 (en `main`) |
| `ganadotrack-1.0.7.0` | Rediseño "Campo cálido" y completitud (F1) |
| `ganadotrack-1.0.9.0` | Ración y carga en 1 toque, protocolos sanitarios, proyección de stock (F2) |
| `ganadotrack-1.1.0.0` | Clima y riesgos, reportes PDF/Excel, PWA sin conexión (F3) |
| `ganadotrack-1.1.1.0` | Versión candidata para la publicación y la validación con el productor (en `main`, 15/10) |
| **`ganadotrack-1.1.1.1`** | **Versión final de la entrega (08/11), con release en GitHub** |
| 1.3.x.x (futuro) | Costos completos, notificaciones, identificación individual |

Los tags se crean con `git tag -a ganadotrack-1.1.1.1 -m "…"` y se publican con `git push origin <tag>`. Las versiones liberadas en `main` tienen además un *release* de GitHub con el detalle de los cambios.

### Artefactos bajo control de configuración

*Tabla 83. Artefactos bajo control de configuración*

| Artefacto | Herramienta | Ubicación |
|---|---|---|
| Código fuente | Git | `backend/`, `frontend/`, `e2e/` |
| Esquema y migraciones de la base | Git (Prisma) | `backend/prisma/` |
| Archivos de configuración | Git (sin secretos) | `.env.example`, `docker-compose.yml`, `.github/workflows/` |
| Documentación | Git (Markdown) | `docs/` |
| Documento de entrega (Word/PDF) y presentación | Git (copia por versión) | `docs/entregas/` |
| Configuración de despliegue | Git | `render.yaml`, `frontend/vercel.json` |

## 19.6 Checklist de revisión de cada pull request

- ☐ El título y los commits referencian el issue (`#n`).

- ☐ Pasa el pipeline de CI (lint, pruebas con cobertura y build).
- ☐ Incluye pruebas para la regla de negocio nueva o modificada.
- ☐ Si cambia el modelo de datos, incluye la migración.
- ☐ Si cambia la API, se actualizó `openapi.yaml`.
- ☐ No incluye credenciales ni archivos `.env`.

## 19.7 Evidencias

Repositorio publicado el 15/10/2026: https://github.com/mateogit114730/ganadotrack.

***Tabla 84.*** *Evidencias*

| Evidencia | Enlace |
|---|---|
| Tablero de GitHub Projects con las columnas y los 29 issues | https://github.com/users/mateogit114730/projects/1 |
| Milestones F0–F6 con su porcentaje de avance | https://github.com/mateogit114730/ganadotrack/milestones |
| Historial de commits que referencian issues | https://github.com/mateogit114730/ganadotrack/commits/develop |
| Ejecuciones de GitHub Actions en verde | https://github.com/mateogit114730/ganadotrack/actions |
| Tags de versión | https://github.com/mateogit114730/ganadotrack/tags |
| *Pull request* con la CI como control (PR #30) | https://github.com/mateogit114730/ganadotrack/pull/30 |
| Issues cerrados, con su milestone y etiquetas | https://github.com/mateogit114730/ganadotrack/issues?q=is%3Aissue+is%3Aclosed |
| Ejemplo de issue con criterios de aceptación y commit que lo resolvió (#21) | https://github.com/mateogit114730/ganadotrack/issues/21 |

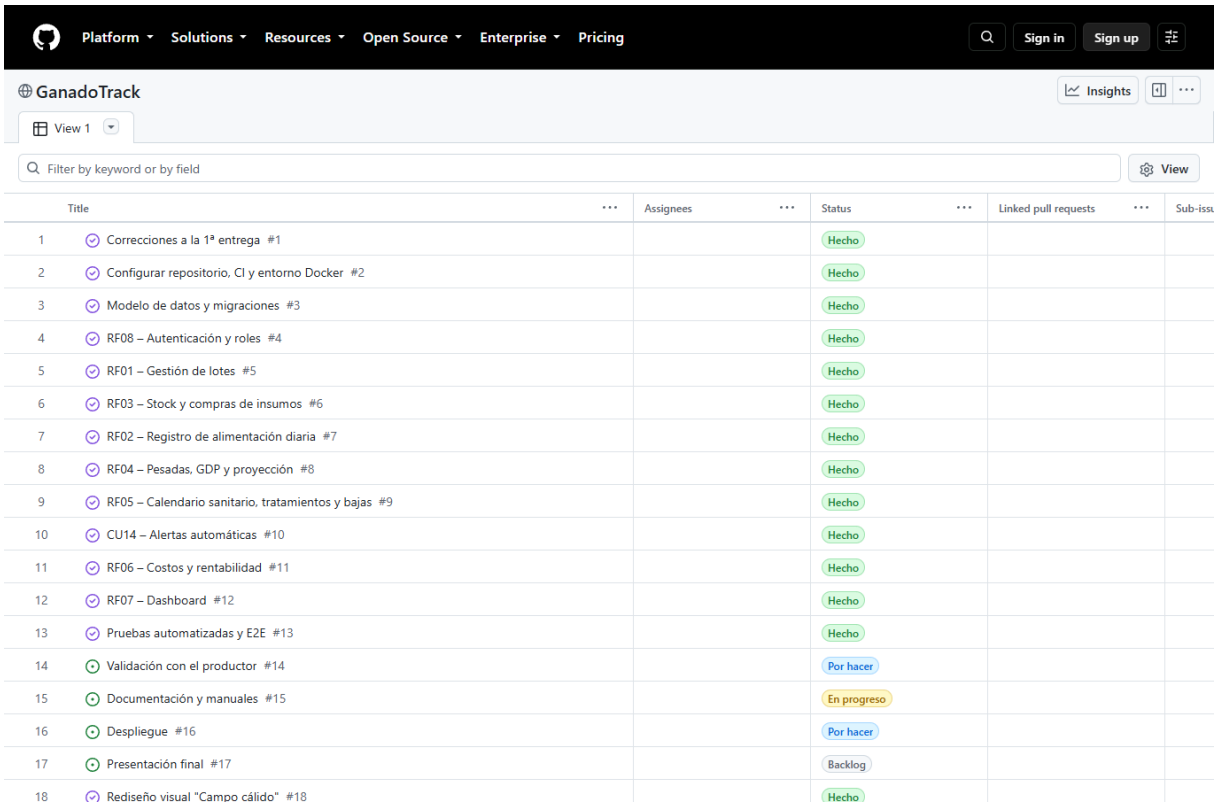



***Figura 17.*** *Tablero de GitHub Projects con las columnas y los 29 issues*

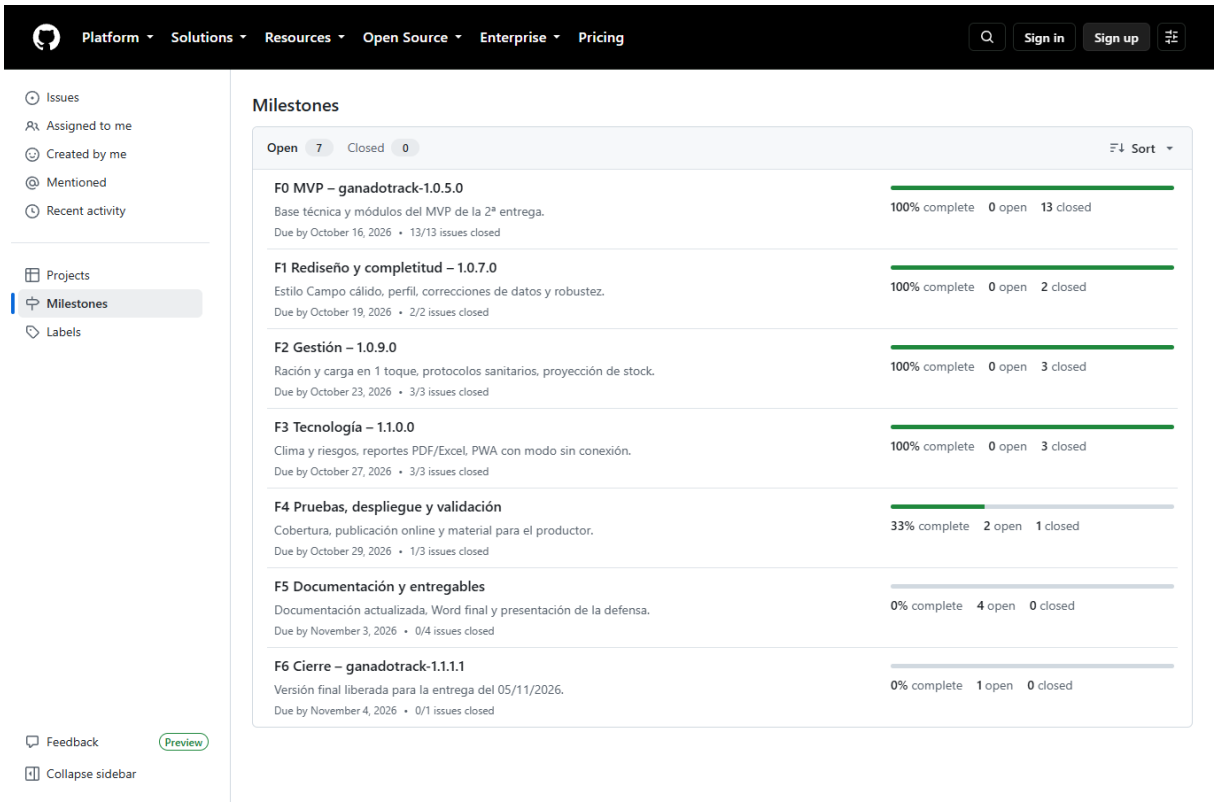



***Figura 18.*** *Milestones F0–F6 con su porcentaje de avance*

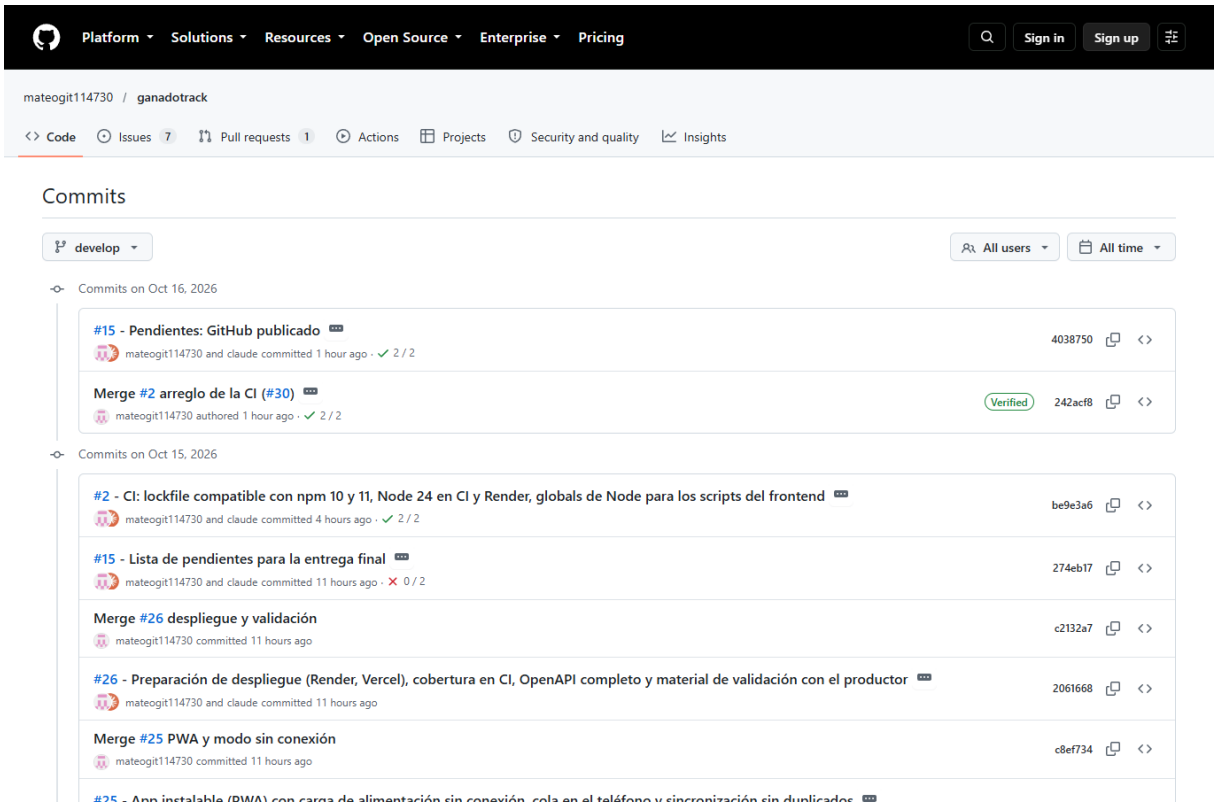



***Figura 19.*** *Historial de commits que referencian issues*

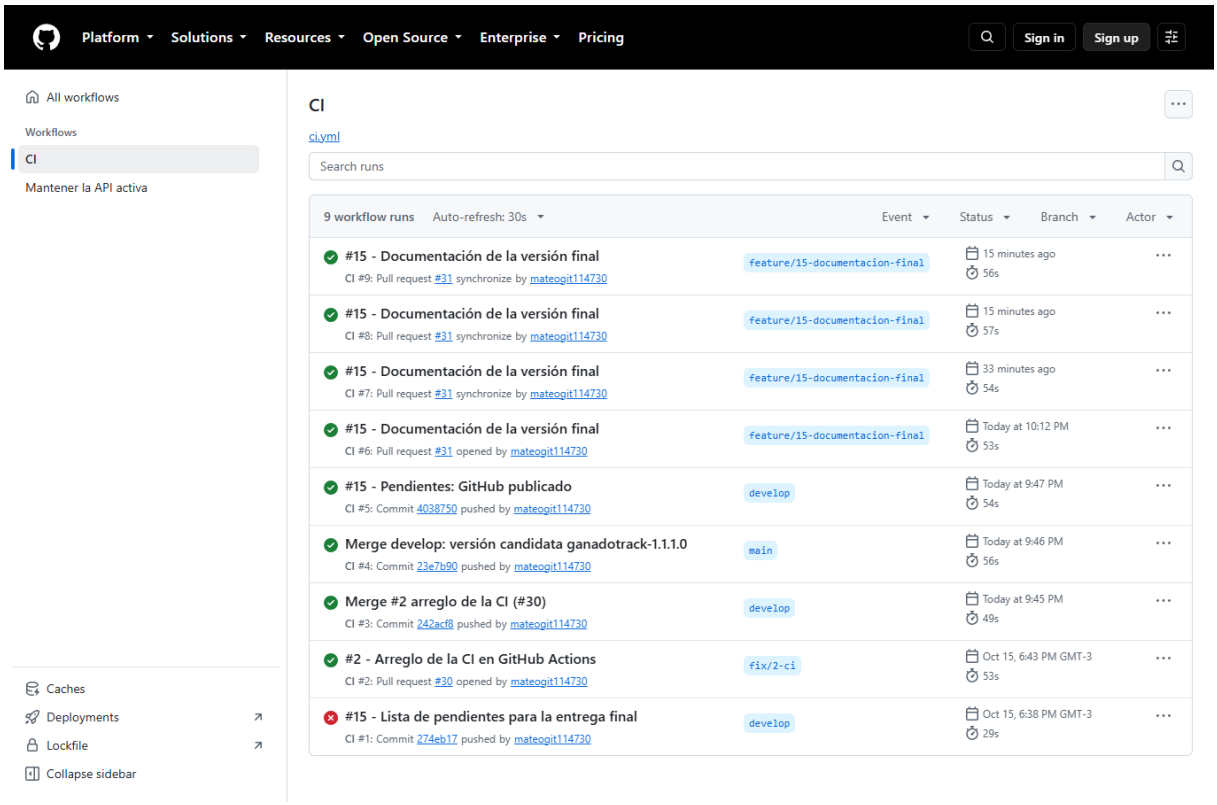



***Figura 20.*** *Ejecuciones de GitHub Actions en verde*

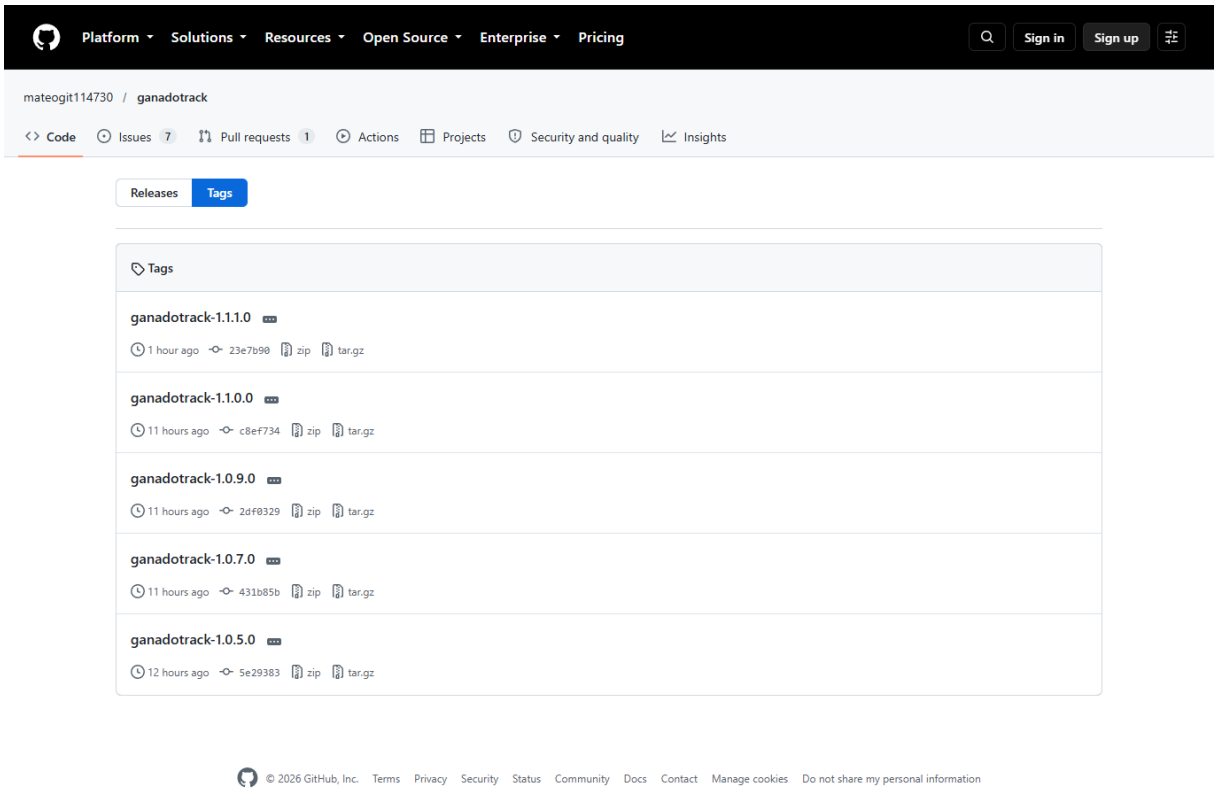



***Figura 21.*** *Tags de versión*

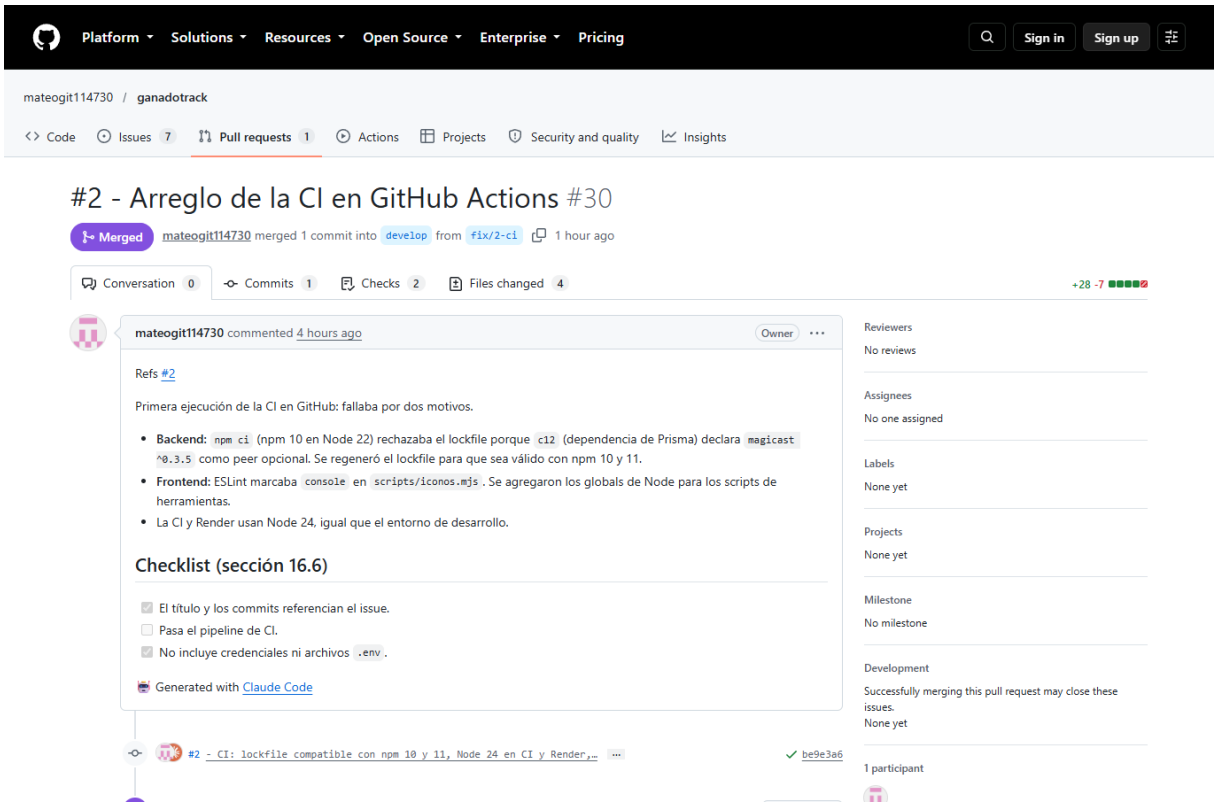



***Figura 22.*** **Pull request* con la CI como control (PR #30)*

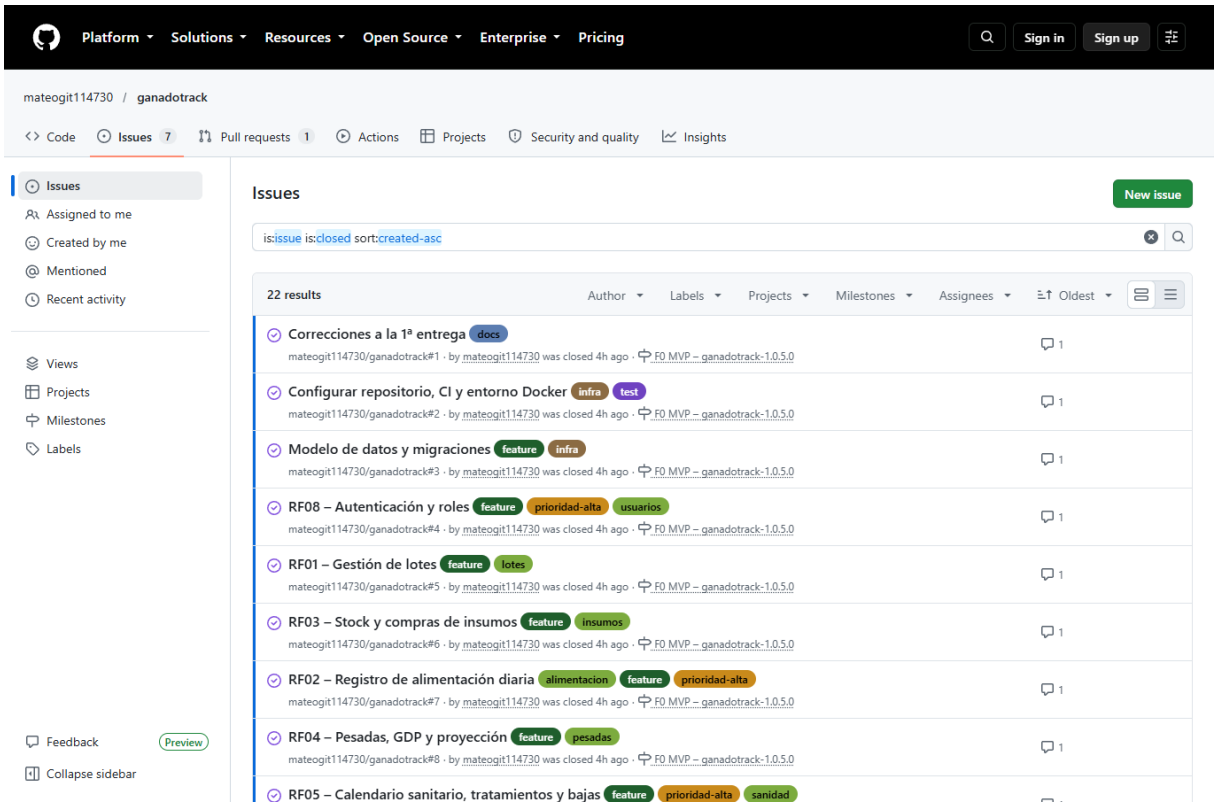



***Figura 23.*** *Issues cerrados, con su milestone y etiquetas*

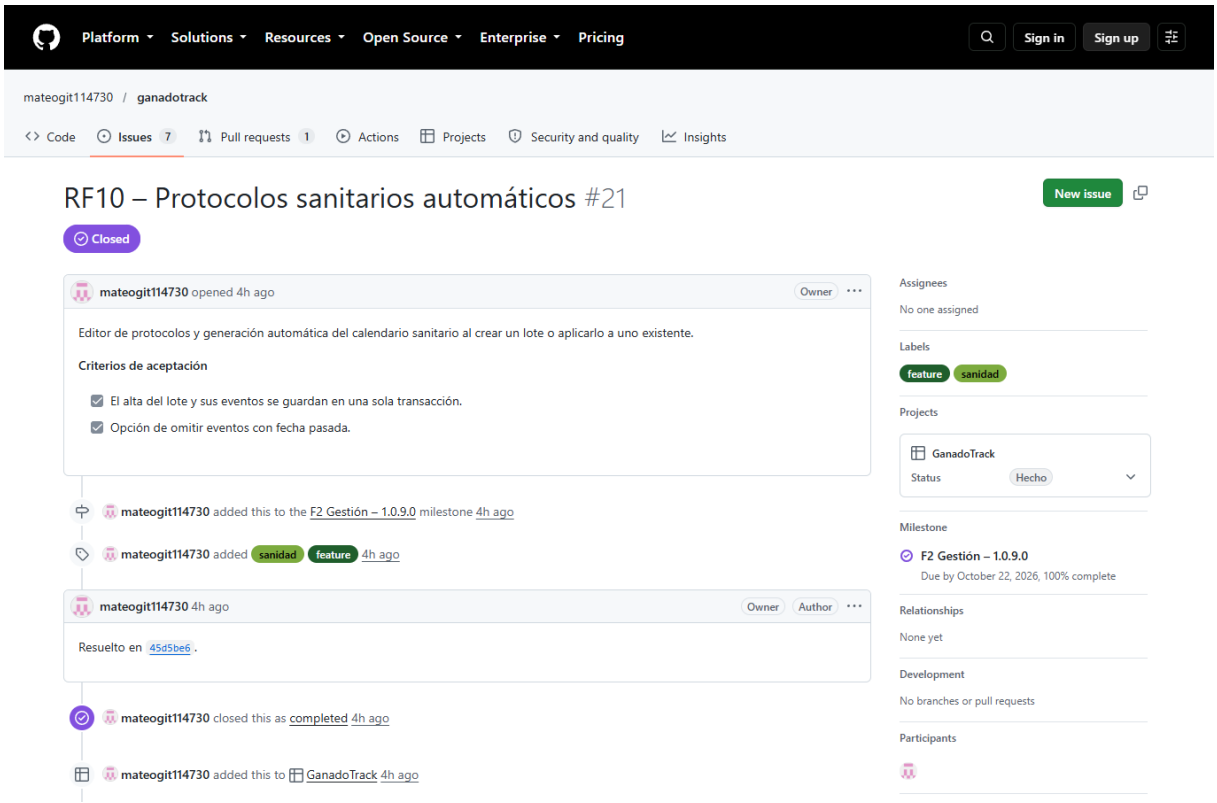



***Figura 24.*** *Ejemplo de issue con criterios de aceptación y commit que lo resolvió (#21)*

## Ejemplo de control de cambios (PR #30):

1. La primera ejecución de la CI en GitHub falló: el *lockfile* no era compatible con npm 10 y un script no pasaba el lint.

2. La corrección se hizo en la rama `fix/2-ci`.
3. Se abrió el PR #30 hacia `develop`.
4. Recién se integró cuando la CI quedó en verde.

Así se ve el flujo completo: issue → rama → commit `#2 - …` → PR → CI → *merge*.

# 20. Resultados obtenidos y conclusiones

## 20.1 Producto desarrollado

GanadoTrack pasó de MVP a **sistema terminado**: una aplicación web instalable que cubre los **14 requerimientos funcionales** y los **22 casos de uso**, con frontend, backend y base de datos integrados, pruebas automatizadas y la configuración para publicarla en internet.

***Tabla 85.*** *Producto desarrollado*

| Módulo | Requerimiento | Estado | Funcionalidad implementada |
|---|---|---|---|
| Gestión de lotes | RF01 | ☑ | Alta con validaciones y protocolo sanitario, días en feedlot, edición, cierre con resumen final previo, historial de lotes cerrados |
| Alimentación diaria | RF02 | ☑ | Pantalla para celular, costo diario automático, acumulado por lote, corrección de errores |
| Stock de insumos | RF03 | ☑ | Stock en kg (silaje mostrado en toneladas), descuento automático, compras y su corrección, alerta de mínimo |
| Pesadas y GDP | RF04 | ☑ | GDP automática, proyección de la fecha de llegada, alerta de GDP baja, gráfico de evolución, eliminación de la última pesada |
| Sanidad y alertas | RF05 | ☑ | Calendario por lote, alerta 7 días antes y de vencidos, garrapata con reprogramación automática, reprogramación de eventos, bajas por causa, costo sanitario |
| Costos y rentabilidad | RF06 | ☑ | Costo acumulado, costo por kg de carne (57 %, descontando bajas), precio de venta y margen bruto |
| Dashboard | RF07 | ☑ | Lotes activos con indicadores, panel de alertas agrupadas, resumen financiero, clima y stock |
| Usuarios y roles | RF08 | ☑ | 3 roles con acceso restringido en el servidor, gestión de usuarios, perfil propio, sesión de 8 h |
| **Ración y conversión** | **RF09** | ☑ | Ración por cabeza con historial, **carga en un toque**, conversión alimenticia en materia seca |
| **Protocolos sanitarios** | **RF10** | ☑ | Editor de protocolos; el calendario de cada tropa se arma solo al crear el lote |
| **Proyección de stock** | **RF11** | ☑ | Días de cobertura, fecha de quiebre, compra sugerida, alerta anticipada y gráfico a 30 días |
| **Clima y riesgos** | **RF12** | ☑ | Pronóstico de la localidad configurada, índice de estrés térmico, riesgo de garrapata y de sequía, alertas |
| **Reportes** | **RF13** | ☑ | Informe de lote y de cierre en PDF, reporte mensual en PDF y planilla Excel para el contador |
| **App instalable y sin conexión** | **RF14** | ☑ | Instalación en el celular; carga sin señal con envío automático y sin duplicados |

Además del alcance funcional, se rediseñó toda la interfaz con el estilo **"Campo cálido"**:

- La paleta y la tipografía acompañan al logo.
- Hay ilustraciones propias (paisaje, alambrado, silueta Angus).
- La pantalla del encargado tiene botones grandes y un modo de alto contraste para usar a pleno sol.

## 20.2 Cumplimiento de los requerimientos no funcionales

*Tabla 86. Cumplimiento de los requerimientos no funcionales*

| RNF | Exigencia | Resultado medido |
|---|---|---|
| RNF01 Usabilidad | Encargado: ≤ 3 clics desde el login; uso en el celular | **1 toque** por lote con la ración definida ("Confirmar ración de hoy"); probado en una pantalla de 390 px |
| RNF02 Disponibilidad | ≥ 95 % | Plataformas gestionadas con HTTPS (Vercel, Render, Neon); la carga del encargado funciona sin señal; si el servicio de clima falla, el resto sigue funcionando |
| RNF03 Rendimiento | Dashboard < 3 s; registro < 2 s | Dashboard ≈ 1,1 s; confirmación de la ración ≈ 0,17 s |
| RNF04 Seguridad | bcrypt, 8 h, RBAC, HTTPS | bcrypt ☑, expiración por inactividad ☑, RBAC en el servidor ☑ (pruebas en cada módulo), HTTPS en el despliegue ☑ |
| RNF05 Escalabilidad | Agregar módulos sin rediseño | Se agregaron 3 módulos nuevos (ración, protocolos, clima) con la misma arquitectura; todas las migraciones fueron aditivas |
| RNF06 Mantenibilidad | Cálculos comentados, estructura estándar | ☑ Funciones de cálculo puras y documentadas; ESLint; CI con cobertura del 88 % |
| RNF07 Operación sin conexión | Carga sin señal, sin duplicados | ☑ Cola local + identificador de operación; probado de punta a punta (CP-84) |

## 20.3 Métricas del proyecto

*Tabla 87. Métricas del proyecto*

| Métrica | MVP (2ª entrega) | Versión final |
|---|---|---|
| Pruebas automatizadas | 58 | **118** (42 unitarias + 70 de integración + 6 del frontend), **100 % aprobadas** |
| Cobertura de líneas del backend | No medida | **88 %** (servicios de dominio: 98 %) |
| Recorridos end-to-end | 10 pasos | 15 pasos con los 3 roles + 3 pasos del modo sin conexión, sin errores |
| Operaciones de la API | 34 | **54** (41 rutas), documentadas en OpenAPI/Swagger |
| Tablas / vistas / triggers | 9 / 1 / 3 | **12** / 1 / 3 |
| Restricciones CHECK / índices | 19 / — | 25 / 16 (incluido un índice único parcial) |
| Pantallas | 9 | **14** |
| Casos de uso | 14 | **22** |
| Issues / commits / tags | — | 29 issues, 7 milestones, 6 tags, CI en verde en `main` y `develop` |
| Líneas de código (backend + frontend, sin pruebas) | — | ≈ 7.500 |

## 20.4 Respuesta a los problemas del relevamiento

*Tabla 88. Respuesta a los problemas del relevamiento*

| Problema relevado | Respuesta de GanadoTrack |
|---|---|
| **Pain point #1:** el productor no conoce su costo de producción por kg de carne | Costo de engorde por kg y margen calculados automáticamente para cada lote, actualizados con cada carga diaria. También se pueden descargar en PDF |

| Problema relevado | Respuesta de GanadoTrack |
|---|---|
| **Pain point #2:** 26 vacas muertas por falta de control sanitario (anaplasmosis) | Calendario sanitario armado **automáticamente** al ingresar cada tropa, alertas 7 días antes y de vencidos, reprogramación automática del control de garrapata y **alerta de riesgo de garrapata** según el clima en los lotes sin tratamiento reciente |
| Registros en papel o verbales | Toda la operación queda registrada, con fecha y usuario responsable; el encargado carga en un toque desde el celular, incluso sin señal |
| Pesadas anotadas sin análisis | GDP automática, comparación con el objetivo, fecha estimada de venta y conversión alimenticia |
| Problemas de abastecimiento en sequía (Bloque 3) | Proyección de cuántos días alcanza cada insumo, compra sugerida y alerta de riesgo de sequía |
| Veterinario comunicado solo por WhatsApp | El veterinario carga los eventos, define los protocolos y la ración directamente en el sistema |
| Información para el contador | Planilla Excel con una hoja por tipo de dato |

## 20.5 Conclusiones

- El sistema resuelve las dos necesidades principales del productor: conocer el costo por kg de carne y no perder el control sanitario. Las funcionalidades agregadas atacan la causa del problema sanitario relevado: el calendario se arma solo y el clima anticipa el riesgo de garrapata.
- Separar la lógica de cálculo en funciones puras permitió verificarla con pruebas automáticas (98 % de cobertura en los servicios de dominio), lo que da confianza en los números que el productor usará para decidir.
- Las reglas críticas se protegen en dos niveles (API y base de datos), de modo que un error de la aplicación no puede dejar el stock o los costos inconsistentes. La carga sin conexión mantiene esa garantía gracias a la idempotencia.
- La arquitectura en capas por módulo permitió sumar seis funcionalidades nuevas (tres de ellas como módulos nuevos) sin rediseñar lo existente, como se había previsto en RNF05.
- La aplicación está publicada y en funcionamiento: https://ganadotrack.vercel.app (frontend en Vercel, API en Render y base en Neon).
- **Pendiente:** la validación con el productor no se completó antes de la entrega. El sistema se validó internamente contra los criterios de aceptación (ver "Validación de aceptación" en la estrategia de pruebas) y el material para el productor (videos, acceso de prueba y cuestionario) quedó preparado. Confirmar con él los valores de referencia, en especial la GDP objetivo, es la primera tarea del trabajo futuro.

# 21. Trabajo futuro

Funcionalidades que quedaron fuera de la versión final (ver Límites) y mejoras detectadas durante el desarrollo, ordenadas por prioridad.

Varias mejoras previstas en la 2ª entrega se incorporaron a la versión final y ya no figuran en esta lista:

- Reportes PDF/Excel (RF13).
- Ración por lote con carga sugerida (RF09).
- Modo sin conexión / PWA (RF14).
- Conversión alimenticia (RF09).
- Planificación ante sequía (RF11 y RF12).

## 21.1 Corto plazo (versión 1.1.3.x)

***Tabla 89.*** *Corto plazo (versión 1.1.3.x)*

| Mejora | Motivo |
|---|---|
| **Costos completos:** valor del animal al ingreso (costo de cría/recría) y otros gastos por lote (energía, combustible, mano de obra, arrendamiento) | Hoy el margen es el **de engorde** y por eso resulta alto. Con estos costos se obtiene el margen real del ciclo. También era un pedido del productor (Bloque 8) |
| **Notificaciones fuera del sistema** (notificaciones *push* de la PWA, email o WhatsApp) para alertas sanitarias vencidas o próximas, stock por agotarse y riesgos de clima | Hoy las alertas se ven al entrar al sistema; el veterinario y el productor se comunican por WhatsApp (relevamiento) |
| **Validación con el productor** y ajustes que surjan: confirmar la GDP objetivo, la ración, los precios, el rendimiento al gancho y el protocolo sanitario | El sistema se validó internamente; falta la confirmación del usuario real. El material (videos, acceso de prueba y cuestionario de 8 preguntas) ya está preparado |

## 21.2 Mediano plazo (versión 1.3.x.x)

***Tabla 90.*** *Mediano plazo (versión 1.3.x.x)*

| Mejora | Motivo |
|---|---|
| **Más pantallas sin conexión** (sanidad y pesadas en la manga) | Hoy solo la carga de alimentación funciona sin señal; el veterinario también trabaja en el corral |
| **Identificación individual por caravana** | El productor está en transición hacia la caravana (Bloque 2) |
| **Ración por fórmula** (porcentajes de la dieta y ajuste automático por peso) | Hoy la ración se define en kg por cabeza; con el peso promedio se podría calcular según el consumo esperado (% del peso vivo) |
| **Historial de precios de los insumos** y comparación entre lotes | Analizar cómo influye el precio del suplemento en el costo por kg |

## 21.3 Largo plazo (versión 2.x)

***Tabla 91.*** *Largo plazo (versión 2.x)*

| Mejora | Motivo |
|---|---|
| **Ciclo completo**: cría, parición, recría, pastoreo y control de nacimientos | Necesidades 1 y 2 del Bloque 7 (nacimientos, cantidad de vacas por lote) |
| **Integración con SENASA** (SIGSA) | Evitar la doble carga de la trazabilidad oficial |

| Mejora | Motivo |
|---|---|
| **Multiestablecimiento** (varios productores en la misma plataforma) | Modelo de negocio con suscripción (el productor valoró pagar ≈ $ 100.000/mes) |
| **Aplicación nativa** o integración con balanzas electrónicas | Cargar las pesadas directamente desde la balanza |

## 21.4 Mejoras técnicas

- Pruebas de componentes React (Testing Library) y recorridos E2E dentro del pipeline de CI.
- Plan pago o servidor propio para evitar la demora de la primera visita (el plan gratuito "duerme" la API).
- Monitoreo y logs centralizados en producción.
- Respaldos automáticos programados con política de retención, además del historial de Neon.
- Auditoría completa de cambios (quién modificó qué y cuándo) en las ediciones y eliminaciones.
- Recuperación de contraseña por email, sin intervención del administrador.

# Anexo I: Documentación de relevamiento

## Datos generales de la entrevista

***Tabla 92.*** *Ficha de la entrevista*

| Campo | Detalle |
|---|---|
| Entrevistador | Mateo Agustín Sgüerzo |
| Entrevistado | Productor ganadero (identidad reservada) |
| Rol | Propietario y administrador del establecimiento |
| Fecha | Primer semestre de 2026 |
| Modalidad | Entrevista semiestructurada realizada mediante mensajería instantánea (WhatsApp), con respuestas escritas del entrevistado por bloque temático |

La entrevista se organizó en bloques temáticos con preguntas abiertas, para recorrer el ciclo completo del negocio ganadero. Las respuestas se registraron por escrito durante la sesión. Se preserva la reserva de identidad del productor.

## Bloque 0: Tipo de operación y especie

**Preguntas realizadas:** ¿Qué especie cría? ¿Qué raza? ¿Qué etapas del ciclo productivo realiza?

**Respuestas obtenidas:** el establecimiento trabaja con ganado bovino de raza Angus y realiza el ciclo completo: cría, recría, engorde a corral (feedlot) y comercialización a través de su carnicería propia.

## Bloque 1: Contexto del negocio

**Preguntas realizadas:**

1. ¿Hace cuántos años está en la actividad ganadera?
2. ¿El campo es propio o alquilado? ¿Cuántas hectáreas trabaja?
3. ¿Cuántos animales tiene actualmente? ¿Y en el pico de mayor actividad?
4. ¿Trabaja solo o tiene empleados? ¿Cuántos y qué rol cumple cada uno?
5. ¿Tiene asesor técnico o veterinario de cabecera? ¿Con qué frecuencia viene?

**Respuestas obtenidas:** el productor lleva diez años en la actividad ganadera. El campo es alquilado y abarca 500 hectáreas. El stock habitual es de 450 cabezas, llegando a 600 en el pico de mayor actividad.

Cuenta con dos empleados. El primero opera el tractor, realiza la siembra y prepara la dieta de engorde. El segundo realiza tareas de mantenimiento general, recorridas a caballo y junta de animales. Ambos participan en las tareas de vacunación del plan sanitario y en la asistencia durante la parición.

El establecimiento cuenta con un veterinario de cabecera que visita el campo al menos cuatro veces al año de manera programada.

## Bloque 2: Compra e ingreso de animales

**Preguntas realizadas:**

1. ¿Cómo consigue los animales? ¿Los compra o son de producción propia?
2. Al comprar o ingresar un lote, ¿qué datos registra?

3. ¿Identifica cada animal individualmente o trabaja por lote/tropa?
4. ¿Hace período de adaptación al ingresar animales?
5. ¿Cuánto tiempo promedio lleva un animal desde que ingresa hasta la venta?

**Respuestas obtenidas:** los animales son de producción propia. Al inicio de la actividad se adquirieron algunos animales en remate feria, pero actualmente todo el stock es de cría propia. Al ingresar un lote se registran: cantidad, peso promedio, procedencia, fecha y raza.

El trabajo se organiza por tropa (lote), aunque el productor se encuentra en proceso de transición hacia la identificación individual por caravana.

Existen períodos de adaptación entre etapas productivas: del destete a la cría, y de la recría al servicio. La duración de cada período depende de la categoría del animal y del tipo de alimentación disponible (pastura o dieta formulada).

En el ciclo completo, el manejo debe dividirse por categoría. Las vacas productivas en buen estado permanecen en el establecimiento entre ocho y diez años. En la etapa de feedlot, los animales permanecen entre 90 y 100 días desde el ingreso al engorde intensivo.

## Bloque 3: Alimentación

**Preguntas realizadas:**

1. ¿Con qué alimenta los animales en cada etapa?
2. ¿Cómo se determina la ración diaria?
3. ¿Registra el consumo de alimento? ¿Sabe el costo por kg de cada alimento?
4. ¿Mantiene stock de alimento? ¿Cómo lo controla?
5. ¿Compra el alimento con anticipación? ¿Ha tenido problemas de abastecimiento?

**Respuestas obtenidas:** la alimentación varía según la etapa productiva:

- Destete: silaje de sorgo + suplemento proteico al 21 %.
- Cría: pasto (pastoreo).
- Vaca en producción: pasto (pastoreo).
- Feedlot: suplemento comprado al 18 % de proteína bruta + silaje de sorgo propio.

El silaje de sorgo es de producción propia; su volumen se calcula y controla en el silo directamente. El suplemento proteico se adquiere semanalmente a un proveedor externo, sin problemas de abastecimiento.

La ración diaria es definida por el veterinario. Se registra la cantidad de kilogramos de alimento dado por lote y la conversión lograda en peso vivo. El costo de los insumos es conocido por el productor.

El único stock que se mantiene físicamente en el campo es el silo de sorgo propio. El alimento comprado se adquiere de forma regular sin necesidad de stockeo previo.

Se ha registrado como problema la falta de pasto en períodos de sequía, lo que obliga a adquirir granos externos de forma urgente y no planificada, con un impacto económico negativo por el costo adicional de ese suplemento de emergencia.

## Bloque 4: Seguimiento del crecimiento

**Preguntas realizadas:**

1. ¿Cada cuánto tiempo pesan los animales? ¿Cómo los pesa?
2. ¿Anota las pesadas? ¿Dónde?

3. ¿Sabe cuál es la Ganancia Diaria de Peso (GDP) de sus animales?
4. ¿Controla la conversión alimenticia?
5. ¿Tiene un peso objetivo de venta? ¿Cómo detecta que un animal no rinde?

**Respuestas obtenidas:** los animales se pesan mensualmente con balanza propia en el establecimiento. Las pesadas se anotan en papel.

El productor no calcula ni conoce la Ganancia Diaria de Peso (GDP) de manera sistemática fuera del feedlot. Dentro del feedlot se controla la conversión alimenticia. El peso de ingreso al feedlot es de 380 kg promedio, y el objetivo ideal es alcanzar los 420 kg al momento del egreso.

La detección de animales que no están comiendo o que no progresan se realiza a través del control del pesaje mensual: la ausencia de aumento de peso en el período esperado es la señal de alerta principal.

## Bloque 5: Sanidad

**Preguntas realizadas:**

1. ¿Qué vacunas aplica de rutina? ¿Registra los tratamientos?
2. ¿Cuánto representa el costo sanitario en el total?
3. ¿Tiene muertes de animales? ¿Las registra? ¿Por qué causa ocurren?
4. ¿Hace desparasitaciones periódicas?

**Respuestas obtenidas:** se aplican todas las vacunas obligatorias exigidas por el SENASA. Además, como el establecimiento se ubica en una zona con presencia activa de garrapata (*Rhipicephalus microplus*), se realizan tratamientos antiparasitarios periódicos, descriptos por el productor como muy costosos y de larga duración.

El productor no conoce el costo sanitario exacto acumulado por lote. Las muertes de animales se registran, y la causa principal identificada es la sequía.

## Bloque 6: Venta y rentabilidad

**Preguntas realizadas:**

1. ¿Cómo vende los animales? ¿A quién?
2. ¿Vende por kg vivo o al gancho?
3. ¿Cuándo decide que un lote está listo para vender?
4. ¿Sabe su costo de producción por kg de carne?

**Respuestas obtenidas:** el productor posee una carnicería propia y realiza la cadena productiva completa: cría en el campo, engorde en feedlot, faena y comercialización minorista directa. No vende la categoría invernada ni realiza ventas en remates o a frigoríficos de terceros.

La decisión de enviar un lote a faena se toma cuando el animal "tiene el peso que tiene que tener y está óptimo para el gancho", combinando la evaluación visual con el peso registrado. La unidad de referencia para la venta es el kilogramo al gancho (peso posfaena), no el kilogramo vivo.

El productor indicó que no conoce su costo de producción por kilogramo de carne. No cuenta con ningún mecanismo para calcular el costo acumulado por lote ni para relacionarlo con el rendimiento al gancho. Esta fue identificada como la principal necesidad insatisfecha (**pain point #1**).

## Bloque 7: Gestión actual y dolor real

**Preguntas realizadas:**

1. ¿Cómo registra hoy la información del campo?
2. ¿Qué información desearía tener disponible pero hoy no tiene?
3. ¿Alguna vez tomó una mala decisión por falta de información?
4. ¿Qué es lo más tedioso o frustrante de gestionar el campo hoy?
5. ¿Cómo interactúa con el contador y con el veterinario?

**Respuestas obtenidas:** el productor confirmó que actualmente no se registra ninguna información administrativa en el establecimiento. No existe ningún sistema de gestión ni parte administrativo formal.

Ante la pregunta sobre qué información desearía tener disponible, el entrevistado identificó cuatro necesidades concretas:

1. Información sobre nacimientos (control de reposición del stock).
2. Control de la cantidad de vacas por lote en todo momento.
3. Conocer qué sanidad tiene cada lote (historial y estado sanitario).
4. Costo de la alimentación por categoría.

Se documentó el siguiente caso concreto de pérdida por ausencia de seguimiento sanitario: la garrapata transmitió la enfermedad anaplasmosis (*Anaplasma marginale*) a un grupo de vacas. Por falta de control sistemático, el cuadro avanzó sin detección temprana, lo que derivó en el colapso del sistema inmune de los animales afectados y la muerte de 26 vacas. Estimando un valor aproximado de USD 1.000 por cabeza, la pérdida total fue de alrededor de USD 26.000. Esta situación fue señalada como el ejemplo más representativo de las consecuencias de la ausencia de información sanitaria oportuna (**pain point #2**).

La principal fuente de frustración indicada por el productor es el desconocimiento de los costos productivos.

El contador externo trabaja exclusivamente en la parte impositiva, sin producir ningún análisis operativo. La comunicación con el veterinario es constante pero informal, a través de WhatsApp.

## Bloque 8: Tecnología y expectativas

**Preguntas realizadas:**

1. ¿Usa el celular para trabajar en el campo? ¿Tiene señal?
2. ¿Tiene computadora o tablet disponible?
3. ¿Usó alguna vez un software de gestión ganadera?
4. ¿Qué tan útil sería un sistema que le mostrara costos, GDP y proyecciones?
5. ¿Pagaría por una herramienta así? ¿Quién más debería tener acceso?

**Respuestas obtenidas:** el productor no utiliza el celular como herramienta de trabajo durante la jornada en el campo, aunque existe señal de telefonía móvil en el predio. No dispone de computadora, pero cuenta con conexión a internet de calidad mediante Starlink (servicio de internet satelital de baja latencia). Nunca utilizó ningún sistema de gestión ganadera.

Ante la pregunta sobre la utilidad de un sistema que mostrara en tiempo real el costo por animal, la ganancia de peso de la semana y la fecha estimada de venta, el productor lo valoró con 8 sobre 10.

Respecto al precio, indicó que en condiciones normales de mercado pagaría aproximadamente $ 100.000 ARS mensuales por una herramienta de este tipo.

En cuanto a los accesos, el propio productor definió tres roles diferenciados:

- **Encargado:** carga los kilogramos de alimento dados por lote cada día.
- **Veterinario:** carga la información sanitaria (eventos, tratamientos, bajas).
- **Administrador:** carga los gastos de insumos y servicios (luz, etc.) y tiene acceso a todos los módulos y reportes.

## Cierre: definición del alcance del MVP

Al finalizar los bloques de la entrevista se realizó una sesión de cierre mediante audios de WhatsApp, en la que se presentaron al productor los posibles módulos del sistema y se discutió el alcance del producto mínimo viable (MVP).

El productor señaló que el ciclo completo de cría, la gestión de vacas madres, la parición y las etapas previas al feedlot involucran demasiadas variables, difíciles de cuantificar, que harían el sistema demasiado complejo para una primera versión. Por este motivo, acordó que el foco del MVP debía ser exclusivamente la etapa de feedlot.

**Alcance acordado del MVP (feedlot únicamente):**

- **M1 – Gestión de lotes:** alta de lote, seguimiento de días, estado, cierre.
- **M2 – Alimentación diaria:** registro de kg dados, cálculo de costo automático.
- **M3 – Stock de insumos:** control de silo y suplemento, alertas de mínimo.
- **M4 – Pesadas y proyección:** GDP automática, fecha estimada de llegada al peso objetivo, alerta si la GDP está por debajo de lo esperado.
- **M5 – Sanidad y alertas:** calendario sanitario, alertas por vencimiento, registro de tratamientos, registro de bajas, costo sanitario acumulado.
- **M6 – Costos y rentabilidad:** costo por kg de carne al gancho (rendimiento 57 % Angus), margen bruto por lote.
- **M7 – Dashboard principal:** resumen de lotes activos, alertas pendientes, resumen financiero.

El ciclo completo (parición, nacimientos, recría, identificación individual por caravana, integración directa con el SENASA) quedó definido como trabajo futuro.

Esta definición del alcance, validada directamente por el productor, es la base del diseño funcional de GanadoTrack.

# Anexo II: Manual de instalación y configuración

## Requisitos

***Tabla 93.*** *Requisitos*

| Software | Versión | Para qué |
|---|---|---|
| Node.js | 22 o superior (recomendado **24**, el usado en desarrollo, CI y producción) | Ejecutar el backend y compilar el frontend |
| Docker Desktop | Cualquier versión reciente | Base de datos PostgreSQL local |
| Git | — | Descargar el repositorio |
| Microsoft Edge | — | Solo para los recorridos E2E (opcional) |

## Instalación local paso a paso

```
git clone https://github.com/mateogit114730/ganadotrack.git
cd ganadotrack

# 1. Base de datos
docker compose up -d                # PostgreSQL 17 en el puerto 5433 (bases ganadotrack y
ganadotrack_test)

# 2. Backend
cd backend
cp .env.example .env                # editar JWT_SECRET (cadena aleatoria larga) y SEED_PASSWORD
npm install                         # también genera el cliente de Prisma
npx prisma migrate deploy           # crea tablas, restricciones CHECK, vista, trigger e índices
(6 migraciones)
npm run db:seed                     # datos de demostración (borra los existentes)
npm run dev                         # http://localhost:4000/api  ·  docs: /api/docs

# 3. Frontend (otra terminal)
cd ../frontend
npm install
npm run dev                         # http://localhost:5173
```

### Variables de entorno del backend (`backend/.env`)

***Tabla 94.*** *Variables de entorno del backend (backend/.env)*

| Variable | Ejemplo | Descripción |
|---|---|---|
| `DATABASE_URL` | `postgresql://ganadotrack:ganadotrack@localhost:5433/ganadotrack?schema=public` | Conexión a PostgreSQL |
| `PORT` | `4000` | Puerto de la API (Render lo asigna solo) |
| `JWT_SECRET` | *(cadena aleatoria de 64+ caracteres)* | Firma de los tokens; **nunca** publicarla |
| `JWT_EXPIRES_IN` | `8h` | Duración de la sesión sin actividad (RNF04) |
| `CORS_ORIGIN` | `http://localhost:5173` | URL del frontend autorizada a usar la API |

| Variable | Ejemplo | Descripción |
|---|---|---|
| `SEED_PASSWORD` | — | Contraseña de los usuarios de demostración |
| `NODE_ENV` | `production` | Solo en producción: registro de peticiones en formato *combined* |

Para generar un `JWT_SECRET`: `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`.

### Variables del frontend (`frontend/.env`)

***Tabla 95.*** *Variables del frontend (frontend/.env)*

| Variable | Descripción |
|---|---|
| `VITE_API_URL` | URL pública de la API (por ejemplo `https://ganadotrack-api.onrender.com/api`). En desarrollo no hace falta: Vite redirige `/api` al backend |

### Datos de demostración

El seed carga:

- **3 usuarios:** administrador, encargado y veterinario, con la contraseña de `SEED_PASSWORD`.
- **2 insumos:** suplemento y silaje, con su % de materia seca.
- **4 lotes:** uno cerrado y tres activos, con raciones, pesadas, alimentación, eventos sanitarios y bajas.
- **2 protocolos sanitarios.**
- **La ubicación de demostración:** Pergamino, Buenos Aires.

Las fechas son relativas al día en que se ejecuta, así que los datos siempre se ven actuales.

## Verificar la instalación

```
cd backend
npm test                # 112 pruebas (requiere la base de Docker levantada)
npm run test:coverage   # ídem, con reporte de cobertura en coverage/
npm run lint

cd ../frontend
npm test                # 6 pruebas de la cola sin conexión
npm run lint
npm run build           # compila la app y genera el service worker
```

Recorridos end-to-end opcionales (requieren Microsoft Edge y modifican los datos de demostración):

```
cd e2e && npm install
SEED_PASSWORD=... npm run recorrido                  # 15 pasos con los 3 roles; capturas en
docs/capturas

# Modo sin conexión: necesita la app compilada, porque el service worker solo existe en el build
cd ../frontend && npm run build && npm run preview     # http://localhost:4173 (otra terminal)
cd ../e2e && SEED_PASSWORD=... node offline.mjs ../docs/capturas

cd ../backend && npm run db:seed                       # restaurar los datos de demostración
```

> *En Git Bash para Windows, anteponer `MSYS_NO_PATHCONV=1` a los comandos de `e2e/` para que no se modifiquen las rutas de los argumentos.*

## Problemas frecuentes

***Tabla 96.*** *Problemas frecuentes*

| Problema | Solución |
|---|---|
| `port is already allocated` al iniciar Docker | Otro servicio usa el puerto 5433. Cambiar el puerto en `docker-compose.yml` y en `DATABASE_URL` |
| `Falta la variable de entorno JWT_SECRET` | Falta el archivo `backend/.env` (copiarlo de `.env.example`) |
| `Can't reach database server` | Docker no está corriendo: abrir Docker Desktop y ejecutar `docker compose up -d` |
| El frontend muestra "No se pudo conectar con el servidor" | El backend no está corriendo en el puerto 4000 |
| `EPERM` al ejecutar `npm install` o `prisma generate` en Windows | La API está corriendo y bloquea el motor de Prisma: detenerla, repetir el comando y volver a iniciarla |
| `npm ci` falla en la CI con "package.json and package-lock.json are not in sync" | Regenerar el *lockfile* con la misma versión de npm de la CI (`npx npm@10 install --package-lock-only`) |
| El clima dice "Configurá la localidad…" | Configurar la ubicación en *Configuración* (o ejecutar el seed, que carga Pergamino) |
| La app instalada no muestra los cambios nuevos | Cerrarla y volver a abrirla con conexión: el *service worker* se actualiza solo |

## Respaldo y restauración de la base

```
# Local
docker exec ganadotrack-db pg_dump -U ganadotrack ganadotrack > respaldo.sql
docker exec -i ganadotrack-db psql -U ganadotrack ganadotrack < respaldo.sql
```

En producción, Neon guarda el historial de la base y permite restaurarla a un momento anterior desde su panel. También se puede usar `pg_dump` con la cadena de conexión de Neon.

## Despliegue online

Arquitectura de producción: frontend en **Vercel**, API en **Render** y base de datos en **Neon**, todo en planes gratuitos y con HTTPS (ver la sección 14.12). La configuración está versionada en el repositorio:

***Tabla 97.*** *Despliegue online*

| Archivo | Qué define |
|---|---|
| `render.yaml` | Servicio web de la API: carpeta `backend`, Node 24, *build* `npm ci --include=dev && npx prisma migrate deploy`, inicio `npm start`, *health check* `/api/health`, región Virginia y variables (las secretas se cargan en el panel) |
| `frontend/vercel.json` | *Rewrite* de todas las rutas a `index.html` (SPA) y caché: el *service worker* se revalida en cada visita y los archivos con *hash* se guardan un año |
| `.github/workflows/ci.yml` | Lint, pruebas con cobertura y *build* en cada *push* y *pull request* |
| `.github/workflows/mantener-activa.yml` | Consulta `/api/health` cada 14 minutos en días hábiles de 8 a 22 h (hora argentina) para que la API no "duerma" |

### Pasos

1. **Neon:** crear el proyecto `ganadotrack` (Postgres 17, región *AWS US East 1*). Copiar la cadena de conexión directa (sin *pooling*).
2. **Render:** *New* → *Blueprint* con el repositorio. Cargar `DATABASE_URL` (la cadena de Neon), `CORS_ORIGIN` (la URL de Vercel) y `SEED_PASSWORD`. `JWT_SECRET` lo genera Render. Cada despliegue aplica las migraciones pendientes antes de iniciar.
3. **Vercel:** importar el repositorio con *Root Directory* `frontend` y la variable `VITE_API_URL` = `https://<servicio>.onrender.com/api`. Si la URL final de Vercel es otra, corregir `CORS_ORIGIN` en Render.
4. **Datos de demostración** (solo la primera vez; borra los datos existentes):

```
cd backend
DATABASE_URL="<cadena de Neon>" SEED_PASSWORD="<contraseña>" npx prisma db seed
```

5. **Verificar:**
   - `https://<servicio>.onrender.com/api/health` responde `{"estado":"ok","baseDeDatos":"ok"}`.
   - La app abre en la URL de Vercel e inicia sesión.
6. **Opcional:** en GitHub → *Settings* → *Secrets and variables* → *Actions* → *Variables*, crear `API_URL` = `https://<servicio>.onrender.com` para activar el flujo que mantiene la API activa.

Desde entonces, cada *push* a `main` vuelve a publicar la API y el frontend automáticamente.

> ***Plan gratuito de Render:*** *la API se "duerme" tras 15 minutos sin uso y la primera visita tarda unos 50 segundos. Antes de una demostración, abrir la app unos minutos antes.*

La guía paso a paso para crear las cuentas está en `docs/entregas/guia-publicacion.md`.

# Anexo III: Manual de usuario

GanadoTrack se usa desde el navegador (Chrome, Edge, Safari) en computadora, tablet o celular. También se puede **instalar como aplicación** en el celular (ver sección 6). Cada persona ingresa con su email y contraseña, y ve solo las opciones de su rol.

***Tabla 98.*** *Manual de usuario*

| Rol | Qué puede hacer |
|---|---|
| **Administrador** (productor) | Todo: lotes, pesadas, costos, insumos y proyección de stock, sanidad, protocolos, raciones, reportes, usuarios y configuración |
| **Encargado** | Cargar la comida del día de cada lote, incluso sin señal |
| **Veterinario** | Calendario sanitario, tratamientos, bajas, protocolos sanitarios, raciones y alertas sanitarias |

## 1. Ingresar al sistema

1. Abrir la dirección de GanadoTrack en el navegador (o la app instalada).
2. Escribir el email y la contraseña, y presionar **Ingresar**.
3. El sistema abre la pantalla de inicio de cada rol.




***Figura 25.*** *Inicio de sesión*

**Barra superior:** muestra el saludo, la fecha y el **clima actual** del campo.

**Menú de usuario:** se abre tocando el círculo con las iniciales, arriba a la derecha. Tiene tres opciones:

- **Mi perfil.**
- **Instalar la app**: aparece cuando el navegador lo permite.
- **Cerrar sesión.**

La sesión se cierra sola después de 8 horas sin uso.

## 2. Encargado: la comida de hoy

La pantalla **Comida de hoy** muestra una tarjeta por cada lote del corral, con sus cabezas y su estado:

***Tabla 99.*** *Encargado: la comida de hoy*

| Estado | Significado |
|---|---|
| *Pendiente* | Todavía no se cargó la comida de hoy |
| *Carga parcial* | Se cargó solo uno de los insumos |
| *Cargado hoy* | Listo; debajo se ven los kilos registrados |
| *En el teléfono* | Se cargó sin señal y se va a enviar sola (ver 2.2) |

### 2.1 Confirmar la ración en un toque

Si el veterinario definió la ración del lote, la tarjeta muestra la **Ración de hoy** ya calculada para las cabezas actuales.

- Si se dio exactamente eso → tocar **Confirmar ración de hoy**. Listo.
- Si se dio otra cantidad → tocar **Di otros kilos**. Se abre el formulario con los kilos sugeridos. Se cambian los que correspondan y se toca **Confirmar**.
- Si el lote no tiene ración definida, el botón es **Cargar kilos** y se escriben los kilos de cada insumo. Si un insumo no se dio, se deja vacío.

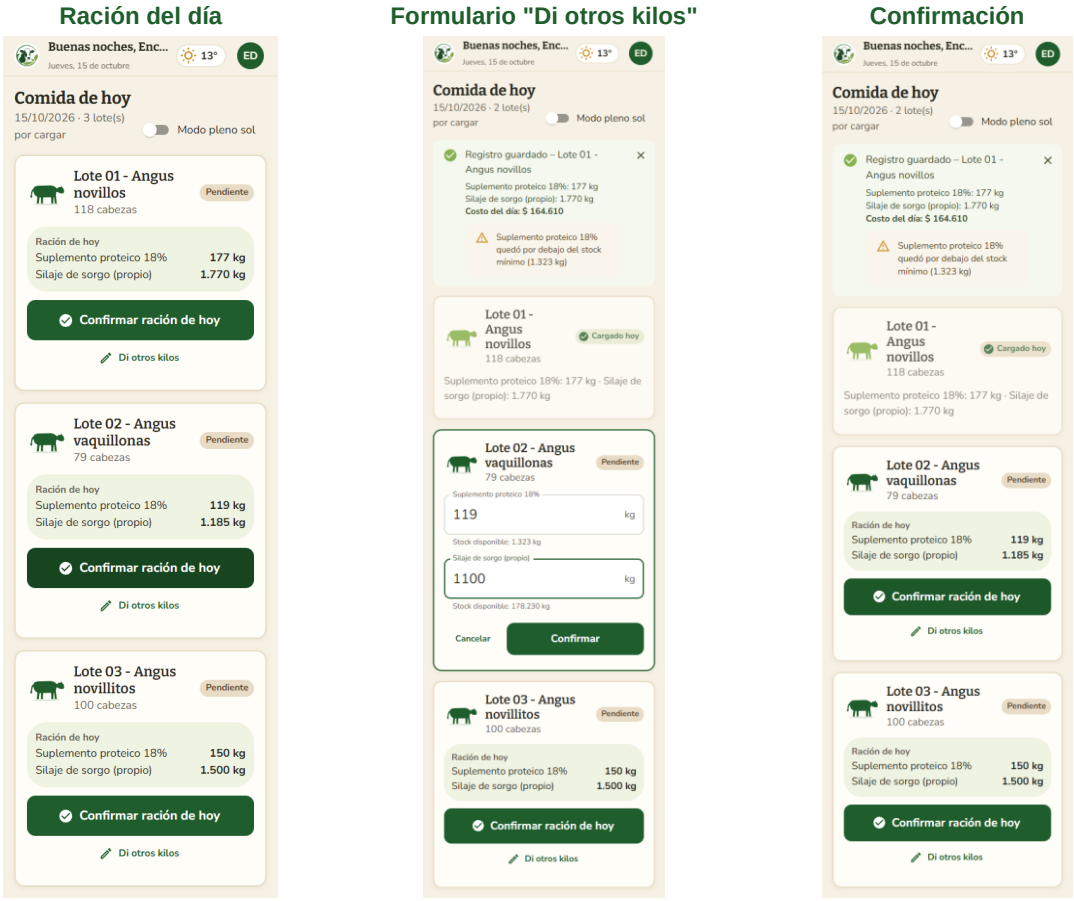



***Figura 26.*** *Comida de hoy · Formulario · Confirmación*

El sistema descuenta los kilos del stock y avisa si algún insumo quedó por debajo del mínimo.

**Modo pleno sol:** el interruptor de arriba agranda la letra y aumenta el contraste para ver mejor la pantalla al aire libre.

## 2.2 Trabajar sin señal

En los corrales sin señal se puede seguir cargando igual:

1. Arriba aparece el aviso **Sin conexión**.
2. Confirmar la ración (o cargar los kilos) como siempre.
3. La tarjeta pasa a **En el teléfono**: la carga quedó guardada en el celular, con la fecha de hoy.
4. Cuando vuelve la señal, las cargas se envían solas. También se puede tocar **Enviar ahora** en el aviso de cargas pendientes.

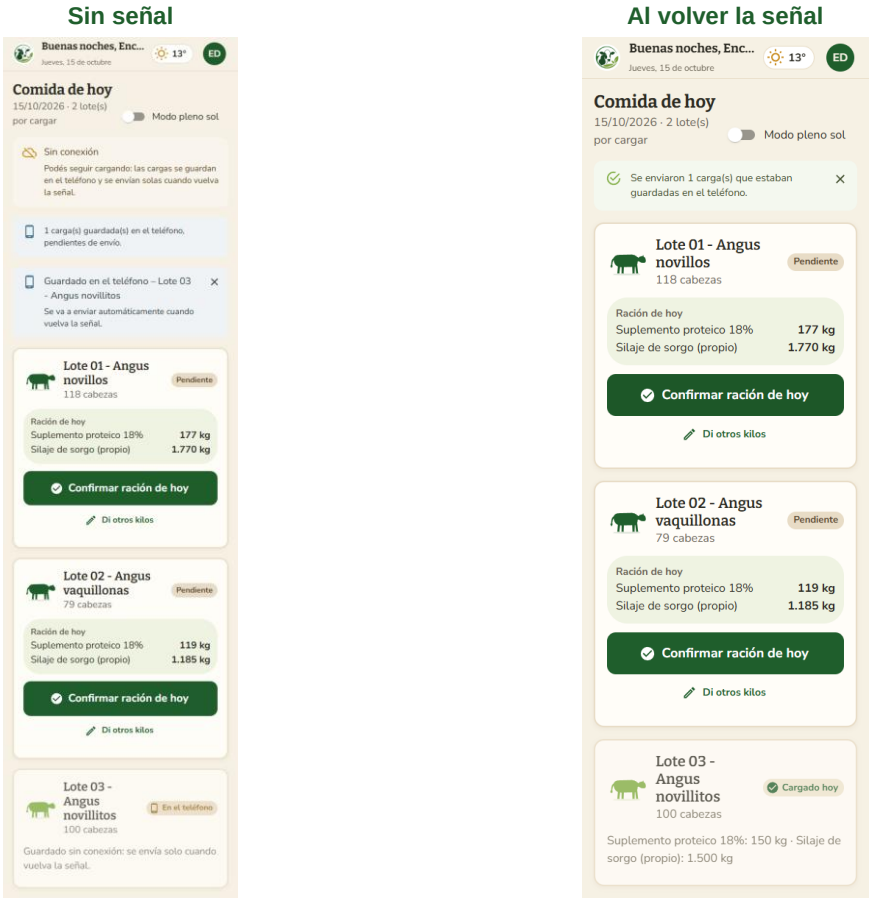



***Figura 27.*** *Sin conexión · Sincronizado*

> *Para que funcione sin señal, la app tiene que haberse abierto al menos una vez **con** señal ese día, para descargar los lotes y la ración. No hace falta tenerla instalada, pero es más cómodo.*

Si al enviarse una carga el sistema la rechaza (por ejemplo, porque no hay stock suficiente), aparece en rojo **"No se pudo enviar la carga de …"** con el motivo. En ese caso, avisar al administrador y tocar **Descartar**.

**Mensajes posibles:**

- *"Ingresá los kilos de al menos un insumo"*: no se cargó ningún valor.
- *"Stock insuficiente…"*: hay menos insumo en stock que lo que se quiere registrar. Avisar al administrador para que registre la compra.

- *"Ya se registró … en esa fecha"*: ese insumo ya se cargó hoy para ese lote. Si fue un error, el administrador puede eliminar el registro.

# 3. Veterinario

## 3.1 Sanidad

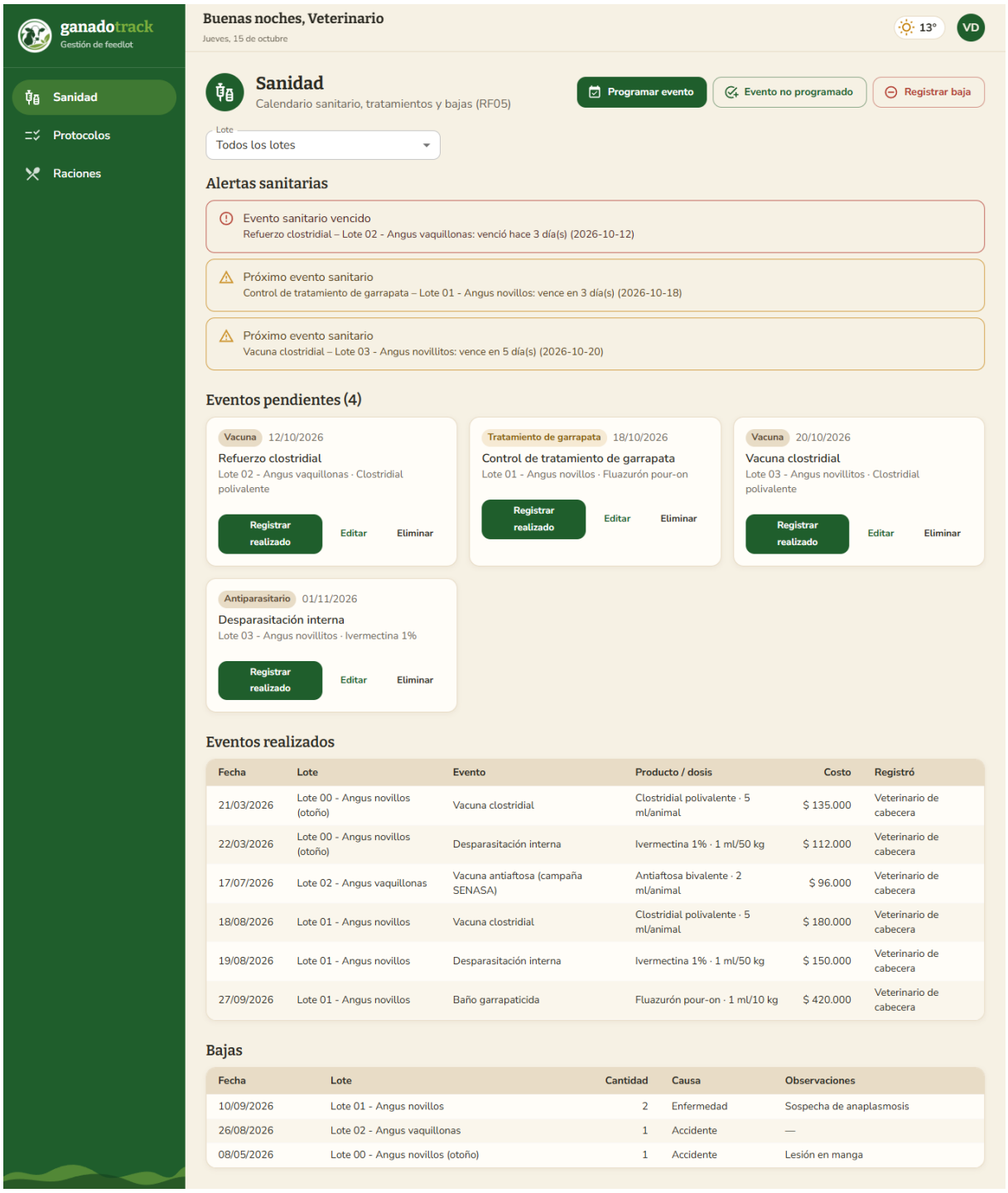



***Figura 28.*** *Pantalla de sanidad*

La pantalla muestra, de arriba hacia abajo:

- **Alertas sanitarias:** en rojo los eventos vencidos y en naranja los que vencen en los próximos 7 días. También aparecen las de **clima** que afectan la sanidad: riesgo alto de garrapata en lotes sin tratamiento reciente y estrés térmico.
- **Eventos pendientes:** el calendario de cada lote.
- **Historial** de eventos realizados y de bajas.

El selector **Lote** permite ver un solo lote.

**Programar un evento: Programar evento** → elegir el lote, el tipo (vacuna, antiparasitario, tratamiento de garrapata, otro), el nombre, la fecha y el producto previsto. Un evento pendiente se puede **reprogramar** (cambiar fecha, nombre o producto) con el botón **Editar** de su tarjeta, o eliminar.

**Registrar un evento realizado:** en la tarjeta del evento pendiente, presionar **Registrar realizado** y cargar la fecha, el producto, la dosis y el costo.

- Si no se carga el costo, se guarda en $ 0 con una advertencia (el costo sanitario del lote queda incompleto).
- En un **tratamiento de garrapata** hay que indicar la **duración en días**: al terminar el tratamiento, el sistema programa automáticamente un control.

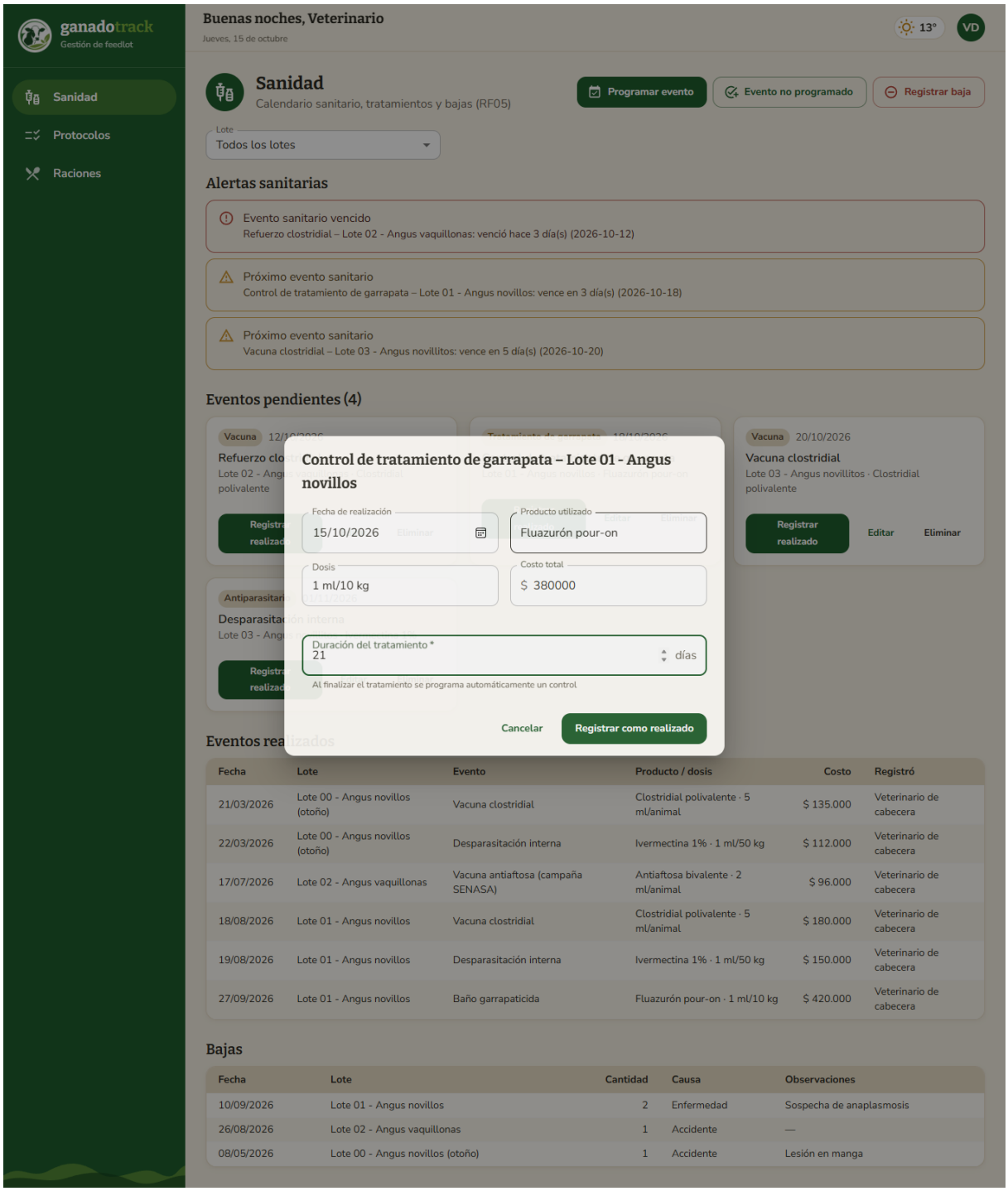



***Figura 29.*** *Registrar evento*

**Evento no programado:** sirve para registrar algo que no estaba en el calendario, como un tratamiento de urgencia.

**Registrar una baja: Registrar baja** → elegir el lote, la fecha, la cantidad de animales y la causa (enfermedad, accidente, sequía, otra). Las cabezas del lote se descuentan y los costos por kg se recalculan.

## 3.2 Protocolos sanitarios

Un protocolo es el **plan sanitario tipo** de una tropa. Cuando el administrador crea un lote con un protocolo, el sistema arma solo todo su calendario de vacunas y tratamientos.

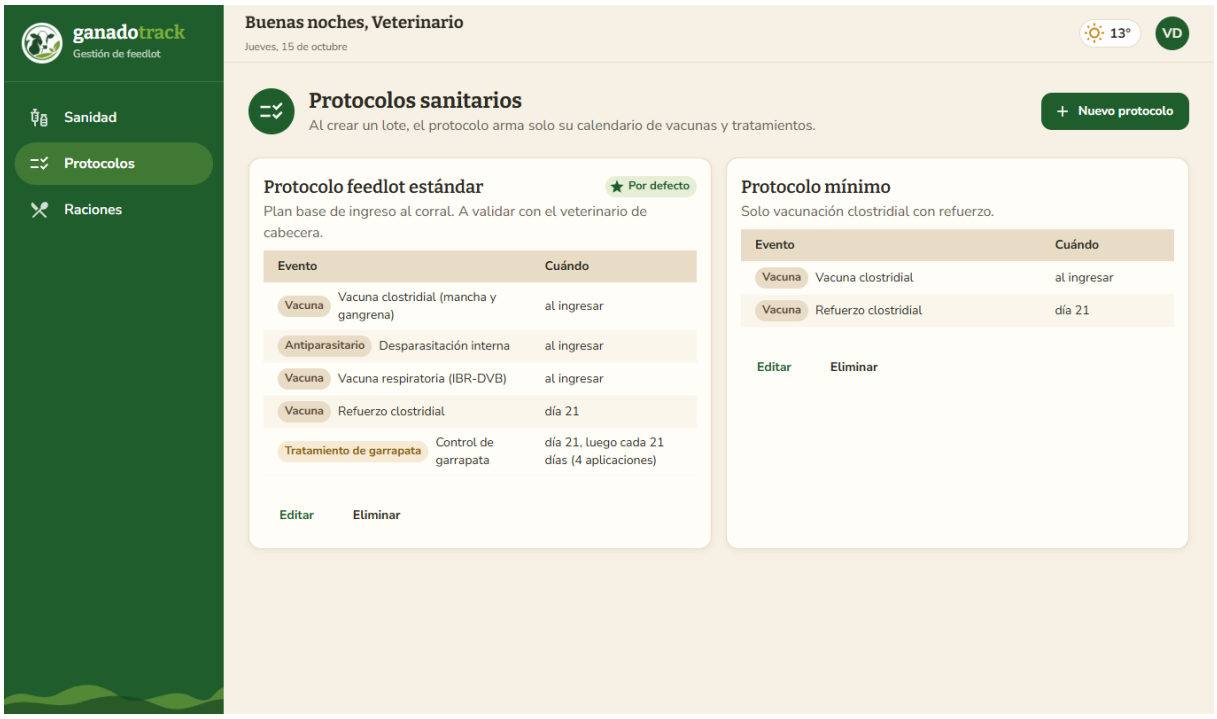



***Figura 30.*** *Protocolos*

1. **Nuevo protocolo** → nombre y descripción.
2. Agregar los eventos. Para cada uno se indica:
    - **Tipo**, **Evento** (nombre) y **Producto**.
    - **Día desde el ingreso**: 0 = el día que entra la tropa.
    - Si se repite: **Cada (días)** y **Aplicaciones**. Por ejemplo, el control de garrapata el día 21, cada 21 días, 4 aplicaciones.
3. Marcar **Usar por defecto al crear lotes** si es el plan habitual. Solo un protocolo puede estar por defecto.
4. Guardar. La tabla muestra **cuándo** se aplica cada evento.

Editar o eliminar un protocolo no cambia los calendarios de los lotes ya creados.

## 3.3 Raciones

Define cuántos **kilos por cabeza por día** de cada insumo recibe cada lote. Con eso, el encargado ve la ración del día lista para confirmar.

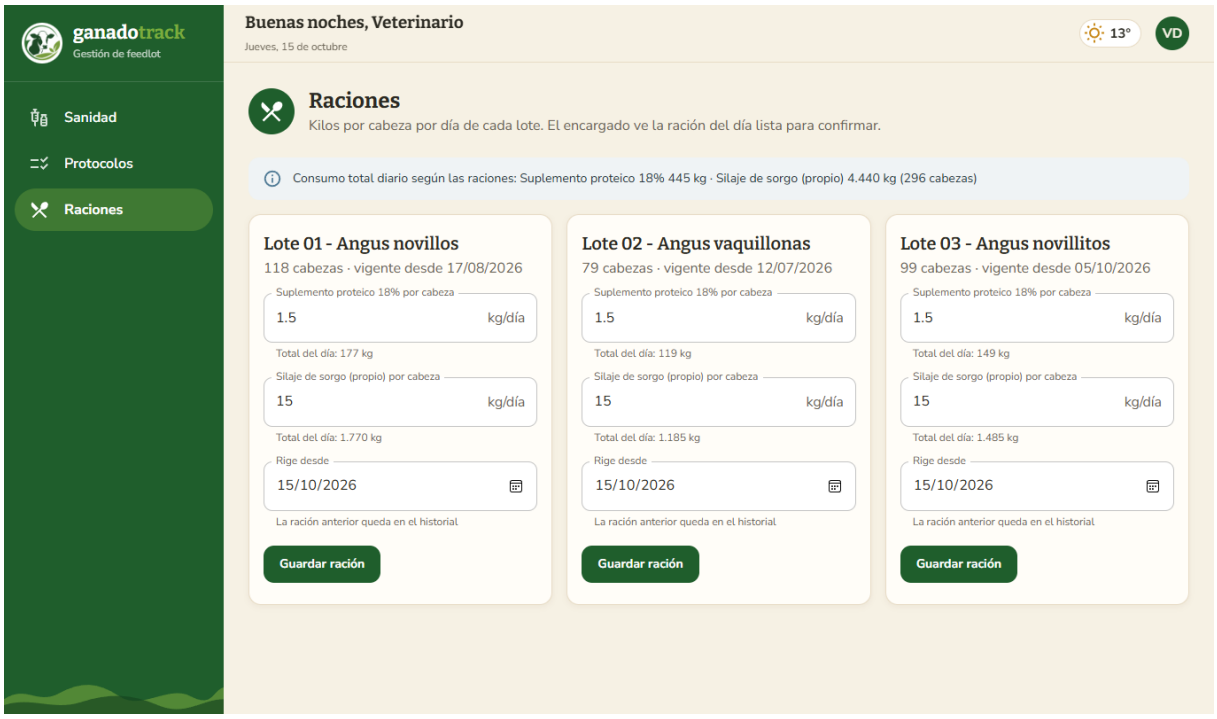



***Figura 31.*** *Raciones*

1. En la tarjeta del lote, escribir los kg por cabeza de cada insumo. Debajo se ve el **total del día** para las cabezas actuales.
2. Elegir desde cuándo **rige** (por defecto, hoy). Si se pone una fecha futura, la ración actual sigue hasta ese día.
3. **Guardar ración**. La anterior queda en el historial.

Para quitar un insumo de la ración, cargar 0. Arriba se muestra el consumo total diario de todos los lotes.

# 4. Administrador

## 4.1 Dashboard

Resumen del feedlot con:

- **Indicadores generales:** cabezas, costo acumulado, carne estimada y margen.
- **Lotes en el corral:** días, peso, GDP, costo, fecha estimada de llegada al objetivo, avance y evolución del peso.
- **Clima del campo:** temperatura actual, pronóstico de 7 días y **riesgos orientativos** (estrés térmico, garrapata y sequía).
- **Alertas:** agrupadas en sanidad, stock, ganancia de peso y clima.
- **Stock de insumos:** con los días que alcanza cada uno.

Tocar un lote, o el enlace *Ver lote* de una alerta, abre su detalle.

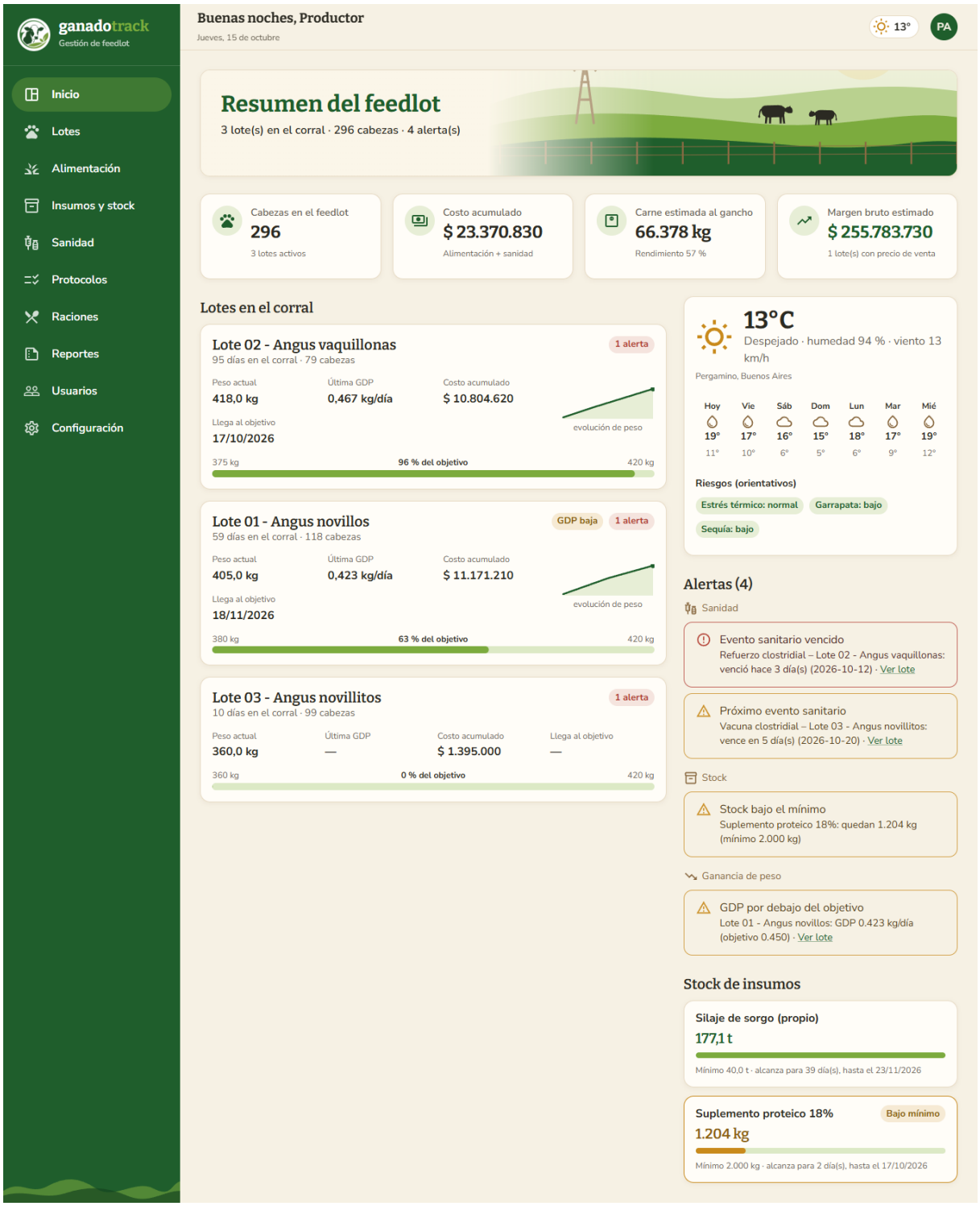



***Figura 32.*** *Dashboard*

> ***Los riesgos de clima son orientativos.*** *Se calculan con el pronóstico y no reemplazan el criterio del veterinario:*

- ***Estrés térmico:*** *según el índice de temperatura y humedad, desde 72.*
- ***Garrapata:*** *alto con más de 20 °C y 70 % de humedad en la semana.*
- ***Sequía:*** *alto con menos de 20 mm de lluvia en 30 días y menos de 5 mm pronosticados.*

## 4.2 Lotes

- **Nuevo lote:** nombre, fecha de ingreso, cantidad de animales, peso de ingreso, peso objetivo, GDP objetivo (ganancia diaria esperada, por ejemplo 0,45 kg/día) y **Protocolo sanitario**.
    - El sistema propone el protocolo marcado por defecto y, al guardar, informa cuántos eventos sanitarios programó.
    - Si el lote ingresó antes de hoy, la opción *"no programar los eventos con fecha ya pasada"* evita cargar vacunas que ya se dieron.
- Las pestañas *Activos / Cerrados / Todos* filtran la lista.

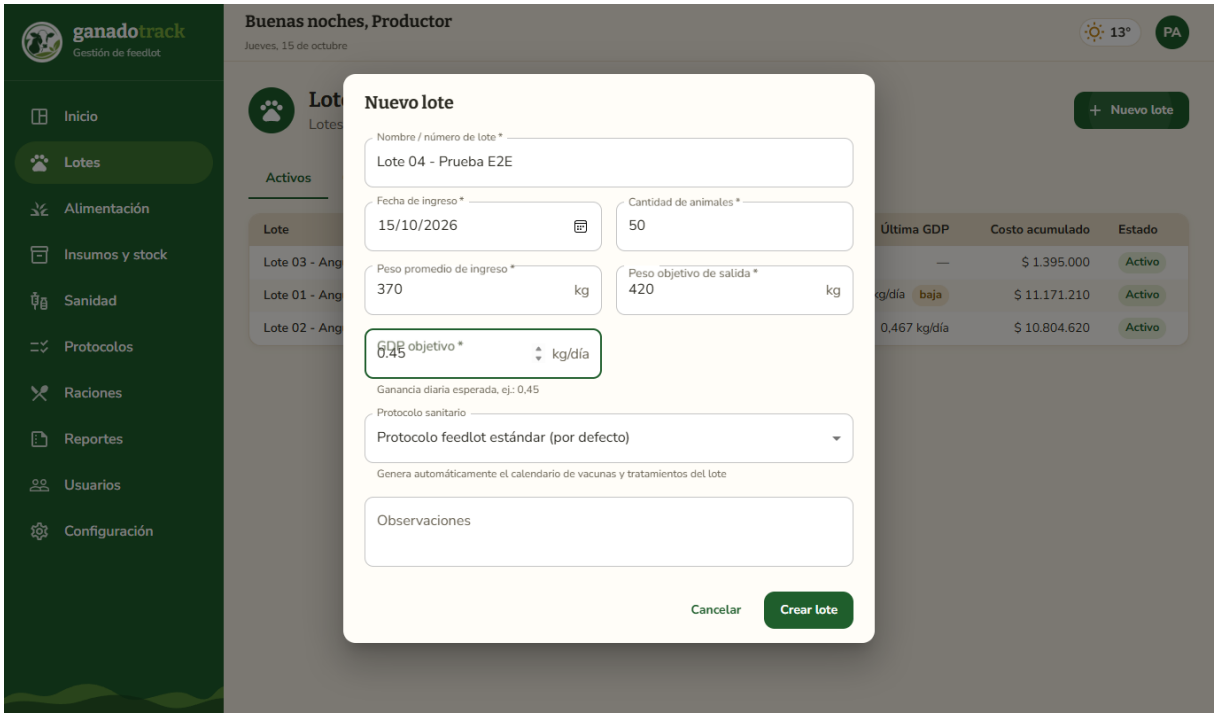



***Figura 33.*** *Nuevo lote*

## 4.3 Detalle del lote

Muestra los indicadores principales, el botón **Informe PDF** y cuatro pestañas:

- **Pesadas:** gráfico de evolución del peso frente al objetivo y tabla con la GDP y la **conversión alimenticia** de cada pesada. Esta última indica cuántos kg de materia seca hicieron falta por kg ganado: cuanto menor, mejor. El ícono 🗑 elimina **la última** pesada si se cargó por error.
- **Alimentación:** registros diarios. El ícono 🗑 elimina un registro cargado por error y devuelve los kilos al stock.
- **Sanidad:** calendario y bajas del lote.
    - **Aplicar protocolo** genera el calendario de un protocolo en un lote que no lo tenía.
    - El ícono 🗑 de una baja la elimina y devuelve las cabezas.
- **Costos:** desglose por insumo y por tipo de evento, kg de carne estimados, costo por kg, conversión acumulada y margen.

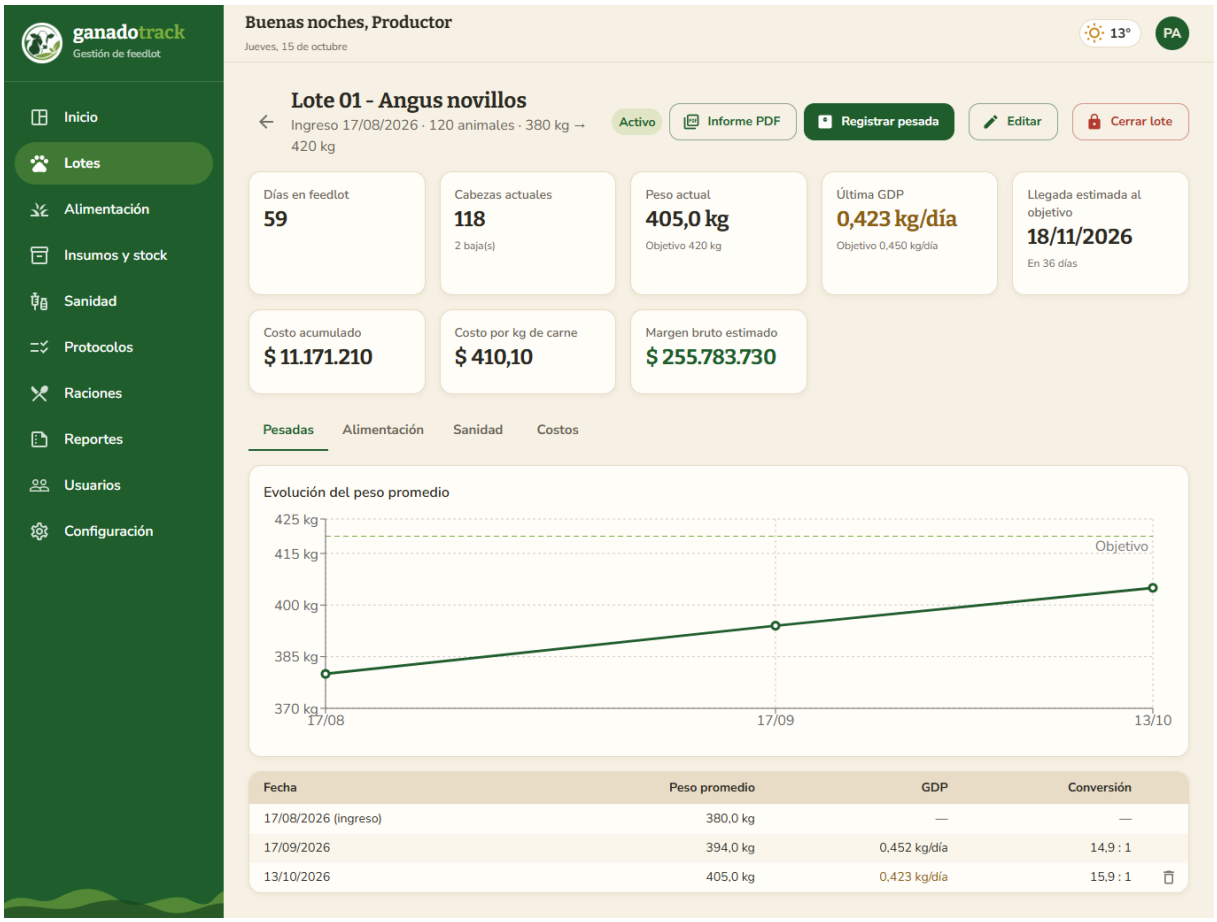



***Figura 34.*** *Detalle del lote*

**Registrar pesada:** botón *Registrar pesada* → fecha y peso promedio del lote.

- El sistema calcula la GDP desde la pesada anterior y actualiza la fecha estimada de llegada al objetivo.
- Si la GDP es menor al objetivo, genera una alerta.
- Si el lote llegó al peso objetivo, sugiere cerrarlo.

Antes de eliminar algo, el sistema siempre pide confirmación:

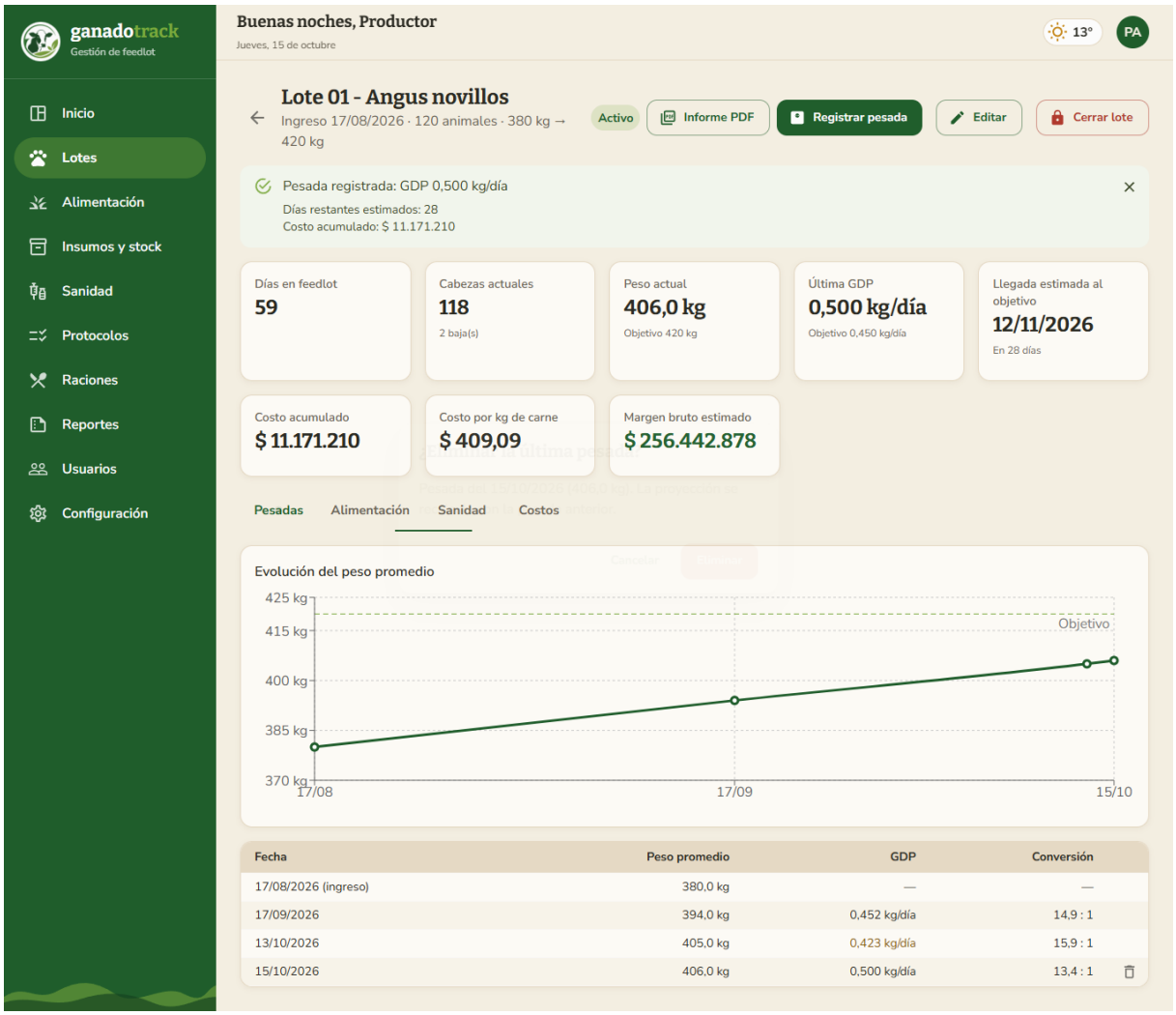



***Figura 35.*** *Confirmar eliminación*

**Precio de venta:** con *Editar* se carga el precio de venta por kg al gancho, que se usa para calcular el margen.

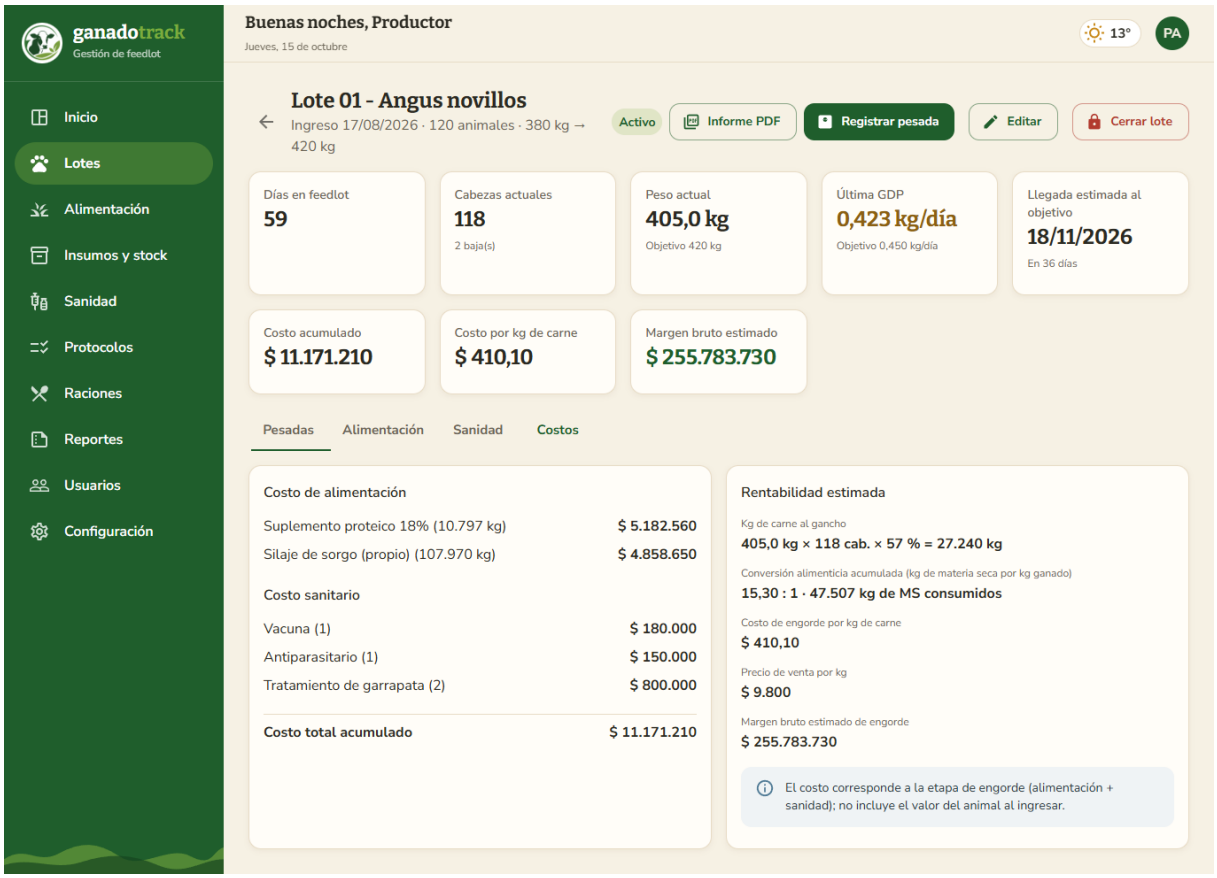



***Figura 36.*** *Costos*

**Cerrar lote:** *Cerrar lote* → fecha de venta/faena, peso final y precio.

1. Primero se muestra el **resumen final**.
2. Recién al presionar *Confirmar cierre* el lote pasa al historial.

Un lote cerrado no admite más registros y su botón pasa a **Informe de cierre**.

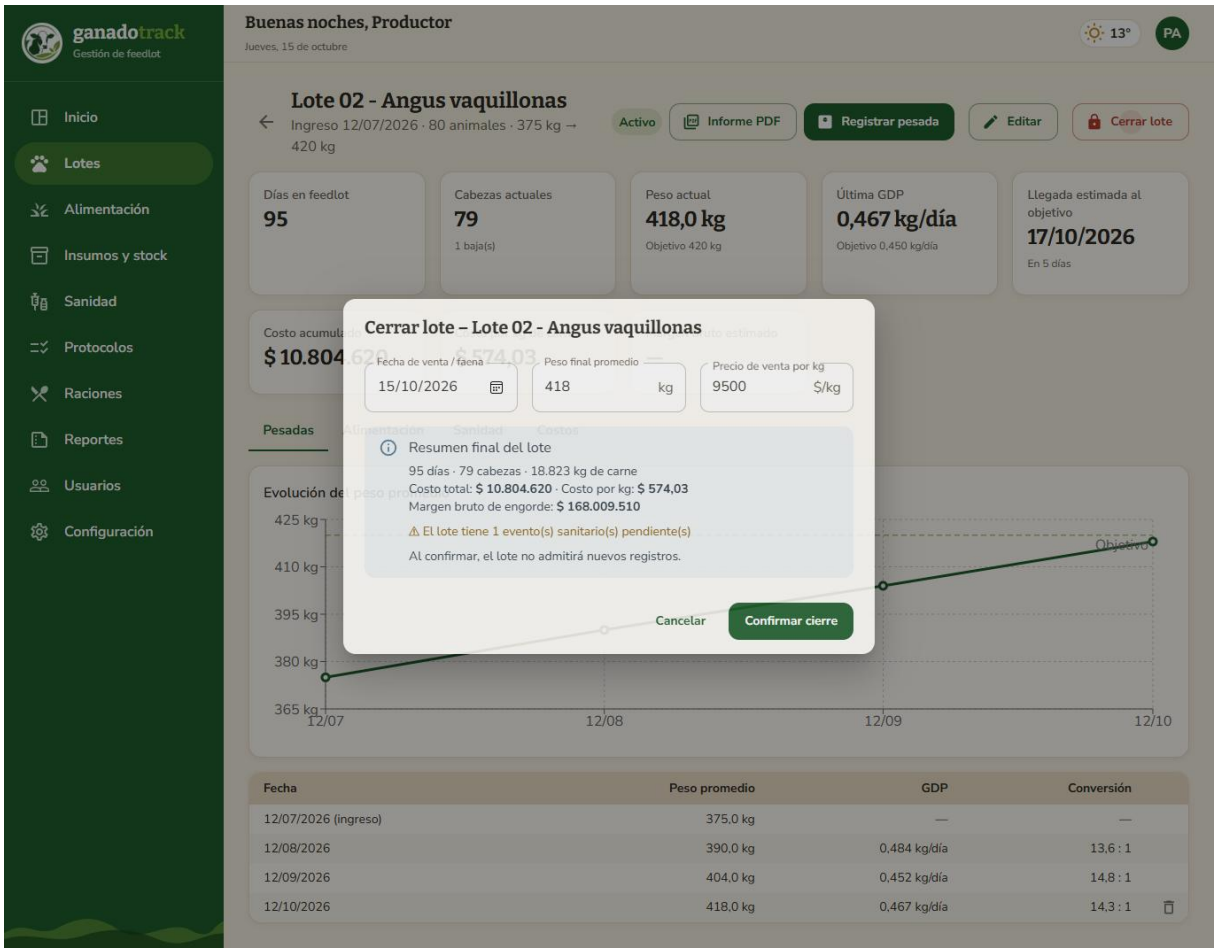



***Figura 37.*** *Cierre de lote*

> ***Cómo se calcula el costo por kg:*** *costo total (alimentación + sanidad) ÷ (peso promedio × cabezas actuales × 57 %). Es el* ***costo de engorde****: no incluye el valor del animal al ingresar al feedlot.*

## 4.4 Insumos y stock

Cada insumo muestra:

- El stock actual (el silaje en toneladas), el mínimo y el precio vigente.
- **Hasta cuándo alcanza:** "Alcanza para N días, hasta dd/mm", según el consumo previsto por las raciones (o el consumo real de las últimas dos semanas si no hay raciones). El chip se pone rojo si alcanza para menos de la cobertura mínima.
- El **gráfico de stock proyectado a 30 días**, con la línea punteada del mínimo.
- **Comprar N kg:** la compra sugerida para cubrir las próximas dos semanas y terminar con el stock mínimo. Abre el registro de compra con esa cantidad precargada.

Acciones:

- **Registrar compra** (suplemento) o **Cargar silaje** (producción propia): cantidad en kg y precio o costo total. El stock aumenta y el precio por kg se actualiza.
- **Configurar:** stock mínimo, precio por kg y **% de materia seca** (para la conversión alimenticia).
- **Corregir una compra:** el lápiz ✎ del historial permite corregir la cantidad o el precio. El stock se ajusta solo; si el insumo ya se consumió, no se permite dejar el stock negativo.

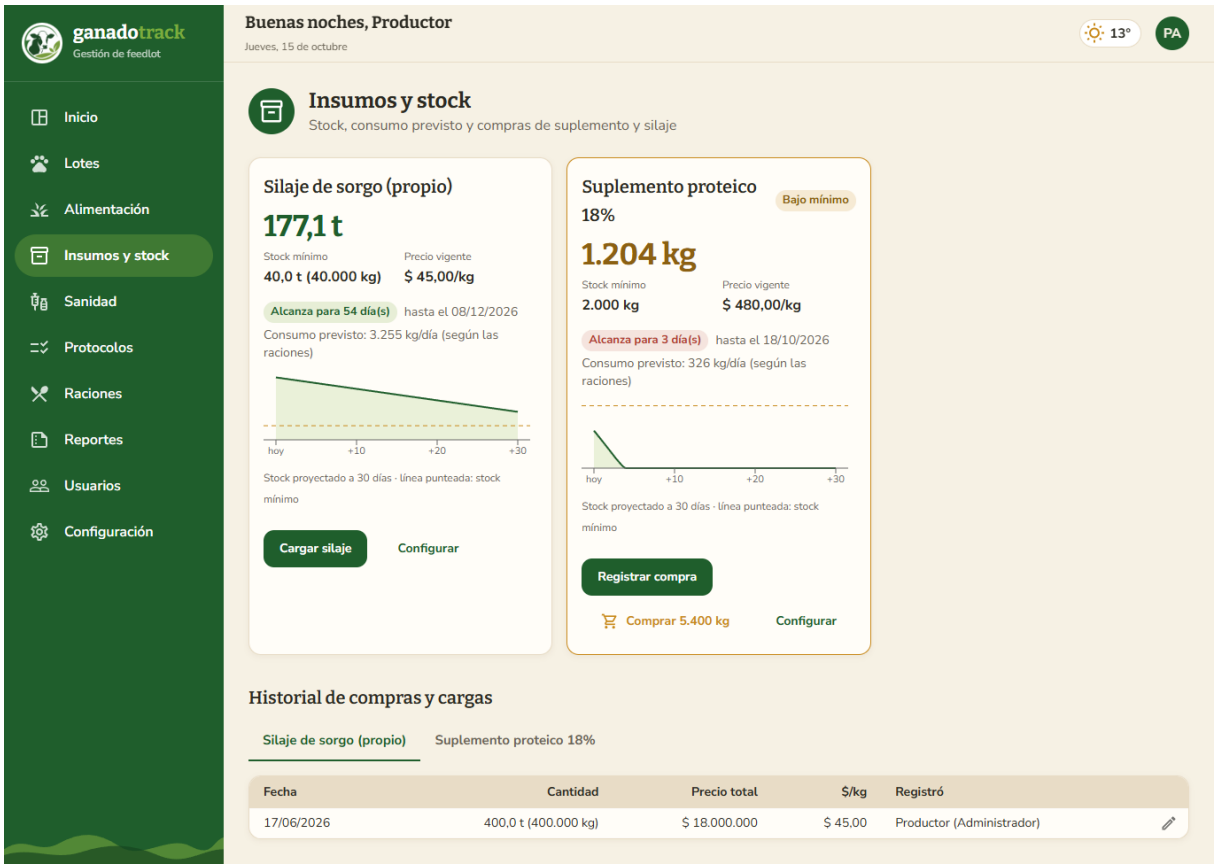



*Figura 38. Insumos*

## 4.5 Reportes

Informes listos para imprimir o compartir:

- **Informe de lote (PDF):** elegir el lote. Incluye indicadores, pesadas con gráfico, alimentación, sanidad, bajas, costos y margen. Para un lote cerrado es el informe de cierre.
- **Reporte mensual (PDF):** elegir el mes. Incluye costos por lote e insumo, eventos sanitarios, bajas y stock.
- **Planilla para el contador (Excel):** elegir el rango de fechas. Tiene una hoja por tipo de dato: lotes, alimentación, compras, sanidad, pesadas y bajas.

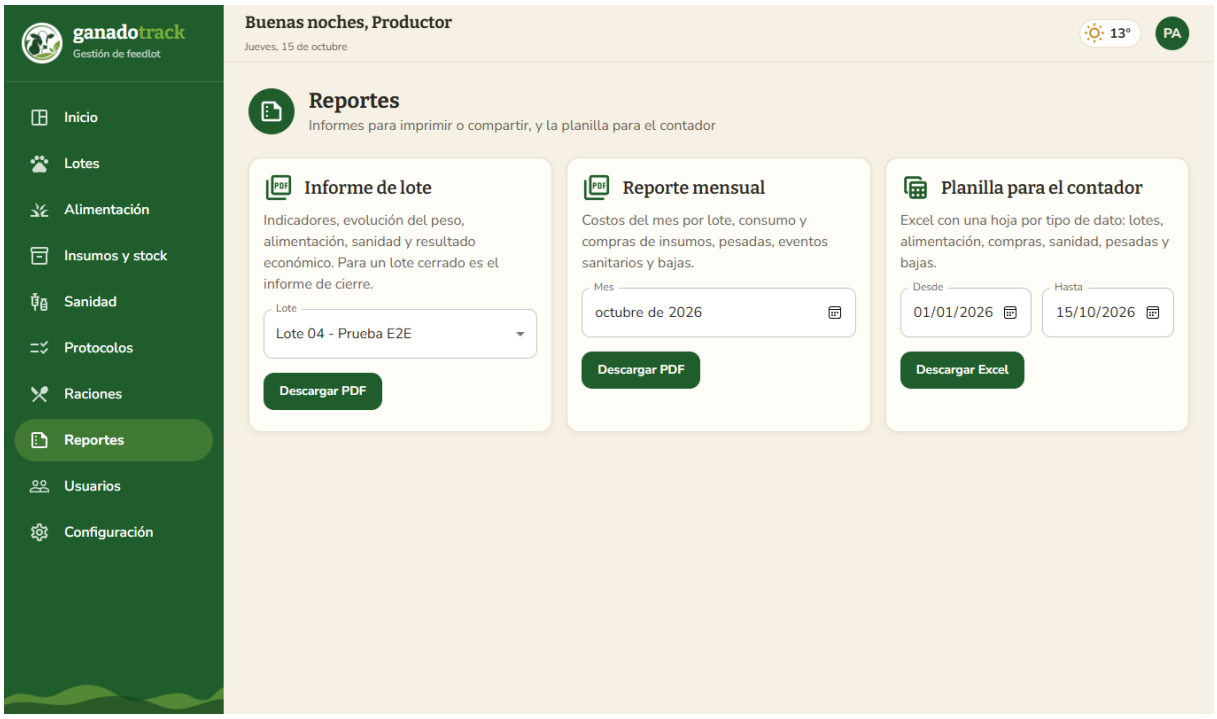



*Figura 39. Reportes*

## 4.6 Usuarios

Desde esta pantalla el administrador puede:

- Dar de alta usuarios con su rol.
- Cambiar el rol de un usuario.
- Desactivar un usuario: pierde el acceso de inmediato.
- Cambiar la contraseña de un usuario.

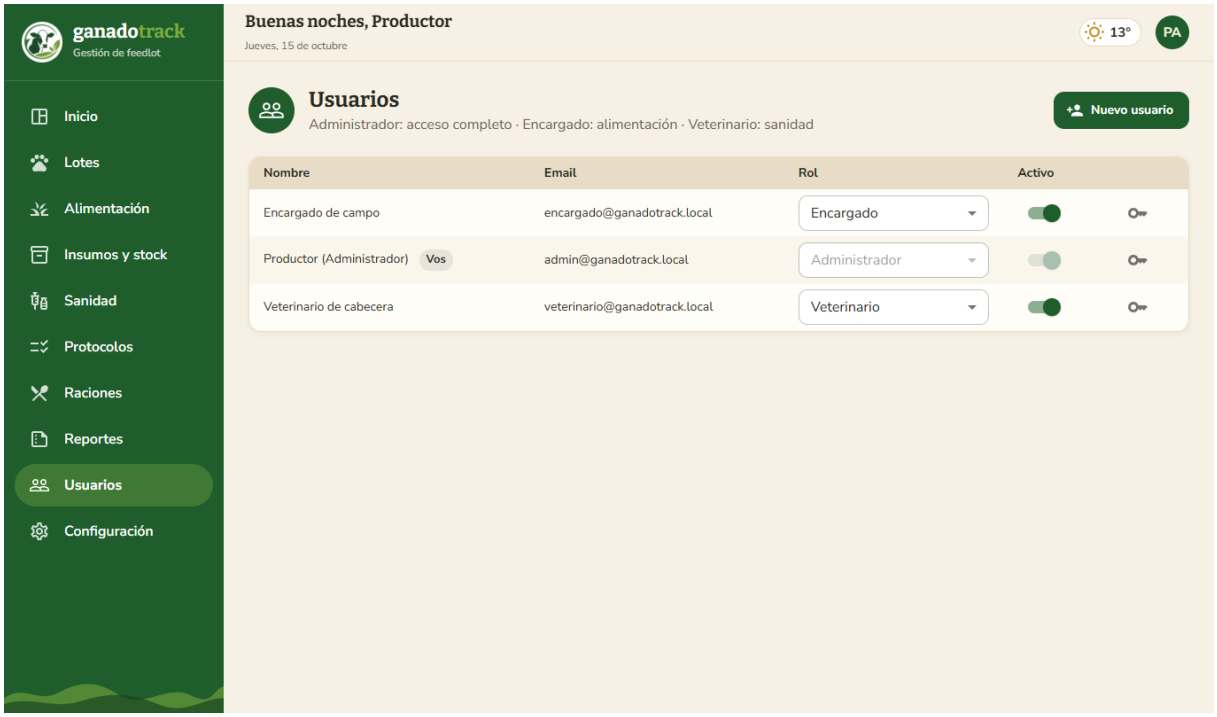



*Figura 40. Usuarios*

### 4.7 Configuración

- **Ubicación del establecimiento:**
  1. Escribir la localidad.
  2. Presionar **Buscar**.
  3. Elegir el resultado correcto.

  Se usa para el clima y los riesgos.
- **Parámetros de cálculo:**
  - Rendimiento al gancho (0,57).
  - Días de anticipación de las alertas sanitarias (7).
  - Días mínimos que debe alcanzar el stock (7).
  - Días de consumo que cubre la compra sugerida (14).

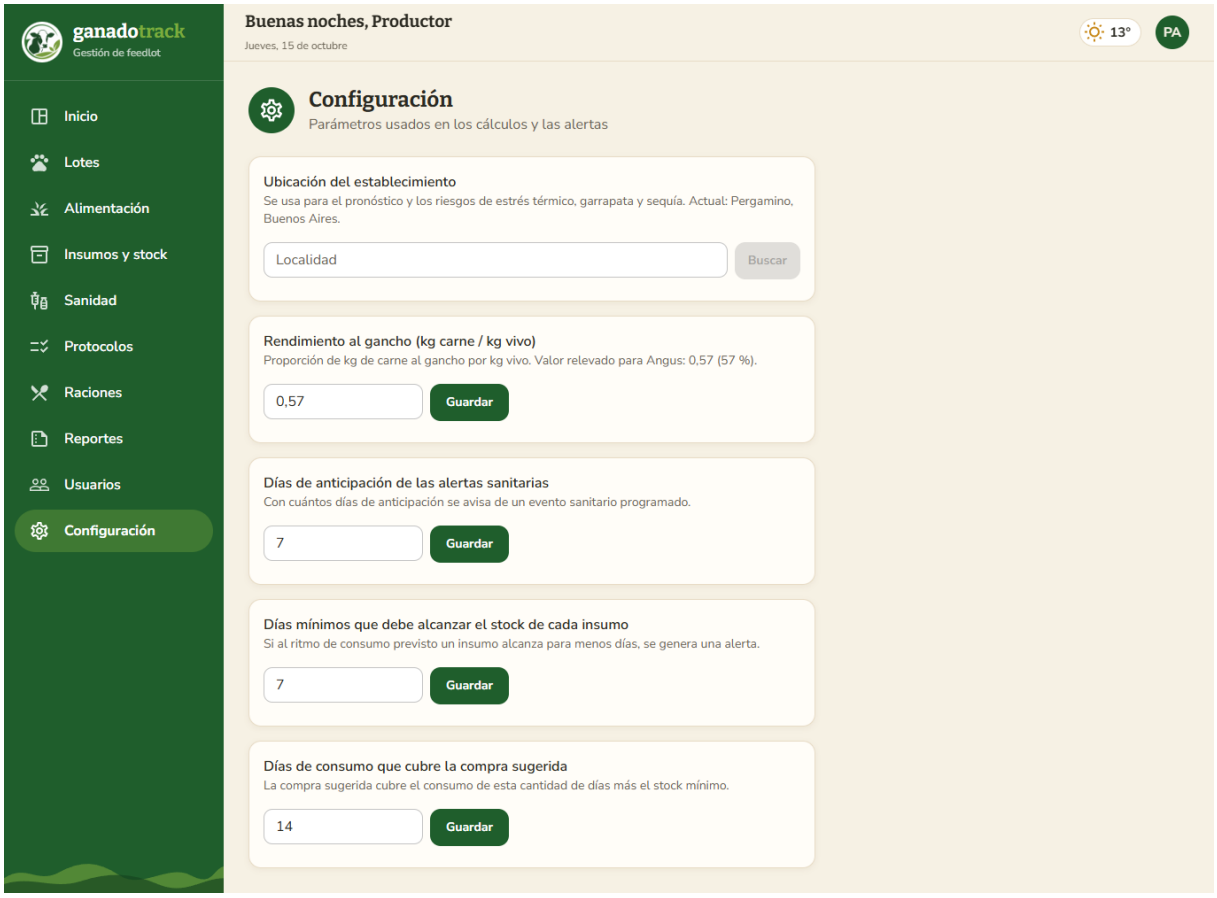



***Figura 41.*** *Configuración*

## 5. Mi perfil

Disponible para todos los roles desde el menú de usuario → **Mi perfil**:

- **Mis datos:** cambiar el nombre y presionar **Guardar nombre**.
- **Cambiar contraseña:**
  1. Escribir la contraseña actual.
  2. Escribir la nueva (al menos 8 caracteres) y repetirla.
  3. Presionar **Cambiar contraseña**.

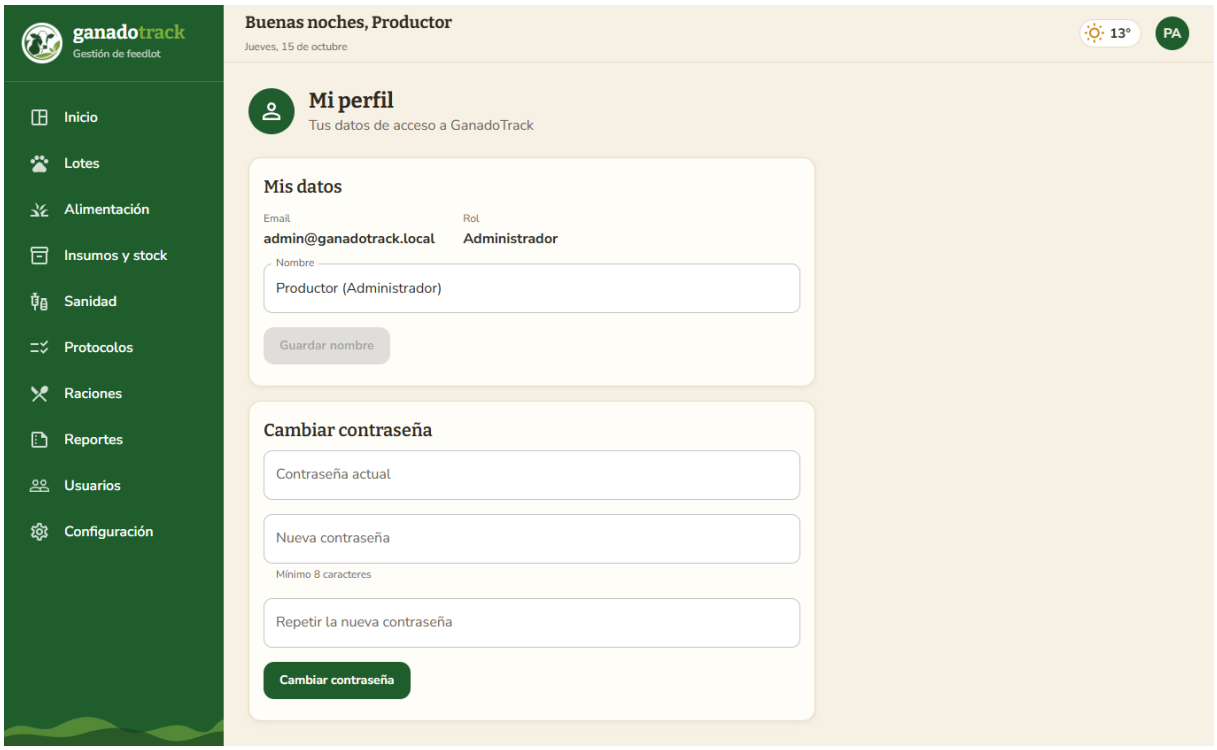



***Figura 42.*** *Mi perfil*

## 6. Instalar la app en el celular

Instalada, GanadoTrack abre como cualquier aplicación, con su ícono en la pantalla de inicio y sin la barra del navegador.

- **Android (Chrome o Edge):** ingresar a GanadoTrack, abrir el menú de usuario y tocar **Instalar la app**. Si no aparece, usar el menú ⋮ del navegador → *Instalar aplicación* o *Agregar a la pantalla principal*.
- **iPhone (Safari):** botón *Compartir* → *Agregar a pantalla de inicio*.
- **PC (Chrome o Edge):** ícono de instalación en la barra de direcciones, o menú de usuario → **Instalar la app**.

Las actualizaciones se instalan solas la próxima vez que se abre la app con señal.

## 7. Uso en el celular

Todas las pantallas se adaptan al celular. En el caso del administrador y del veterinario, el menú se abre con el botón ☰.

**Dashboard en el celular** **Menú**

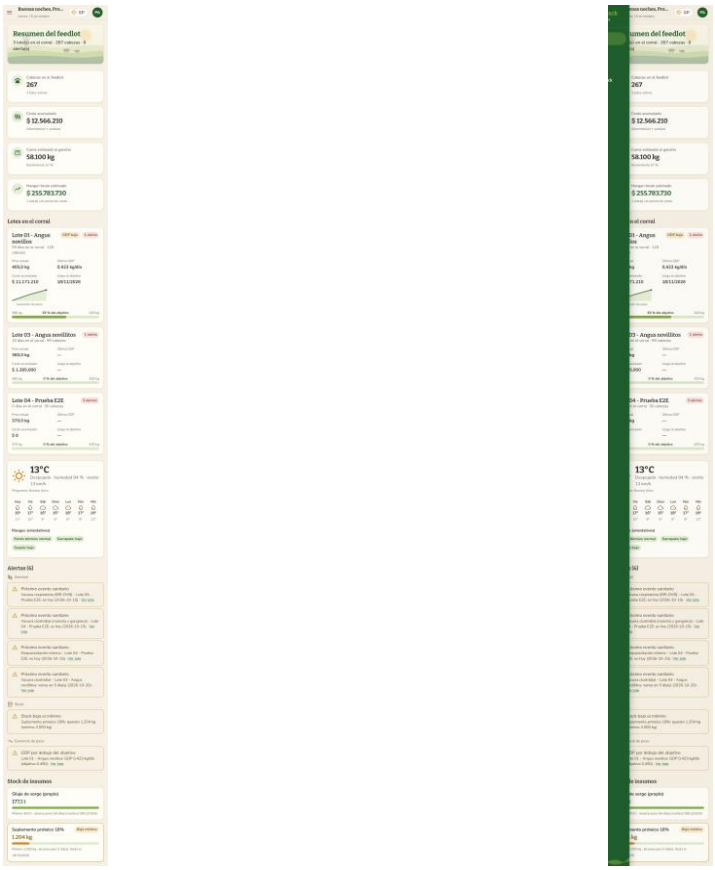


***Figura 43.*** *Dashboard móvil · Menú móvil*

## 8. Problemas frecuentes

***Tabla 100.*** *Problemas frecuentes*

| Situación | Qué hacer |
|---|---|
| La primera vez del día la app tarda en responder (unos 50 segundos) | Es normal en el servidor gratuito: la API "despierta" después de un rato sin uso. Esperar y volver a intentar |
| Aparece "Sin conexión" | El encargado puede seguir cargando (sección 2.2). Las demás pantallas necesitan señal |
| El panel de clima dice que el servicio no responde | El servicio externo de clima no está disponible. El resto del sistema funciona normalmente; volver a mirar más tarde |
| El clima pide configurar la ubicación | El administrador debe cargar la localidad en *Configuración* |
| Se escribe una dirección que no existe | Aparece la página "Esta tranquera está cerrada", con un botón para volver al inicio |
| Se olvidó la contraseña | El administrador puede asignar una nueva desde *Usuarios* |

# Anexo IV: Material para la validación con el productor

La validación con el productor no se completó antes de la entrega (ver sección 18.5). Para hacerla se prepararon tres videos de demostración generados con Playwright (el encargado en el celular, el productor y el veterinario), un acceso de prueba a la aplicación publicada y el siguiente cuestionario, que se enviará por WhatsApp. Sus respuestas se usarán para ajustar los valores de referencia del sistema.

## Cuestionario (8 preguntas)

1. **Ganancia de peso:** en el sistema puse que los animales entran al feedlot con unos **380 kg** y salen con **420 kg** en **90 a 100 días** (una ganancia de 0,4 kg por día). ¿Es así, o los kilos de entrada/salida son otros? ¿Cuánto esperás que ganen por día?
2. **Ración:** ¿cuántos kilos de suplemento y de silaje le dan por animal por día, más o menos? (en la demo puse 1,5 kg de suplemento y 15 kg de silaje).
3. **Precios:** ¿cuánto te cuesta hoy el kilo de suplemento? ¿Y cuánto calculás que te sale producir el silaje (por tonelada)?
4. **Rendimiento al gancho:** ¿el 57 % sigue siendo el valor que usás para el Angus?
5. **Plan sanitario:** el sistema arma solo el calendario de cada tropa (vacuna clostridial y desparasitación al ingresar, refuerzo a los 21 días y control de garrapata cada 21 días). ¿Le cambiarías o agregarías algo? ¿Tu veterinario lo vería útil?
6. **Carga en un toque y sin señal:** ¿le resultaría fácil al encargado cargar la comida con un solo toque? ¿Hay lugares del campo sin señal donde le serviría que funcione igual?
7. **Lo más útil y lo que falta:** ¿qué es lo que más te sirve de lo que viste? ¿Qué le agregarías o cambiarías?
8. **Nota:** del 1 al 10, ¿qué tan útil te resultaría para manejar el feedlot? (en la entrevista le pusiste 8 a la idea).

# Anexo V: Correcciones respecto de la 1ª entrega

Revisión del documento de la 1ª entrega. Las correcciones ya están aplicadas en las secciones de este documento; el detalle de cada una está en `docs/entrega2/00-correcciones.md` del repositorio.

***Tabla 101.*** *Correcciones aplicadas a la 1ª entrega*

| # | Problema detectado | Corrección aplicada | Sección |
|---|---|---|---|
| 1 | La numeración de los casos de uso de la lista no coincidía con la del diagrama | Lista unificada CU01–CU22, igual en el texto, el diagrama y el sistema | 13 |
| 2 | La ficha de CU06 estaba duplicada y había fichas de solo 3 de los 12 casos | Fichas completas de los 22 casos de uso | 13.4 |
| 3 | Errores en el diagrama de casos de uso: el encargado accedía a alertas, un «include» mal nombrado, un «extend» confuso y faltaban casos | Diagrama rehecho, con los actores Sistema y Servicio de clima | 13.3 |
| 4 | En la ficha de pesada, "el peso supera el objetivo" figuraba como excepción | Pasó a flujo alternativo; se agregaron las excepciones reales (fecha y peso inválidos) | 13.4 (CU07) |
| 5 | La entrevista decía 8 bloques pero eran 9 (0 a 8), y el Bloque 0 no estaba transcripto | Texto corregido y Bloque 0 agregado | 9.3 y Anexo I |
| 6 | Redacción confusa en el primer hallazgo | Redacción corregida | 9.4 |
| 7 | El título de la sección 16 no coincidía con el índice | Título unificado: "Seguimiento del proyecto (GitHub Projects)" | 19 |
| 8 | Las fórmulas de costo por kg y margen usaban la cantidad de ingreso (sin restar las bajas) y no aclaraban que el costo es solo de engorde | Cabezas actuales = ingreso − bajas; "costo de engorde" y "margen de engorde"; límite del costo del animal al ingreso | 10 (RF06) y 7 |
| 9 | La GDP objetivo no tenía dónde configurarse | Campo GDP objetivo en el lote | 10 (RF01) |
| 10 | Se controlaba stock de suplemento aunque el productor dijo no stockearlo | Justificación: la compra semanal genera un stock de corto plazo | 9.4 |
| 11 | El silaje se registraba en toneladas y la alimentación en kg | Todo en kg; el silaje se muestra además en toneladas | 10 (RF03) |
| 12 | El pedido de cargar gastos de servicios no figuraba en ningún lado | Agregado a Límites y a Trabajo futuro | 7 y 21 |
| 13 | Los importes no indicaban la moneda | Todos los importes en ARS | 8 y 9.2 |
| 14 | El diagrama entidad-relación estaba incompleto (campos del lote, auditoría, estado de los eventos, parámetros) | Modelo de datos rehecho con diccionario de datos | 15 |